Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

ASAM MDF

Measurement Data Format

Programmers Guide

Version 4.2.0

Date: 2019-09-30

Base Standard

© by ASAM e.V., 2019

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Disclaimer

This document is the copyrighted property of ASAM e.V.
Any use is limited to the scope described in the license terms. The license terms can be viewed at www.asam.net/license

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table of Contents

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Foreword

MDF stands for Measurement Data Format. It is a binary file format to store measured or calculated data for post-measurement processing or long-term conservation. Common sources of the data to be stored are sensors, ECUs or bus monitoring systems. In addition to the plain measurement data, MDF also contains descriptive and customizable meta data within the same file.

The format is organized in loosely coupled binary blocks to ensure high performance reading and writing. The measurement data is stored channel-oriented and organized in records. MDF supports non-equidistant sampling rates and multiple rates per file. Master channels are used for synchronization which can be time, angle, distance or simply index related.

MDF allows storage of raw measurement values and corresponding conversion formulas. It supports special data types and information particularly required in the automotive area, and is closely related to other ASAM standards, e.g. ASAM MCD-2 MC (ASAP2) [1].

The format definition is published in the base standard which describes the following content:

- Version history
- Version handling and compatibility rules
- MDF block structure
- General rules
- List of all binary block types and their contents
- MDF meta data format (XML)

The description how to store information for certain use cases is published in different associated standards. Such use cases are:

- **Naming of Channels and Channel Groups [10]**
  This associated standard describes how to set the different names for channels and channel groups for common use cases in order to provide important information to the user and to achieve a unique identification of the channels.
- **Bus Logging [7]**
  This associated standard describes how to store the traffic of common bus systems.
- **Measurement Environment [9]**
  This associated standard describes how to store information about the measurement environment.
- **Classification Results [8]**
  This associated standard describes how to store classification results.

Binary example MDF files are contained in the deliverable of the respective standard or may be requested from the ASAM MDF working group (mdfproject@asam.net).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

1 Introduction

1.1 Overview

This specification describes the ASAM MDF (Measurement Data Format) Version 4.2.
This specification is intended for implementers of ASAM MDF Version 4.2. It shall be used as a technical reference with examples how to read and write MDF files.
This specification consists of this document and a number of XML schema files.

1.2 Conventions

The ASAM MDF specification comprises a variety of aspects around storing and retrieving measurement data and follows some conventions:

- The specification is divided into several chapters. Specification means the whole set of chapters.
- A chapter describes one specific aspect of the ASAM MDF standard; it is identified by a level-1 heading and is the upmost hierarchy within the specification.
- Each hierarchy level below a chapter is called a section. Each section is uniquely numbered and can be referenced from any location within the specification.
- The specification uses two kind of text fonts:
  - proportional font: used for descriptive text
  - monospace font: used for code (XML, XSD) and code examples

1.3 Motivation

Based on the successful MDF 3 file format, this standard aims to be a general purpose file format for all kinds of measured data. Coming from an automotive background, this standard is suited for all phases of the automotive development life cycle. Using the standard to store measured data provides access to a large range of tools that are capable of accessing the data stored in MDF4 files.

1.4 Scope

The file format defined by this standard is suitable for storing measured and calculated data for post-measurement processing and analysis as well as long-term storage. In principle, there is no limit to the type of data that can be stored inside an MDF4 file but coming from an automotive background, MDF4 is best suited for storing generalized time-series data and provides out of the box mappings for the most common data sources in the automotive industry (sensors, ECU data, bus monitoring data).

MDF4 files contain the meta-information to decode and interpret the data contained in them. The standard also provides ample configuration points to store additional meta-data that is required for understanding the data but unique to the situation where the file is used. This, combined with it being an ASAM standard makes MDF4 a good choice as data-exchange and long-term storage format.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

1.5 File Extension

The MDF file extension starts always with ‘mf’ for measure file and followed by the major version number. Other sub-versioning is not covered by the extension.

Example: ‘my_measure_file.mf4’, where ‘.mf4’ indicates the format MDF V4.x.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

2 Relations to Other Standards

2.1 Backward Compatibility to Earlier Releases

2.1.1 MDF Versions

The MDF format has been extended several times since its creation in 1991. This chapter provides a brief version history of the MDF format and instructions how to handle future updates.

2.1.2 Version History

Table 1 Major Revisions

Year	Version	Description
1991	2.03	First official release of MDF
1996	2.11	New conversion type (CCBLOCK): MCD-2 MC text.
2000	2.12	New conversion type (CCBLOCK): MCD-2 MC text table. New field (CNBLOCK): "Long Signal Name".
2000	2.13	New extension type (CEBLOCK): Vector CAN block.
2001	2.15	New data types (CNBLOCK): "String" and "Byte Array". New conversion types (CCBLOCK): "Date" and "Time".
2002	3.00	New conversion type (CCBLOCK): MCD-2 MC text range table. New fields (CNBLOCK): "Display Name" and "Additional Byte Offset".
2005	3.01	N-dimensional dependency in CDBLOCK.
2006	3.10	New channel data types (CNBLOCK) to define a specific Byte order for integer and floating-point signal values (can differ from default Byte order). Allow crossing of Byte boundaries for Integer signals with bit offset > 0.
2008	3.20	Exact start time and time quality information (HDBLOCK.) LINK changed from signed to unsigned integer (UINT32) (max. file size of an MDF file thus extended from (approx.) 2 GB to 4 GB)
2009	3.30	Sample reduction for a channel group (SRBLOCK) Code page for all strings in MDF file (except string signals)
2009	4.00	ASAM standardization of completely revised MDF 4.0 format with new block structures. Incompatible to previous MDF format versions except of IDBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

2012	4.10	Compression of data blocks with new block types DZBLOCK and HLBLOCK. New channel types in CNBLOCK for virtual data and maximum length signal data (MLSD). New links in CNBLOCK for attachment list and default X signal. New flags in CNBLOCK for monotonous values and bus event. New flags in CGBLOCK for bus logging. New member in CGBLOCK for path separator. New types for CABLOCK for classification results and interval axes. New generic tags in <common_properties> to model lists. First release of associated standards for bus logging, channel identification and measurement environment.
2014	4.11	Identification and flags for "unfinalized" MDF in IDBLOCK (version independent feature) Extension of version pattern, e.g. used in <tool_version> Additional attribute "average" for <raster> tag.
2019	4.20	New block types DV-/DI-/RV-/RI- and LDBLOCK to separate storage of data from storage of invalidation bits and to support column-oriented storage for faster reading. Extension of CGBLOCK to reference a "master" channel group for column-oriented storage and for convenient addition of channels to an existing channel group. Extension of DLBLOCK to store first master channel value per data block for faster seeking. Extension of EVBLOCK and new flag in CGBLOCK to support event signals as alternative to linked list of EVBLOCKs (for faster reading of a large number of events). New flag for variable length (VLSD) channels to indicate that SDBLOCK data is stream-like which allows faster reading. New data type "complex number" in CNBLOCK. Support of half precision floating-point channel data type in CNBLOCK. New conversion rule type "bitfield text table" in CCBLOCK. New flag in SRBLOCK to use dominant invalidation bits in reduction. New flag in CABLOCK to mark a standard axis. Removed requirement for master channel values to be strictly monotonously increasing: simple monotony is sufficient[1]. Best practice for unique identification of related measurement files. Best practice for documentation of lost time synchronization.

2.1.3 Version Handling

The MDF version number consists of three digits, a major version number, a minor version number and a revision number. Normally only the major and minor version numbers are stated, e.g. V4.0 with major version number "4" and minor version number "0". In the IDBLOCK, an additional revision number is appended. For example, if the version string reads "3.10", this must be interpreted as V3.1, revision "0" (and not as minor version number

[1] A non-strictly monotonously increasing master channel is an indicator for an error in the measurement setup in most cases though.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

"10"). The restriction to three digits implies that both minor version number and revision number must never exceed nine.

Each change in the MDF file format must result in a higher version number, i.e. the three-digit-number consisting of major, minor and revision number must be larger than that of the previous version (e.g. 410 > 401 for versions 4.10 and 4.01).

Here are some general guidelines when to increase which of the numbers:

Table 2 Version handling

Element	Description
Major version number	Changes in the MDF file format that may result in a misinterpretation of the data when evaluated by some older tool require a change of the major version number. A tool that evaluates an MDF file that has a higher major version number than what is supported by the tool should reject reading the file and generate a warning or error message.
Minor version number	Changes in the MDF file format that may not result in a misinterpretation of the data when evaluated by some older tool require only a change of the minor version number. Still the older tool may miss new features/information stored in the new MDF version. Normally it would simply ignore them and generate a warning message.
Revision number	Changes in the MDF file format that do not affect the interpretation of an older tool may be done by changing the revision number only.

2.1.4 Rules to Ensure MDF Compatibility between Versions

To ensure a high degree of compatibility with future versions, new link entries must only be appended at the end of the link section (see section General Block Setup). Adding new data fields or bit flags, reserved Bytes or bit flags can be used if sufficient; otherwise the fields must be appended at the end of the data section. The default value of a new data field or bit flag must be zero.

Writing an MDF file, reserved Bytes must be filled with zeros and reserved bit flags (i.e. which are currently not defined) must not be set.

Furthermore, blocks in an MDF file must not be longer than applies for the MDF version of the written file. They may be shorter, but not shorter than applies for the major MDF version number (e.g. a block in an MDF 4.x file must not be shorter than expected for this block type in MDF 4.0). This allows increasing the minor version number of an MDF file without having to re-write all contained blocks with a possibly larger length, but it disallows to simply shorten the link or data section if members at the end are zero.

This implies the following consequences for the reading tool:

A tool must first check the MDF version number of the MDF file to be read. If the MDF version of the file ("file version") has a major MDF version number larger than the MDF version supported by the tool ("tool version"), the file contains additional information which is essential for interpretation of the data, but which is not known in the tool version. Hence, the tool should reject reading the MDF file (see also Table 2).

If the major file version number is smaller than the major tool version number, there might have been a compatibility break (like between MDF 3.x and MDF 4.x) and the tool may only

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

be able to read the MDF file if it also supports the older MDF version. For future increments of the major MDF version number, backward compatibility possibly can be maintained and a tool which supports the new MDF version might be able to read the old MDF version despite the lower major version number, but there is no guarantee for this.

If the major file version number and the major tool version number are equal, the file generally should be readable by the tool although features introduced in a newer minor version might be missed.

Note: For the following recommendations we assume that the major file version number is equal to the major tool version number.

If a tool detects a block that has a smaller block size (i.e. a smaller number of links or a smaller data section size) than expected for the tool version, the additional links or data fields are assumed to contain their default value (i.e. zero). This may be the case, if the tool version is higher than the file version. However, if the block is smaller than the minimum size expected for this block type in this major version number, this should be considered an error.

When reading a block that is larger than expected, it contains additional links or data fields which are not known in the tool version. If the file version is less than or equal to the tool version, this should be considered an error because the block is longer than applies for its MDF version. If the file version is higher than the tool version, the additional links and data fields can be ignored because they are not essential for interpretation of the data (otherwise the major version numbers should be different).

As best practice, additional links and data field values should be preserved when changing an MDF file, and additional links should be updated if the respective block is moved in the file, e.g. when sorting the file.

Similar to additional links and data fields, reserved Bytes and bit flags generally can be ignored when reading a block. Again, they should be preserved when changing the MDF file. The same also applies to unknown XML tags in the MDBLOCK (see XML Versioning).

Apart from unknown links, data fields or XML tags there also may be unknown values for an enumeration or unexpected block types for a link when reading an MDF file that has a higher version than the tool version. In this case, the respective block or feature should be ignored. This also depends on the importance and impact of the information carried by the enumeration or the link, i.e. whether it is required for interpretation of other data (like the data type of a channel) or only used for documentary information (like the cause of an event). For instance, in case of an unknown channel data type (cn_data_type), the complete channel should be ignored whereas an unknown cause of an event (ev_cause) still allows displaying the event. As best practice the tool should report a warning in this case.

Reading an MDF file that has a lower version than the tool version, reserved Bytes or bits of its MDF version should be zero (see above). If they are not, the tool may decide whether to ignore them, report an error or use them if they are valid in the MDF version currently supported by the tool. The same holds for XML tags, block types or enumeration values which are known in the tool version, but unknown in the file version.

The flow chart in Figure 1 summarizes some of the rules specified above.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

f	MDF version number read from file (e.g. 400)
a	MDF version number supported by reading tool (e.g. 400)
mj(x)	major version number of version x mj(x) = x div 100
BS(x)	block size expected by version x
minBS(x)	minimum bock size expected by version x minBS(x) = BS(mj(x) * 100) = BS((x div 100) * 100)
BSR	bock size read from file

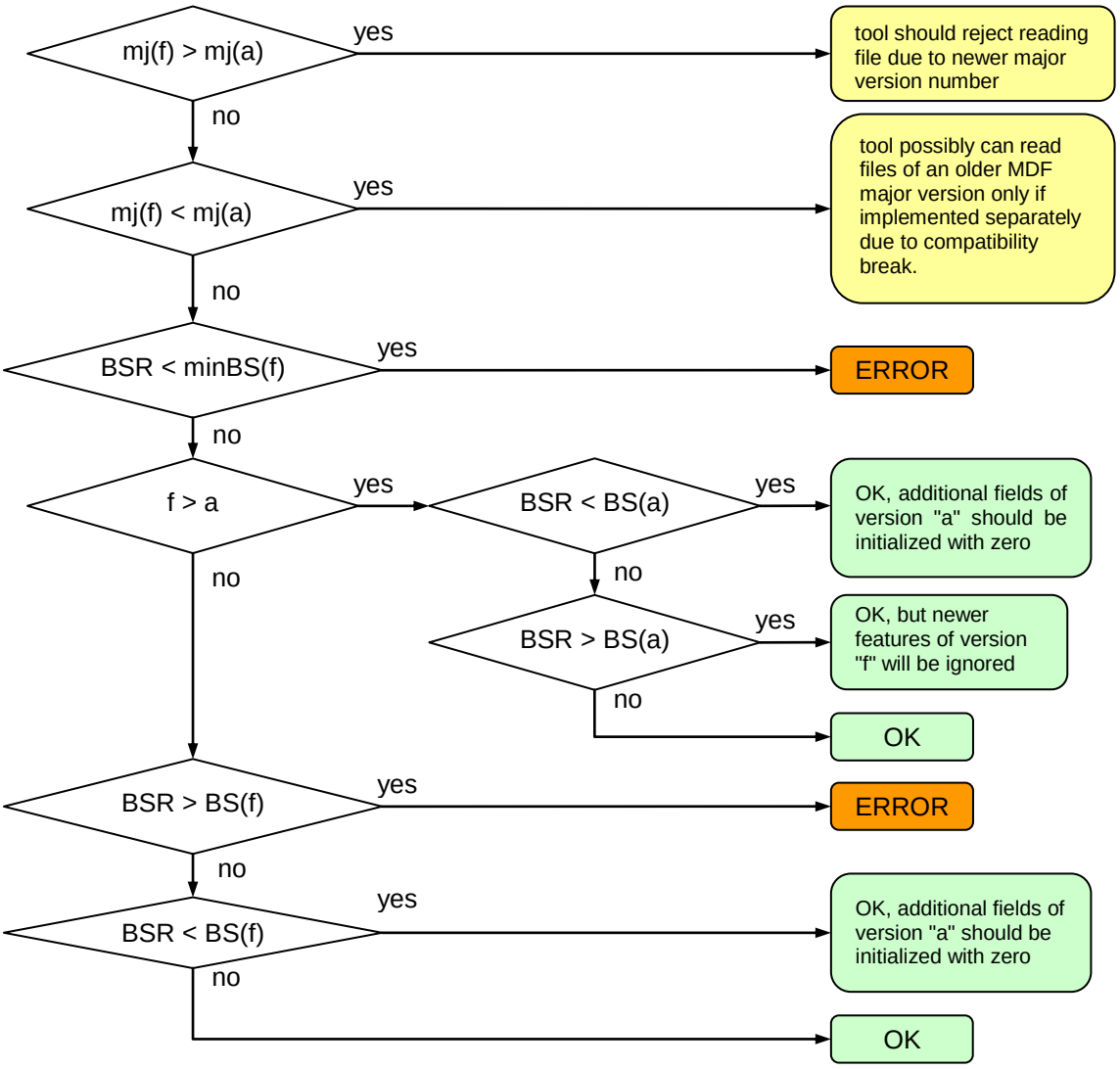


Figure 1 Logic when reading MDF blocks

2.2 Compliance to ASAM ODS Standard

If the MDF file follows certain rules, it is possible to directly use the MDF file as external component in an ODS data base or an ATF-XML component file. For details please refer to [6].

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

3 Introduction

Introduced as a file format to store measurement data for post-measurement processes, **M**easurement **D**ata **F**ormat (MDF) saw the light of the day in the early 90s. The basic concepts as binary, block-structured and channel-oriented format which is split into descriptive information and data matured in the last years form a company comprehensive agreement to a well-established de-facto standard in the automotive industry without any major changes.
The success of the format is founded in the compact description and fast access to the file content, independent of the file length and also in the basic concept that has been used almost unchanged since the beginning. This leads to high interest into the format, which is still increasing.

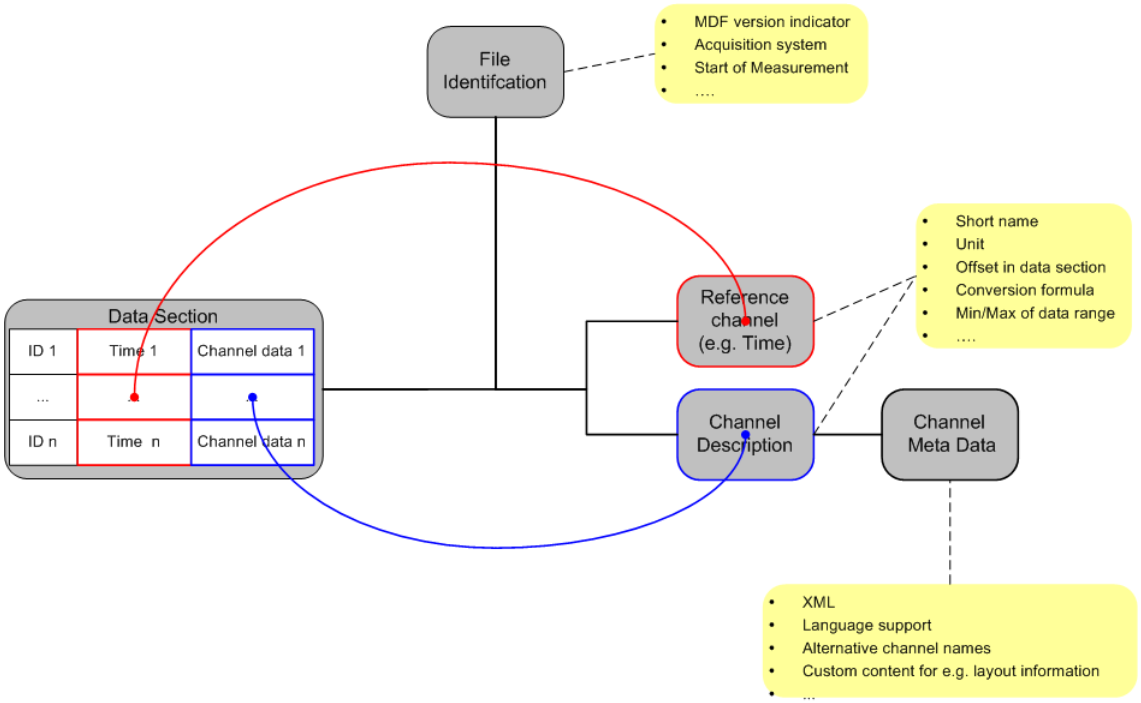


Figure 2 Base concept of MDF 4.x structure

The automotive market moved on in the last years and developed new features like Adaptive Cruise Control with the necessity of video/audio analysis capability, and navigation systems with GPS data visualization. This and the fact that the amount of data to process has become huge for some areas, e.g. for fuel cell development, has increased the demand for new requirements on MDF.

Pressure from the Asian market requires internationalization. Changes in national laws, regulations and existing standards need to be considered and request higher process reliability.

Additionally, the technology of measurement systems matured: today systems exist on the market which are not time related anymore but angle, index or distance based.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

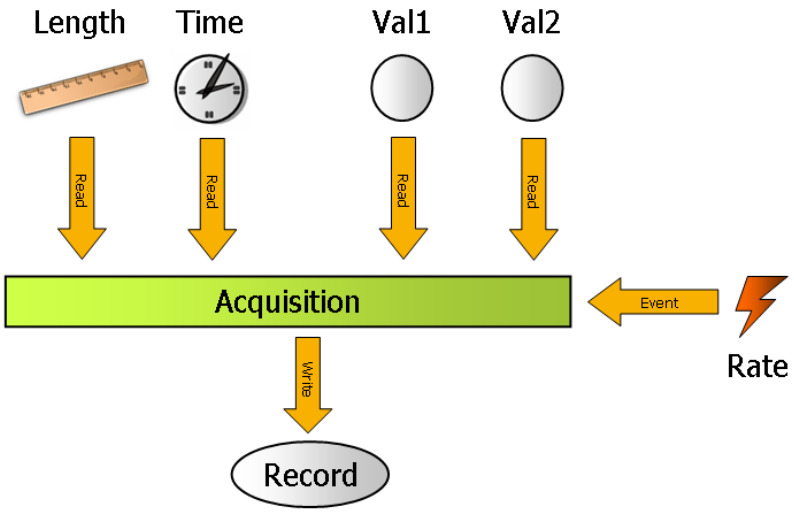


Figure 3 Basic Principle of Measurement

And at last the initiators of the original MDF noticed that some structures were unused or insufficient. Also, workarounds or company specific solutions were implemented which prevented an easy exchange between different tools. So, the initiators wanted to clean-up and improve the structures of MDF, e.g. for efficient storage of calibration data like CURVE or MAP (see MCD-2 MC [1]), and to open the format as a real exchange format.
To boost the comprehensive company agreement to an official standard the originators decided in 2007 to promote the MDF standard to an ASAM standard. Since the first release of ASAM MDF 4.0.0 in 2009, the standard is maintained and extended by the ASAM MDF working group. Also of interest is the alignment to existing and related standards in ASAM.

Note: The workgroup members tried to reuse as much of the concepts from the existing MDF V3.x specification as possible. But to fulfill all of those requirements mentioned, new data types and concepts had to be introduced. Thus MDF 4.x is closely related to the previous version but not backwards compatible anymore.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4 MDF General Block Format

An MDF file is composed of a series of blocks. Each block consists of a number of contiguous Bytes and can be seen as a record or structure of data fields. There are different types of blocks. Each single block is divided into three sections: header, links and corresponding data section. The link section contains pointers to other blocks given as absolute Byte positions within the file, starting at the beginning of the file. Thus, a normally tree-like hierarchy of blocks is formed (see 4.4.4 Linking of Blocks).

An MDF file always starts with the file identification block IDBLOCK. The file header block HDBLOCK follows at byte position 64. All other blocks are linked via LINKs and can be stored in arbitrary order in the file.

Usually a link always references a certain blocks type. However, for some links, several block types are possible (e.g. a text block TXBLOCK or a meta data block MDBLOCK). In this case, the type in the header of the referenced block must be examined.

4.1 Overview of Block Types Used

Table 3 Block type overview

Block Type	ID	Description	Purpose
IDBLOCK	ID	Identification block	Identification of the file as MDF file
HDBLOCK	HD	Header block	General description of the measurement file
MDBLOCK	MD	Metadata block	Container for an XML string of variable length
TXBLOCK	TX	Text block	Container for a plain string of variable length
FHBLOCK	FH	File history block	Change history information of the MDF file
CHBLOCK	CH	Channel hierarchy block	Definition of a logical structure/hierarchy for channels
ATBLOCK	AT	Attachment block	Container for binary data or a reference to an external file
EVBLOCK	EV	Event block	Description of an event
DGBLOCK	DG	Data group block	Description of data block that may refer to one or more channel groups
CGBLOCK	CG	Channel group block	Description of a channel group, i.e. channels which are always measured jointly
SIBLOCK	SI	Source information block	Specifies source information for a channel or for the acquisition of a channel group.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Block Type	ID	Description	Purpose
CNBLOCK	CN	Channel block	Description of a channel, i.e. information about the measured signal and how the signal values are stored.
CCBLOCK	CC	Channel conversion block	Description of a conversion formula for a channel
CABLOCK	CA	Channel array block	Description of an array dependency (N-dimensional matrix of channels with equal data type) including axes.
DIBLOCK	DI	Data invalidation data block	Container for invalidation data
DTBLOCK	DT	Data block	Container for data records with signal values (value + invalidation data)
DVBLOCK	DV	Data values block	Container for value records with signal values
SRBLOCK	SR	Sample reduction block	Description of reduced/condensed signal values which can be used for preview
RDBLOCK	RD	Reduction data block	Container for data record triples with result of a sample reduction (value + invalidation data)
RIBLOCK	RI	Reduction invalidation data block	Container for invalidation data for sample reduction
RVBLOCK	RV	Reduction values block	Container for value record triples with result of a sample reduction
SDBLOCK	SD	Signal data block	Container for signal values of variable length
DLBLOCK	DL	Data list block	Ordered list of links to (signal/reduction) data blocks if the data is spread over multiple blocks
LDBLOCK	LD	List data block	Ordered list of links to (signal/reduction) value / invalidation blocks.
DZBLOCK	DZ	Data zipped block	Container for zipped (compressed) data section of a DT-/SD-/RD-/DV-/DI-/RV-/RIBLOCK as replacement of such a block.
HLBLOCK	HL	Header of list block	Header information for linked list of data blocks

4.2 Definition of Data Types Used And Mapping to ASAM Data Types

ASAM has specified standardized data types ([3]). These data types show standardized names, always beginning with A_. Since older MDF specifications used an own designation for data types, also including semantic information like LINK, the MDF designations have not

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

been changed. Instead, the following table describes the relationship between the ASAM data types and the MDF data types. So a one-to-one mapping is easily possible, if needed. For all tables later on, the Type column will denote the MDF data type.

Table 4 Used data types and mapping to ASAM data types

ASAM Data Type	MDF Data Type	Format
A_UINT8	UINT8	8-bit unsigned integer
A_UINT8	BYTE	same as UINT8, but only serves for plain data or reserved Bytes
A_UINT8	CHAR	same as UINT8 but each byte representing a character encoded in standard ISO-8859-1 (Latin character set)
A_UINT16	UINT16	16-bit unsigned integer
A_INT16	INT16	16-bit signed integer
A_UINT32	UINT32	32-bit unsigned integer
A_INT32	INT32	32-bit signed integer
A_UINT64	UINT64	64-bit unsigned integer
A_INT64	INT64	64-bit signed integer
A_FLOAT64	REAL	Floating-point compliant with IEEE 754, double precision (64 bits) (see [13]) An infinite value (e.g. for tabular ranges in conversion rule) can be expressed using the NaNs INFINITY resp. –INFINITY
A_INT64	LINK	64-bit signed integer, used as byte position within the file. If a LINK is NIL (corresponds to 0), this means the LINK cannot be de-referenced. A link must be a multiple of 8.

4.3 General Block Setup

Each block is divided into three sections: header, links and the corresponding data.
Exception: for compatibility to older formats of MDF the IDBLOCK is using the old format (only data part with fixed length).

Table 5 General Block Setup

Block
Header
Links
Data

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.3.1 Header section

Common header for all blocks (except for IDBLOCK). The header section contains information about the type, the length and the number of links in the attached link section of a block.
The header section has a fixed length of 24 Bytes.

Table 6 Header section

Name	Type	Count	Description
Header section			
id	CHAR	4	ID in format: "##[Blocktype ID]" e.g. "##DG" for a data group block
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block in Bytes
link_count	UINT64	1	Number of links in link section

4.3.2 Link section

The link section of a block is a variable length list of links which point to other blocks that are attached to this block. The meaning of each link depends on the type and possibly other options of the block. A link can be empty (NIL) if no block is attached.
The link section has a variable length of (link_count * sizeof(LINK)) Bytes.

Table 7 Link section

Name	Type	Count	Description
Link section			
…	LINK	1	Link to 1st attached block
...	LINK	1	Link to 2nd attached block
…	LINK	1	Link to 3rd attached block
…	LINK	1	…

4.3.3 Data section

The data section of a block contains all its data information. Usually the data section can be seen as a sequence of data fields which are described for the respective block type.
The data section has a variable length of (length – 24 – link_count * sizeof(LINK)) Bytes.

Table 8 Data section

Name	Type	Count	Description
Data section			
…	…	…	1st data member
…	…	…	2nd data member
…	…	…	…

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.3.4 Description of Members

For each block type, its link and data sections have different individual members, i.e. different links for the link section and different data fields for the data section. These members are listed and explained in a table for each block type.

For each member its MDF data type and its count are specified in the table. A count > 1 indicates that the member is an array of values. Sometimes, the count is variable and depends on the values of some other members. In certain cases, the count may even be zero, i.e. the member then is missing.

Although not relevant for the format description, a unique name is given to each member because this simplifies referencing this member in the description. We recommend using this member name when implementing reading and/or writing MDF files in a specific programming language.

The naming of the members uses the following rules:

- Member names are given as lower-case C identifiers using an underscore to separate different parts of the name for better readability, e.g. prefix or postfix.
- Members of the header section are named equally for each block, i.e. "id", "reserved", "length" and "link_count".
- All other members use the two-digit block type ID as prefix, e.g. "dg" for a data group block. Thus, a member name immediately can be related to its block type.
- A link pointing to a specific block type uses the target block type ID after the prefix, e.g. dg_cg_first for the link pointing to the first channel group (CG) block from a data group (DG) block. This also holds for links pointing to a meta data (MD) block, although they generally may point to a text (TX) block (see 4.7 The Meta Data Block MDBLOCK). That means that hd_md_comment may either point to a MDBLOCK or to a TXBLOCK. For all other links that can point to different block types, for better readability the target block type IDs are not listed as prefixes in the member name. For example, cn_data may point to a AT-, SD-, DL- or CGBLOCK.
- In some cases, forward linked lists of blocks are maintained (e.g. for FH- or ATBLOCKs). In these cases, _first will be used for the link to the start of the linked list, and _next for the link to the next block in the list. The only exceptions are cn_composition and ca_composition because they may point to different block types.

4.4 General Rules

There is a set of general rules which must be considered for MDF:

4.4.1 String Encoding, Byte Order and Alignment

- The string encoding used in an MDF file is UTF-8 (1-4 Bytes for each character, see [21]). This applies to TXBLOCK and MDBLOCK data.
  - For the strings in the IDBLOCK and for the block identifiers, always single byte character (SBC) encoding is used (standard ISO-8859-1 Latin character set).
  - For string signal values stored in the DTBLOCK, RDBLOCK or SDBLOCK, different encodings can be specified by the channel data type or the MIME type.
- The Byte order in an MDF file is Little Endian (Intel Byte order)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

  - All values (i.e. integers, floats) have to be interpreted accordingly. The only exception is that signal values stored in DTBLOCK, RDBLOCK or SDBLOCK, or that attachments stored as embedded data in ATBLOCK could use a different Byte order. In this case, the Byte order specified by the channel data type or by the MIME type overrules the default Byte order.
- The start address of a block must always be a multiple of 8 Byte (8-Byte alignment).
  - As a consequence, if the length of a block is not a multiple of 8, there will be a gap between this and the next block. However, this can only happen for blocks which do not automatically have a length that is a multiple of 8 Byte, e.g. for DTBLOCK, RDBLOCK, SDBLOCK, ATBLOCK, TXBLOCK or MDBLOCK.
  - Note that for records in DTBLOCK or RDBLOCK, or for the VLSD values in SDBLOCK, only a 1-Byte alignment is used.
- To achieve 8-Byte alignment of the block structures, reserved fields are inserted. Writing an MDF file, all reserved Bytes must be filled with zeros. For details please refer to Rules to Ensure MDF Compatibility between Versions.

4.4.2 Naming Rules

Several elements define a name, e.g. the channel name in CNBLOCK. Like other strings, a name also is UTF-8 encoded and has no special restriction on its length. It can contain any printable character except of

- double quotes (")
  If double quotes are in the original name, it is recommended to replace them by two single quotes.
- white space characters other than normal space (e.g. tab, carriage return, new line)

This applies to

- channel name (cn_tx_name in CNBLOCK)
- source name and path (si_tx_name and si_tx_path in SIBLOCK)
- hierarchy name (ch_tx_name in CHBLOCK)
- event name (ev_tx_name in EVBLOCK)
- conversion rule name (cc_tx_name in CCBLOCK)

Note: For the contents of XML tags, the rules are specified by the respective XML schema.

However, names used in the MDF file often origin from some other standardized source, e.g. from an MCD-2 MC database file ("A2L"), and thus follow more restrictive naming rules defined by the respective standard.

In practice, some tools may have problems with the rather unconstrained MDF naming rules because they rely on stricter naming rules. To be on the safe side, as a general recommendation, names in MDF files should be restricted, if possible, to the rules for identifiers in the C programming language:

- only allow characters A-Z and a-z, digits 0-9 and underscore (_)
- name must not start with a digit
- length of name is restricted to 255 characters

Alternatively, the rules for identifies in MCD-2 MC may be used, which are less restrictive than the C identifier rules and allow to divide the name into different parts by using dots.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.4.3 Identification of Channels

The identification of a channel is not coupled to the start address of its CNBLOCK because this may change, e.g. when sorting the MDF file (see 4.21.3 Sorted and Unsorted Data). Instead a channel should be identifiable by one or more human readable text fields and (only for array members) index values.

The identification of a channel within a measurement file must be unique. Also, it should be possible to compare measurement files and to detect identical channels.

Obviously, the channel name (cn_tx_name) is the most important identification string. Generally, the signal name from some source database format (e.g. FIBEX or A2L) should be used as channel name. Within the source database, each signal name usually will be unique.

However, an MDF file may store signals from different sources. Thus, for uniqueness, not only the signal name, but also the signal source must be considered. Information about the signal source is stored in the SIBLOCK. Note the components of a composed signal always must have the same source as the composed signal itself. Hence, for component channels (members of a structure or an array), no SIBLOCK is specified. Instead the SIBLOCK of the root CNBLOCK (direct child of a channel group) must be used.

Each source usually has a name (si_tx_name) and optionally can have a tool-specific "path" string (si_tx_path). The path string should be used to ensure uniqueness of the source name within the MDF file and may store additional, preferably human readable information about the source. Each tool can decide how to generate the path and which information to include. Therefore, the path string may not be comparable between files generated by different tools.

A channel generally can be seen as port or connection over which signal information is received and acquired. Writing an MDF file, the same signal information may be acquired and stored in different ways, e.g. with different sampling or precision. Hence, a channel not only is identified by the signal name and source, but also by the acquisition method for the signal data.

Note that we only distinguish between channel and signal for this explanation. Generally, an MDF channel can be seen as the signal itself.

All channels which are always acquired together, i.e. at the same time instants, can be collected in a channel group. Hence, the acquisition is not specified for each channel, but only for the channel group. Therefore, the CGBLOCK stores the name and the source information about the acquisition method (cg_tx_acq_name and cg_si_acq_source). Like for the signal source (cn_si_source), the relevant information about an acquisition source consists in a source name and a tool-specific path. Note that the acquisition source may be identical to the signal source, e.g. if an ECU raster is used to acquire a signal from the ECU.

Now let us put the different parts of the identification of a channel together. For simplicity, we will use the following terms and abbreviations:

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 9 Channel Naming

Term	Abbreviation	Meaning	Member
"channel name"	cn	(signal) name of a channel	cn_tx_name
"channel source"	cs	(signal) source name of a channel	si_tx_name of cn_si_source
"channel path"	cp	(signal) source path of a channel	si_tx_path of cn_si_source
"group name"	gn	(acquisition) name of a channel group	cg_tx_acq_name
"group source"	gs	(acquisition) source name of a channel group	si_tx_name of cg_si_acq_source
"group path"	gp	(acquisition) source path of a channel group	si_tx_path of cg_si_acq_source

Unique identification of a CNBLOCK within an MDF file

The combination of channel name, source and path (cn, cs, cp) together with the group name, source and path (gn, gs, gp) of its parent CGBLOCK must be unique for each CNBLOCKs across the entire file.

Note that for uniqueness the channel name itself even may be sufficient. All other parts which are not necessary to achieve uniqueness thus could be omitted (empty string or even omitting the SIBLOCK). However, it generally is recommended to provide these parts of information because they allow comparing different MDF files (see below).

Unique identification of a CNBLOCK within a channel group

For each CNBLOCK, the combination of its channel name, source and path (cn, cs, cp) must be unique within all CNBLOCKs which belong to the same channel group (CGBLOCK). This also applies to the CNBLOCKs of component channels (which use the SIBLOCK of their root channel as explained above).

Unique identification of a channel

A channel is identified like its CNBLOCK, except that for array members also the indices for each dimension of the array must be specified that lead to the specific array member. See also 4.18.2 Arrays.

Unique identification of a signal

If two channels in an MDF file have an equal combination of channel name, source and path (cn, cs, cp), they should be considered to be acquisitions of the same signal. Vice versa, if the same signal is stored in different channels in the MDF file using different acquisition methods, channel name, source and path (cn, cs, cp) should be identical.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Unique identification of an acquisition

If two channel groups in an MDF file have an equal combination of group name, source and path (gn, gs, gp), they should be considered to use the same acquisition method. Vice versa, if channels acquired by the same acquisition must be distributed to different channel groups (e.g. due to a multiplexor signal), the channel group name, source and path (gn, gs, gp) should be identical.

Note: As a consequence, group name, source and path (gn, gs, gp) generally cannot be used to uniquely identify a CGBLOCK.

Comparing different MDF files

When trying to match the channels of two MDF files, a tool can compare the channel name and source strings (cn, cs) for each pair of channels. The channel path (cp) usually may only be considered for comparison if both files have been generated by the same tool (assuming that the tool always uses the same strategy to create the cp string).

If the strings match, the tool may assume that both channels have been acquired from the same signal. Being more tolerant, the tool even might only compare the channel names and omitting the source name because different tools might not fill in the source name equally.

However, if within one MDF file, there are several channels matching one or more channels of the other file with regard to cn, cs and possibly cp, then the tool also might check the group name and source (gn, gs). Again, the group path (gp) might not be comparable if generated by different tools.

Best Practice for Naming of Channel and Acquisition

To allow comparing and merging of MDF files recorded by different tools, the choice of channel name and source (cn, cs) and group (i.e. acquisition) name and source (gn, gs) should be based on the original description files like MCD-2 MC or FIBEX. The following table specifies best practices for a few examples, for more use cases please refer to [10].

Table 10 Naming of Channel and Acquisition

Original description	Channel name (cn)	Acquisition name (gn)	Source name (cs, gs)
MCD-2 MC	CHARACTERISTIC / MEASUREMENT / AXIS_PTS name	SOURCE / EVENT name	MODULE name
FIBEX	<SIGNAL> name	<FRAME> OR <PDU> name	<ECU> name
I/O	Channel name chosen by user	periodic rate with unit (number >= 1): 100us, 1ms, 5.5ms	Device name chosen by user
Calculated Signals	Signal name chosen by user	periodic rate OR tool specific	Tool name

Usually the channel group and the channel have the same source, for example when an ECU signal is measured using an ECU raster. There are however some cases where the sources are different.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Examples:

Table 11 Source naming

Use case	Channel name (cn)	Channel source (cs)	Acquisition name (gn)	Acquisition source (gs)
Tool polling of ECU signal	signal name	ECU name	periodic rate OR tool specific	Tool name
Recording of calibration changes	calibration signal name	ECU name	"OnChange"	Tool name

If a single signal is acquired in several rates at the same time there are two possibilities for storing the data in MDF: Either as separate channels with different channel groups or as one channel in a merged channel group. In the second case, the acquisition name shall be constructed from all individual acquisition names by concatenation in increasing alphabetical order using the separator character "+".

4.4.4 Linking of Blocks

Note that in the MDF file only the IDBLOCK and the HDBLOCK have a fixed position. Any block of some other block type has an arbitrary start address in the file. The links in the link section of each block form a network of blocks. Any block (except of ID- and HDBLOCK) must be accessible by a path of links starting from the HDBLOCK which represents the root of the MDF file.

There are two types of links: referencing and unique links. Any number of referencing links can point to the same block whereas only one unique link may point to a block. For example, an ATBLOCK must be linked exactly once either by hd_at_first or by at_at_next. In addition, it may be referenced any number of times by ev_at_reference or by cn_data links.

Table 6 lists the set of unique links and referencing links for each block type. A block of the given block type must be linked exactly once by one of the links in column "unique links", and it may be referenced any number of times by the links in column "referencing links". The unique links rule also avoids circular references, e.g. in a linked list. For some links additional rules apply, which are explained locally for each block type definition. Figure 4 shows the hierarchy of uniquely linked blocks.

As mentioned, ID- and HDBLOCK can only occur once and both have a fixed position. Thus, no linking is necessary.

Note that there are block types for which there are no unique links. These blocks are called "sharable" blocks. Currently, TX-, MD-, SI- and CCBLOCKs are sharable blocks. Sharing may be used to reduce duplication of identical blocks (e.g. several channels sharing the same CCBLOCK or SIBLOCK).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

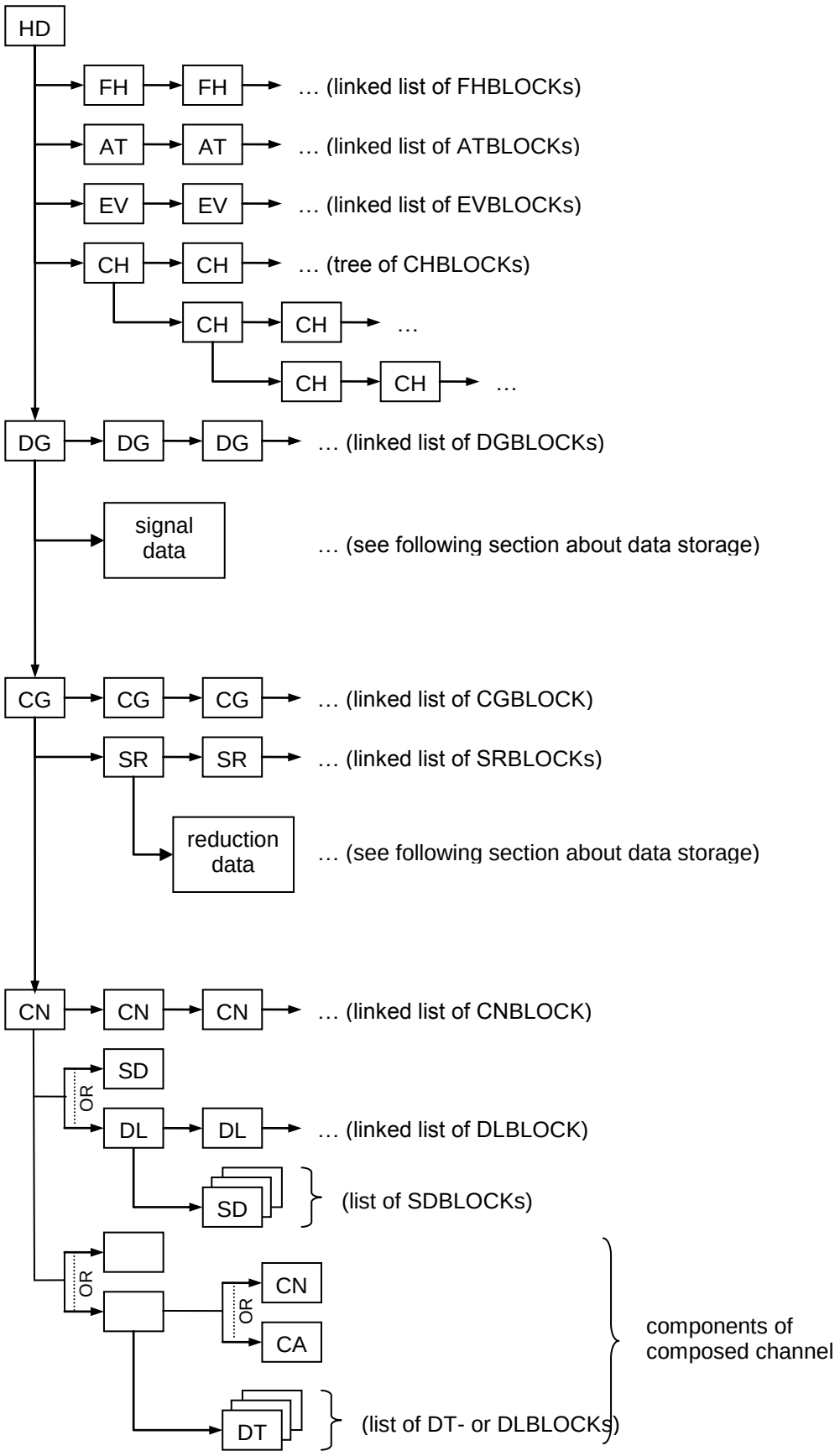


Figure 4 Schematic hierarchy of uniquely linked blocks

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 12 Linking of Blocks

Block Type ID	Unique links	Referencing links
ID, HD	[fixed position]	
TX, MD, SI, CC	[sharable blocks]	
FH	hd_fh_first fh_fh_next	
CH	hd_ch_first ch_ch_next ch_ch_first	
AT	hd_at_first at_at_next	ev_at_reference cn_at_reference cn_data
EV	hd_ev_first ev_ev_next	ev_parent ev_ev_range cn_data
DG	hd_dg_first dg_dg_next	ch_element ca_dynamic_size ca_input_quantity ca_output_quantity ca_comparison_quantity ca_axis
CG	dg_cg_first cg_cg_next	ch_element ca_dynamic_size ca_input_quantity ca_output_quantity ca_comparison_quantity ca_axis ev_parent ev_scope cn_data cg_cg_master
CN	cg_cn_first cn_cn_next cn_composition ca_composition	ch_element ca_dynamic_size ca_input_quantity ca_output_quantity ca_comparison_quantity ca_axis ev_scope cn_default_x cn_data
CA	cn_composition ca_composition	
SR	cn_sr_first sr_sr_next	
DT DZ (for DT)[2]	dl_data dg_data ca_data[i] for i > 0	ca_data[0]
DV DZ (for DV)[1]	ld_data dg_data ca_data[i] for i > 0	ca_data[0]

[2] DZBLOCK replaces a block of the respective block type, see dz_org_block_type of DZBLOCK

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Block Type ID	Unique links	Referencing links
DI DZ (for DI)[1]		ld_inval_data
RD DZ (for RD)[2]	dl_data sr_data	
RV DZ (for RV)[1]	ld_data sr_data	
RI DZ (for RI)[1]		ld_inval_data
SD DZ (for SD)[2]	dl_data cn_data	
DL	hl_dl_first dl_dl_next dg_data cn_data sr_data ca_data[i] for i > 0	ca_data[0]
LD	ld_ld_next dg_data sr_data ca_data[i] for i > 0	ca_data[0]
HL	dg_data cn_data sr_data ca_data[i] for i > 0	ca_data[0]

4.4.5 Data storage

Data can be stored in multiple different ways inside a MDF4 file. The available options for storing the data depend on the configuration of the data group (unsorted, sorted and column-oriented).

Signal or reduction data:
The simplest way for storing signal or reduction data is linking a single DTBLOCK (for signal data) or RDBLOCK (for reduction data). This option is available in unsorted and sorted configurations. The DT/RDBLOCK can also be a DZBLOCK wrapping the respective DT/RDBLOCK to enable compression. The original block should not be larger than 4 Mbytes in this case (see 4.31 The Data Zipped Block DZBLOCK for more information).

Another way of storing signal or reduction data is by linking a DLBLOCK. The DLBLOCK then contains links to the data blocks and, if required, a link to the next DLBLOCK in the list. If the DLBLOCK should contain links to compressed data, a HLBLOCK must be between the DGBLOCK and the DLBLOCK. This is also possible in sorted and unsorted.

The third possibility of storing signal or reduction data is by linking a LDBLOCK. It is possible for sorted and required for column-oriented configurations. This way of storing data separates the storage location of the actual data and the invalid bits. The LDBLOCK contains links to DV/RVBLOCKs containing the signal values and, if required, links to DI/RIBLOCKs containing the signal invalidation values. If no invalidation data is used, a DV/RVBLOCK may be used instead (as alternative to DT/RDBLOCK, necessary if parent CGBLOCK references a remote master group using cg_cg_master).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

VLSD data:
Data with variable length can be stored using either a single SDBLOCK or a DLBLOCK referencing multiple SDBLOCKs. Similar to signal and reduction data, the SDBLOCK can be a DZBLOCK containing the SDBLOCK for compression. If the DLBLOCK contains compressed data blocks, a HLBLOCK must be between the CNBLOCK and the DLBLOCK.

CA data in DG template:
The data for a channel array in the DG template appears in a sorted configuration of the group. Each ca_data link in the CABLOCK may either reference a DTBLOCK (DZBLOCK if compressed) or a DLBLOCK. If the DLBLOCK contains compressed data, the ca_link must link a HLBLOCK that references the DLBLOCK. Alternatively, a LDBLOCK or a single DVBLOCK can be used.

4.4.6 Synchronization Domains

Synchronization domains allow an ordering (and thus synchronization) of recorded samples or events with respect to their occurrence.

MDF supports four possible synchronization domains:

- time (in seconds)
- angle (in radians, recorded continuously increasing, e.g. 4π are 2 circles)
- distance (in meters)
- index (as zero-based index without unit)

Each domain must have monotonously increasing[3] values. The value of a synchronization domain will be denoted as synchronization value, its domain as synchronization type.

The synchronization values can be recorded and stored for an event or in a data record. For a data record, the index is given implicitly by the zero-based record index within the channel group. Hence the index can only be used within the scope of this channel group, whereas time, angle and distance have a global scope within the MDF file. Implicitly all signal values within the same data record are synchronized by the record index.

Time, angle or distance synchronization values can be stored in a data record (referenced by so-called "master" channels), or they can be calculated from the record index (referenced by so-called "virtual master" channels), e.g. for an equidistant raster. In each case they are relative to the respective start value defined globally for the MDF file (in the HDBLOCK).

A channel group may contain more than one (virtual) master channel as long as each belongs to a different synchronization domain (i.e. has a different synchronization type). An event, however, can only contain a synchronization value for one of the domains.

[3] The requirement is weaker compared to previous versions of the standard. This acknowledges that in some rare circumstances, equal master values cannot be prevented (e.g. event signals). It is however important to note that non-strictly monotonously increasing master values have their root cause usually in an error of the measurement setup and should not easily be accepted.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

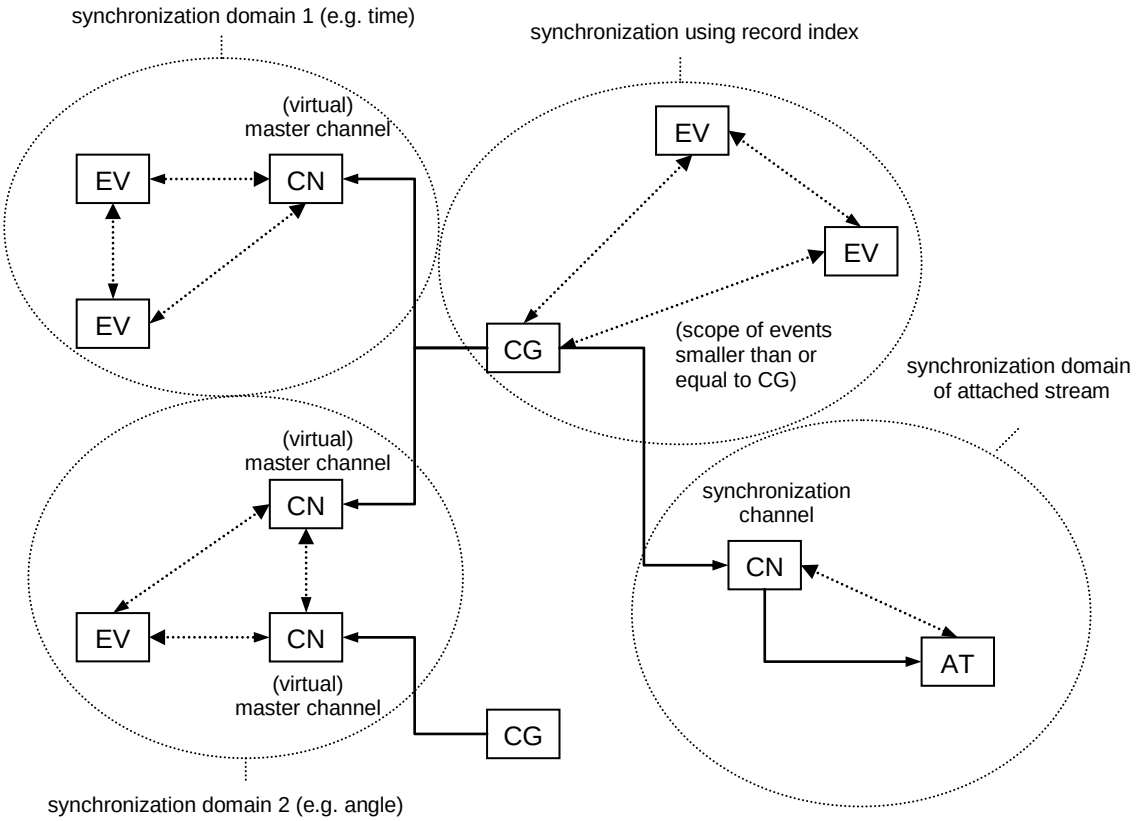


Figure 5 Possible synchronization domains

The synchronization values can be used to synchronize records (samples) and events, as long as they have the same synchronization type (and the same scope in case of index). Master channels also allow synchronization of samples from different channel groups.

Another kind of synchronization is done for a synchronization channel (cn_type = 4). Here the master channel values of the current sample are synchronized with some value that is used as synchronization value in a stream given as attachment (see 4.11 The Attachment Block ATBLOCK). Each attachment thus has an own synchronization domain.

Figure 5 shows schematically the possible synchronization domains.

4.5 The File Identification Block IDBLOCK

The IDBLOCK always begins at file position 0 and has a constant length of 64 Bytes. It contains information to identify the file. This includes information about the source of the file and general format specifications. To be compliant with older MDF formats in this section each CHAR must be a 1-Byte ASCII character. The IDBLOCK is the only block without a Header section and without a Link section.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.5.1 Block Structure of IDBLOCK

Table 13 Structure of IDBLOCK

Name	Type	Count	Description
id_file	CHAR	8	File identifier, always contains "MDF␣␣␣␣␣" ("MDF" followed by five spaces, no zero termination), except for "unfinalized" MDF files (see 4.5.2 Unfinalized MDF). The file identifier for unfinalized MDF files contains "UnFinMF␣" ("UnFinMF" followed by one space, no zero termination).
id_vers	CHAR	8	Format identifier, a textual representation of the format version for display, e.g. "4.20" (including zero termination) or "4.20␣␣␣␣" (followed by spaces, no zero-termination required if 4 spaces).
id_prog	CHAR	8	Program identifier, to identify the program which generated the MDF file (no zero-termination required). This program identifier serves only for compatibility with previous MDF format versions. Detailed information about the generating application must be written to the first FHBLOCK referenced by the HDBLOCK. As a recommendation, the program identifier inserted into the 8 characters should be the base name (first 8 characters) of the EXE/DLL of the writing application. Alternatively, also version information of the application can be appended (e.g. "MyApp45" for version 4.5 of MyApp.exe).
id_reserved1	BYTE	4	Reserved (must be 0 for compatibility reasons!)
id_ver	UINT16	1	Version number of the MDF format, i.e. 420 for this version
id_reserved2	BYTE	30	Reserved
id_unfin_ flags	UINT16	1	Standard flags for unfinalized MDF Bit combination of flags that indicate the steps required to finalize the MDF file. For a finalized MDF file, the value must be 0 (no flag set). See also the description in section 4.5.2 Unfinalized MDF and for id_custom_unfin_flags.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			The bit flags for id_unfin_flags are defined below in Table 14. Note that for the currently defined standard flags, the respective finalization steps must be executed in the following order (given that the respective bit is set in id_unfin_flags): 1. Update DLBLOCK (bit 4) 2. Update DT-/RDBLOCK (bit 2 and bit 3) 3. All other standard steps Custom finalization flags (see id_custom_unfin_ flags) also may require a specific order which may be intertwined with the above ordering.
id_custom_ unfin_flags	UINT16	1	Custom flags for unfinalized MDF Bit combination of flags that indicate custom steps required to finalize the MDF file. For a finalized MDF file, the value must be 0 (no flag set). See also 4.5.2 Unfinalized MDF. Custom flags should only be used to handle cases that are not covered by the (currently known) standard flags in id_unfin_flags. The meaning of the flags depends on the creator tool, i.e. the application that has written the MDF file or executed the most recent modification (see id_prog and last entry in file history). No tool should modify the MDF file (and thus write a new entry to the file history) before it was finalized correctly at least for all custom finalization steps. Finalization should only be done by the creator tool or a tool that is familiar with all custom finalization steps required for a file from this creator tool.

4.5.2 Unfinalized MDF

There may be situations where a tool just writing an MDF file ("creator tool") may not be able to successfully finalize it. This can be due to a premature termination of the program, e.g. by power off or an application error, etc. Depending on the writing strategy of the creator tool, the MDF file then may be left in an "unfinalized" state. This state may violate some format rules of MDF, or it does not show or contain all recorded data.

In many cases, a recovery of the unfinalized MDF file, i.e. a finalization by post-processing, is possible. The creator tool itself may offer this capability. Alternatively, the recovery may be done by another tool or even "manually" using some kind of editor.

However, if a third tool not being aware of the unfinalized state performs some post-processing step with the unfinalized MDF file (e.g. sorting), the recovery may become impossible and some part or even all of the contained data may be destroyed.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Hence a not yet finalized MDF file should be marked as "unfinalized" in a way that any post-processing tool either rejects it or possibly is able to recover the file by itself.
The marking of an unfinalized MDF file is independent of the used MDF version and simply consists of a different file identifier. Such a marked file should be rejected by already existing tools because they will not find the required MDF file identifier.
In addition to the file identifier for unfinalized MDF, the last four bytes of the IDBLOCK are used to hold bit flags that indicate how the file can be finalized. The flags are separated into standard flags and custom flags:

- The standard flags indicate common finalization steps which may be supported by any tool. It is up to each tool if it supports finalization or not. It may also support only some of the standard finalization steps.
- The custom flags can be used by the creator tool to document own steps specific to this tool. These steps are not described here and can possibly be executed only by the creator tool itself.

Since some of the finalization steps have to be executed in a specific order, a tool must not finalize an unfinalized file (not even partially) unless it knows all standard and custom flags and the necessary order of the respective finalization steps.

Any tool that successfully finalizes the MDF file must reset the bit flags and replace the file identifier with the normal MDF file identifier (`"MDF␣␣␣␣␣␣␣␣"`).

Note that there is no extra file extension defined for an unfinalized MDF file, it may even use the standard file extension (i.e. *.mf4).

Table 14 Bit flags for id_unfin_flags (Bit 0 = LSB)

Bit	Description
Bit 0	Update of cycle counters for CG-/CABLOCK required. The value for cg_cycle_count in a CGBLOCK and the values for ca_cycle_count in a CABLOCK with CG/DG template may be wrong (zero or not up-to-date). For sorted data, the number of records can be calculated from the respective data block length(s). For unsorted data, it is necessary to iterate over all records, so this also may be done while sorting the data. The precondition for both cases is that the length of each referenced data block is correct. Thus, if bit 2 (see below) is set, then the length of the last DTBLOCK must be corrected first.
Bit 1	Update of cycle counters for SRBLOCKs required. The value of sr_cycle_count in a SRBLOCK may be wrong (zero or not up-to-date). It can be calculated from the length(s) of the respective RDBLOCK (s) for this SRBLOCK. Thus, if bit 3 (see below) is set, then the length of the last RDBLOCK must be corrected first.
Bit 2	Update of length for last DTBLOCK required. The length of the last DTBLOCK in each data block list (single DTBLOCK or last DTBLOCK of last DLBLOCK in chained DLBLOCK list) may be wrong and must be updated.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Bit	Description
	This flag must only be set in case a successful restoration is possible, i.e. if any of the following restrictions is fulfilled: • The DTBLOCK is limited by the end of the file (EOF). • The DTBLOCK is limited by the start of another MDF block within the block hierarchy. • The DTBLOCK is limited by a value not used as record ID (only in case a record ID is used). • The DTBLOCK is limited by a record that contains an invalid master value, e.g. a time stamp that is not larger than the previous time stamp within the same channel group (only if a non-virtual master channel is present). In case of an unsorted data block, its length may also be determined while sorting the data block. If bit 4 (see below) is set, then the DLBLOCK list must be corrected first.
Bit 3	Update of length for last RDBLOCK required. The length of the last RDBLOCK in each data block list (single RDBLOCK or last RDBLOCK of last DLBLOCK in chained DLBLOCK list) may be wrong and must be updated. This flag must only be set in case a successful restoration is possible, i.e. if any of the following restrictions is fulfilled: • The RDBLOCK is limited by the end of the file (EOF). • The RDBLOCK is limited by the start of another MDF block within the block hierarchy. • The RDBLOCK is limited by a record that contains an invalid value for the interval start, e.g. a value not larger than the previous one (the interval start is the value of the master channel matching sr_sync_type stored in sub record 1 of the sample reduction record, i.e. this is only possible if a non-virtual master channel is present). If bit 4 (see below) is set, then the DLBLOCK list must be corrected first.
Bit 4	Update of last DLBLOCK in each chained list of DLBLOCKs required. The last DLBLOCK in a list of chained DLBLOCKs may not be filled completely, i.e. dl_count may be too high and the last links in dl_data list may be NIL. Equally the offset list (if used) may be too long. For finalization, the number of valid links must be determined (i.e. all links that are not NIL) and the DL must be shortened accordingly, i.e. reducing the number of links and possibly the size of the offset list.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Bit	Description
Bit 5	Update of cg_data_bytes and cg_inval_bytes in VLSD CGBLOCK required (unsorted data only). Both values may be wrong (zero or not up-to-date). Together these two values specify the total size in Bytes of all variable length signal values for the recorded samples of this channel group (see 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK). Thus, finalization here requires iterating over all records in the unsorted data block in order to sum up the lengths of the VLSD values for this channel group. On the other hand, if the data is to be sorted, then the VLSD CGBLOCK will be eliminated anyway. In this case a finalization of the values may not be necessary; but the sorting routine must not rely on these values, e.g. to calculate and reserve space for the final SDBLOCK for the sorted data. If bit 2 (see above) is set, then either the length of the (unsorted) DTBLOCK must be corrected first, or summing up the VLSD lengths is already done while iterating over all samples when trying to determine the DTBLOCK length.
Bit 6	Update of offset values for VLSD channel required in case a VLSD CGBLOCK is used (unsorted data only). Usually, when using a VLSD CGBLOCK for writing the values for a VLSD channel (cn_type = 1), then the VLSD values are stored in so-called "VLSD records" in the (unsorted) data block whereas the "parent" record (with fixed length) contains an offset value for the VLSD channel just as if the VLSD values would be stored in a SDBLOCK (see 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK). If this bit is set, then the offset values may be wrong (typically zero). Finalization requires to iterate over all records in the unsorted data block and to determine the respective offset value from the accumulated lengths of the previous VLSD records for this channel (length of VLSD value + 4 byte for length info). This may also be done while sorting the unsorted data. If bit 2 (see above) is set, then either the length of the (unsorted) DTBLOCK must be corrected first, or the correction of the offset values can be done while iterating over all samples when trying to determine the DTBLOCK length.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.6 The Header Block HDBLOCK

The HDBLOCK always begins at file position 64. It contains general information about the contents of the measured data file and is the root for the block hierarchy.

4.6.1 Block Structure of HDBLOCK

Table 15 Structure of HDBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##HD”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
hd_dg_first	LINK	1	Pointer to the first data group block (DGBLOCK) (can be NIL)
hd_fh_first	LINK	1	Pointer to first file history block (FHBLOCK) There must be at least one FHBLOCK with information about the application which created the MDF file.
hd_ch_first	LINK	1	Pointer to first channel hierarchy block (CHBLOCK) (can be NIL).
hd_at_first	LINK	1	Pointer to first attachment block (ATBLOCK) (can be NIL)
hd_ev_first	LINK	1	Pointer to first event block (EVBLOCK) (can be NIL)
hd_md_comment	LINK	1	Pointer to the measurement file comment (TXBLOCK or MDBLOCK) (can be NIL) For MDBLOCK contents, see Table 16.
Data section			
hd_start_time_ns	UINT64	1	Time stamp at start of measurement in nanoseconds elapsed since 00:00:00 01.01.1970 (UTC time or local time, depending on "local time" flag, see [20]). All time stamps for time synchronized master channels or events are always relative to this start time stamp.
hd_tz_offset_min	INT16	1	Time zone offset in minutes. The value is not necessarily a multiple of 60 and can be negative! For the current time zone definitions, it is expected to be in the range [-840,840] min. For example a value of 60 (min) means UTC+1 time zone = Central European Time (CET).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Only valid if "time offsets valid" flag is set in time flags.
hd_dst_offset_min	INT16	1	Daylight saving time (DST) offset in minutes for start time stamp. During the summer months, most regions observe a DST offset of 60 min (1 hour). Only valid if "time offsets valid" flag is set in time flags.
hd_time_flags	UINT8	1	Time flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: Local time flag If set, the start time stamp in nanoseconds represents the local time instead of the UTC time. In this case, time zone and DST offset must not be considered (time offsets flag must not be set). Should only be used if UTC time is unknown. If the bit is not set (default), the start time stamp represents the UTC time.
			Bit 1: Time offsets valid flag If set, the time zone and DST offsets are valid. Must not be set together with "local time" flag (mutually exclusive). If the offsets are valid, the locally displayed time at start of recording can be determined (after conversion of offsets to ns) by Local time = UTC time + time zone offset + DST offset.
hd_time_class	UINT8	1	Time quality class 0 = local PC reference time (Default) 10 = external time source 16 = external absolute synchronized time See also suggestion to document temporary loss of synchronization for hd_time_class > 0 in section 4.12.6.
hd_flags	UINT8	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: Start angle valid flag If set, the start angle value below is valid.
			Bit 1: Start distance valid flag If set, the start distance value below is valid.
hd_reserved	BYTE	1	Reserved
hd_start_angle_rad	REAL	1	Start angle in radians at start of measurement (only for angle synchronous measurements)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Only valid if "start angle valid" flag is set. All angle values for angle synchronized master channels or events are relative to this start angle.
hd_start_distance_m	REAL	1	Start distance in meters at start of measurement (only for distance synchronous measurements) Only valid if "start distance valid" flag is set. All distance values for distance synchronized master channels or events are relative to this start distance.

4.6.2 Contents of the Meta Data Block for the Header Comment

The content of the MDBLOCK for the comment of the header is defined by the XML schema hd_comment.xsd. In addition to the standard tags it includes the following tags specific to the HDBLOCK.

Table 16 Tags for MDBLOCK hd_md_comment

Tag name	Description
<TX>	Comment for MDF file in plain text (default tag, required)
<time_source>	Name of time source (optional) Example: "local PC reference timer" or "GPS reference timer"
<constants>	Definition of global system constants (optional) See MCD-2 MC [1] keyword SYSTEM_CONSTANT. The constants can be used in syntax expressions for trigger/alarm conditions (see 4.12 The Event Block EVBLOCK), calculated signals (see 4.16.1 Block Structure of CNBLOCK) or algebraic conversion rule (see 4.17The Channel Conversion Block CCBLOCK). Contains a number of sub tags <const>

<const>	Sub-tag of <constants> to define the single constants. (optional)
	The contents of the tag can be a numerical value or a formula part (expression in GES) used to replace the constant name in a formula. Macro expressions may use any constant name defined previously in the list (see example below).
	The name of the constant is specified by the required attribute "name":
	name must be a valid C identifier, i.e. only characters a-z, A-Z, 0-9 and underscore allowed, must not start with a digit 0-9. The name must be unique within the list of constants und must not be equal to a reserved keyword in ASAM General Expression Syntax.
	Example:
	`<constants>`
	`  <const name="PI">3.14159265</const>`
	`  <const name="DED2RAD">PI/180</const>`
	`  <const name="sin_45">sin(45*DEG2RAD)</const>`
	`</constants>`

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Tag name	Description

<ho:UNIT-SPEC>	Catalogue of HDO definitions (optional)
	The definitions can be referenced by XML in other MDBLOCKs by using the ID-REF or unit_ref attribute as appropriate.
	Can contain complete grammar of HDO unit definitions.
	Note: common unit definitions can simply be copied from file units4fibex.xml provided for ASAM FIBEX standard.
	Example:
	`<ho:UNIT-SPEC>`
	`  <ho:PHYSICAL-DIMENSIONS>`
	`  <ho:PHYSICAL-DIMENSION ID="siBase_m">`
	`  <ho:SHORT-NAME>length</ho:SHORT-NAME>`
	`  <ho:LONG-NAME xml:lang="de">Länge</ho:LONG-NAME>`
	`  <ho:LONG-NAME xml:lang="en">length</ho:LONG-NAME>`
	`  <ho:DESC xml:lang="en">base quantity: length</ho:DESC>`
	`  <ho:LENGTH-EXP>1</ho:LENGTH-EXP>`
	`  </ho:PHYSICAL-DIMENSION>`
	`  </ho:PHYSICAL-DIMENSIONS>`
	`  <ho:UNITGROUPS>`
	`  <ho:UNITGROUP>`
	`  <ho:SHORT-NAME>Metric</ho:SHORT-NAME>`
	`  <ho:CATEGORY>COUNTRY</ho:CATEGORY>`
	`  <ho:UNIT-REFS>`
	`  <ho:UNIT-REF ID-REF="siBase_m"/>`
	`  </ho:UNIT-REFS>`
	`  </ho:UNITGROUP>`
	`  </ho:UNITGROUPS>`
	`  <ho:UNITS>`
	`  <ho:UNIT ID="unitSiBase_meter">`
	`  <ho:SHORT-NAME>meter</ho:SHORT-NAME>`
	`  <ho:LONG-NAME xml:lang="de">Meter</ho:LONG-NAME>`
	`  <ho:LONG-NAME xml:lang="en">meter</ho:LONG-NAME>`
	`  <ho:DESC xml:lang="en">named SI unit for base quantity</ho:DESC>`
	`  <ho:DISPLAY-NAME>m</ho:DISPLAY-NAME>`
	`  <ho:PHYSICAL-DIMENSION-REF ID-REF="siBase_m"/>`
	`  </ho:UNIT>`
	`  </ho:UNITS>`
	`</ho:UNIT-SPEC>`

Note: MDF versions prior to version 4.0 defined four additional fields for the header block to specify the author, the department, the project and a subject string. As best practice, these fields should be mapped to MDF4 by using the generic <e> tag on top level of <common_properties> with the following values for the name attribute: "author", "department", "project" and "subject". In addition, further fixed <e> tag names are listed in Table 17 and explained in the following section.

Table 17 Fixed Values for Name Attribute of <e> Tags in MDBLOCK hd_md_comment

Name	Description
author	Name of author
department	Name of organization or department
project	Name of project
subject	Name of subject/system under test, e.g. vehicle information
Measurement.UUID	Universally unique identifier (UUID) to identify files with synchronized master channel values (i.e. files from same measurement, see section 4.6.3)
Recorder.UUID	Universally unique identifier (UUID) to identify files with same *Measurement.UUID* and equally assembled channel groups (i.e. files from same recorder, see section 4.6.3)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Recorder.FileIndex	Index for temporal ordering of files with same *Recorder.UUID* , same *Measurement.UUID* and with same start time (i.e. files from a file series where the records of matching channel groups can simply be concatenated without adaption, see section 4.6.3)

4.6.3 Best Practice for Unique Identification of Related Measurement Files

This section suggests best practices to identify associated measurements files using generic <e> tags on top level of <common_properties> in hd_md_comment with the following values for the name attribute (see also Table 17)

- Measurement.UUID
- Recorder.UUID
- Recorder.FileIndex

This mechanism can safely be used in older MDF 4.x versions. The <e> tags typically are considered “read-only”, i.e. the “ro” attribute should be set to true.

Measurement.UUID
The value of this field should be a universally unique identifier (UUID) which is a string representing a 128-bit number used to identify information in computer systems. Details about UUIDs can be found in ISO 9834-8 [14].

Requirement: Files with the same *Measurement.UUID* must have synchronized master channel values, i.e. they must be synchronized according to the synchronization domains time, angle or distance.

Explanation: The *Measurement.UUID* serves to identify files recorded during the same “measurement”. Measurement here means the period during which data from possibly different sources is acquired and recorded into one or multiple files based on the same time base (more generally: in the same synchronization domain). The data acquisition and/or recording may be executed on the same or on different systems. Acquisition on different systems requires a time synchronization based on a common time (e.g. UTC time via GPS) or, more generally, on a shared synchronization signal.

Note: Only part of the data acquired during a measurement may be stored in the files, e.g. due to interruptions or trigger conditions. Typically, a tool generates a new value for the *Measurement.UUID* at start of a measurement. Several consecutively executed “measurements” might also be considered as one single measurement with interrupts instead of individual measurements.

Recorder.UUID
Like for the *Measurement.UUID*, the value of this field should be a universally unique identifier (UUID), see above.

Requirement: Files with the same *Recorder.UUID* value must have the same *Measurement UUID*. In addition, they must have equally assembled channel groups: if the same signal is recorded into two files with the same *Recorder.UUID*, the channel groups containing this signal must have equal identification fields (acquisition name and source) and have the same record layout. This implies that both channel groups have the same record length and that all matching signals must have the same data type, bit/byte offset, invalidation bit, conversion rule, unit, source information, etc. Only runtime information of the signals like min/max values may be different. However, the start time hd_start_time_ns and other information about the file may have different values.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Note: Typically, all signals of the two channel groups match. In some cases, one group may contain additional signals without changing the record layout of the others.

Explanation: The *Recorder.UUID* serves to identify files of a “file series” recorded during the same “measurement” where each file of the series contains a clipped section (subset) of the complete measurement period.

This may be used to distribute the measurement data to several smaller files instead of one huge file (“splitting” of measurement files). Or each of the single files contains a so-called “trigger window” where the measurement data is not recorded constantly but only when a certain trigger condition occurs. A trigger window often also stores data before and/or after the trigger event (pre- and post-trigger time). Note that due to the pre- and post-trigger time, the timelines of the files in the file series may overlap.

The *Recorder.UUID* should be created at start of a measurement for each “recorder”. The channel group configuration must not be changed during the complete measurement except for appending or removing complete channel groups. Furthermore, modifications to existing channel group configurations that do not change the record layout are also allowed with the same Recorder.UUID.

Recorder.FileIndex
The value of this field should be a zero-based index (Integer value).

Requirement: *Recorder.FileIndex* can only be used in combination with a *Recorder.UUID* and a *Measurement.UUID*. The value of *Recorder.FileIndex* defines the temporal ordering of files with the same *Recorder.UUID.* Files with same *Recorder.UUID* and same *Measurement.UUID* and which contain a *Recorder.FileIndex* must have equal start time (hd_start_time_ns) resp. start values for the master channels in HDBLOCK. Matching channel groups in the files of this file series must not have overlapping timelines, i.e. the last master channel value of a channel group in file with *Recorder.FileIndex* N must be less or equal to the first master channel value in the respective channel group in the file with *Recorder.FileIndex* (N+1). Consequently, the files can be concatenated by appending the unmodified records of matching channel groups, considering the ordering according to *Recorder.FileIndex*.

Recorder.FileIndex should be omitted if no file series will be written. However, it is a valid use case that only one file of a file series is written which then will have zero as *Recorder.FileIndex*.

Explanation: The *Recorder.FileIndex* in combination with *Recorder.UUID* serves to identify the ordering of files in a “file series” recorded during the same “measurement”. Sorting the files according to the value of *Recorder.FileIndex*, it must be possible to concatenate the files (i.e. concatenate records of matching channel groups), to receive a timeline of the measurement as if it had been recorded into one single file.

Note that two files with equal *Recorder.UUID* and *Recorder.FileIndex* either must contain the same time range, or the time range of one of the two files must be a subset of the other. This can happen when extracting a time range of one file and keeping the comment information.

Attention: sample reduction records usually cannot be appended since they are calculated per individual file of the series and thus may not match respective to interval and sampling rate.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Example:
hd_md_comment for first file of a file series

```
<HDcomment>
<TX></TX>
<common_properties>
  <e name="Measurement.UUID" ro="true">44891e3b-265a-4adc-ac6e-
   7436679e94bc</e>
  <e name="Recorder.UUID" ro="true">75935bc7-3032-42b5-8638-
   325dd468484b</e>
  <e name="Recorder.FileIndex" type="integer" ro="true">0</e>
</common_properties>
</HDcomment>
```

4.7 The Meta Data Block MDBLOCK

The MDBLOCK contains information encoded as XML string. For example, this can be comments for the measured data file, file history information or the identification of a channel. This information is ruled by the parent block and follows specific XML schemas definitions.

The MDBLOCK can also contain customized information. For further details about the XML structure please refer to chapter MDF Meta Data Format.

If only the main textual information in the MDBLOCK (contents of the <TX> element) is used, i.e. if it only carries a plain string as information, then the MDBLOCK can be replaced by a simpler TXBLOCK containing the plain string. Note that this is not possible for the MDBLOCK referenced by fh_md_comment because its XML schema definition specifies other required tags in addition to the <TX> tag. Writing a TXBLOCK instead of an MDBLOCK can be used to reduce the file size and to simplify reading the data (no XML parsing required).

Please note that this rule does not apply vice versa, i.e. if explicitly a TXBLOCK is required, it cannot be replaced by an MDBLOCK.

The length of the XML string results from the block size.

4.7.1 Block Structure of MDBLOCK

Table 18 Structure of MDBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##MD”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
Data section			
md_data	BYTE	variable	XML string UTF-8 encoded, zero terminated, new line indicated by CR and LF.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Note: A file may reserve spaces to allow modifying the XML text within its reserved range without the need to rewrite the entire MDF file. The Bytes after the zero termination of the XML string should not be considered and must not hold any hidden information.

4.8 The Text Block TXBLOCK

The TXBLOCK is very similar to the MDBLOCK but only contains a plain string encoded in UTF-8. The text length results from the block size.

4.8.1 Block Structure of TXBLOCK

Table 19 Structure of TXBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##TX”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
Data section			
tx_data	BYTE	variable	Plain text string UTF-8 encoded, zero terminated, new line indicated by CR and LF.

Note: A file may reserve spaces to allow modifying the text within its reserved range without the need to rewrite the entire MDF file. The Bytes after the zero termination of the text string should not be considered and must not hold any hidden information.

4.9 The File History Block FHBLOCK

The FHBLOCK describes who/which tool generated or changed the MDF file. Each FHBLOCK contains a change **log** entry for the MDF file. The first FHBLOCK referenced by the HDBLOCK must contain information about the tool which created the MDF file. Starting from this FHBLOCK then a linked list of FHBLOCKs can be maintained with a chronological change history, i.e. any other application that changes the file must append a new FHBLOCK to the list. A zero-based index value is used to reference blocks within this linked list, thus the ordering of the blocks must not be changed.

4.9.1 Block Structure of FHBLOCK

Table 20 Structure of FHBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##FH”
reserved	BYTE	4	Reserved used for 8-Byte alignment

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
fh_fh_next	LINK	1	Link to next FHBLOCK (can be NIL if list finished)
fh_md_comment	LINK	1	Link to MDBLOCK containing comment about the creation or modification of the MDF file.
Data section			
fh_time_ns	UINT64	1	Time stamp at which the file has been changed/created (first entry) in nanoseconds elapsed since 00:00:00 01.01.1970 (UTC time or local time, depending on "local time" flag).
fh_tz_offset_min	INT16	1	Time zone offset in minutes. The value is not necessarily a multiple of 60 and can be negative! For the current time zone definitions, it is expected to be in the range [-840,840] min. For example a value of 60 (min) means UTC+1 time zone = Central European Time (CET). Only valid if "time offsets valid" flag is set in time flags.
fh_dst_offset_min	INT16	1	Daylight saving time (DST) offset in minutes for start time stamp. During the summer months, most regions observe a DST offset of 60 min (1 hour). Only valid if "time offsets valid" flag is set in time flags.
fh_time_flags	UINT8	1	Time Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: Local time **flag** If set, the start time stamp in nanoseconds represents the local time instead of the UTC time, In this case, time zone and DST offset must not be considered (time offsets flag must not be set). Should only be used if UTC time is unknown. If not set (default), the start time stamp represents the UTC time.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Bit 1: Time offsets valid flag If set, the time zone and DST offsets are valid. Must not be set together with "local time" flag (mutually exclusive). If the offsets are valid, the locally displayed time at start of recording can be determined (after conversion of offsets to ns) by Local time = UTC time + time zone offset + DST offset.
fh_reserved	BYTE	3	**Reserved**

4.9.2 Contents of the Meta Data Block for the File History Comment

The content of the MDBLOCK for the comment of the file history is defined by the XML schema fh_comment.xsd. In addition to the standard tags it includes the following tags specific to the FHBLOCK.

Table 21 Tags for MDBLOCK fh_md_comment

Tag name	Description
<TX>	Comment describing the creation or modification of the MDF file.
<tool_id>	Name/identification of the tool that has written this log entry (required)
<tool_vendor>	Name of the vendor/author of the tool (required)
<tool_version>	Version info about the tool that has written this log entry (required)
<user_name>	Name / identification of the user who has written this log entry (optional)

4.10 The Channel Hierarchy Block CHBLOCK

The CHBLOCKs describe a logical ordering of the channels in a tree-like structure. This only serves to structure the channels and is totally independent to the data group and channel group structuring. A channel even may not be referenced at all or more than one time.

Each CHBLOCK can be seen as a node in a tree which has a number of channels as leafs and which has a reference to its next sibling and its first child node (both CHBLOCKs). The reference to a channel is always a triple link to the CNBLOCK of the channel and its parent CGBLOCK and DGBLOCK. Each CHBLOCK can have a name.

4.10.1 Block Structure of CHBLOCK

Table 22 Structure of CHBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##CH”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
link_count	UINT64	1	Number of links
Link section			
ch_ch_next	LINK	1	Link to next sibling CHBLOCK (can be NIL)
ch_ch_first	LINK	1	Link to first child CHBLOCK (can be NIL, must be NIL for ch_type = 3 ("map list")).
ch_tx_name	LINK	1	Link to TXBLOCK with the name of the hierarchy level. Must be NIL for ch_type ≥ 4, must not be NIL for all other types. If specified, the name must be according to naming rules stated in 4.4.2 Naming Rules, and it must be unique within all sibling CHBLOCKs.
ch_md_comment	LINK	1	Link to TXBLOCK or MDBLOCK with comment and other information for the hierarchy level (can be NIL)
ch_element	LINK	N x 3	References to the channels for this hierarchy level. Each reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel. Thus the links have the following order DGBLOCK for channel 1 CGBLOCK for channel 1 CNBLOCK for channel 1 … DGBLOCK for channel N CGBLOCK for channel N CNBLOCK for channel N None of the links can be NIL.
Data section			
ch_element_count	UINT32	1	Number of channels N referenced by this CHBLOCK.
ch_type	UINT8	1	Type of hierarchy level: (see also Table 23 for allowed child types)
			0 = group All elements and children of this hierarchy level form a logical group (see MCD-2 MC [1] keyword GROUP).
			1 = function All children of this hierarchy level form a functional group (see MCD-2 MC [1] keyword FUNCTION) For this type, the hierarchy must not contain CNBLOCK references (ch_element_count must be 0).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			2 = structure All elements and children of this hierarchy level form structure, see 4.18.1 Structures. Together with the remote master link, the structure can be compact if all linked elements form a logical group. If the elements of the structure are from different groups, they form a “fragmented” structure. Note: Do not use "fragmented" and "compact" structure in parallel. If possible prefer a "compact" structure.
			3 = map list All elements of this hierarchy level form a map list (see MCD-2 MC [1] keyword MAP_LIST): • the first element represents the z axis (must be a curve, i.e. CNBLOCK with CABLOCK of type "scaling axis") • all other elements represent the maps (must be 2-dimensional map, i.e. CNBLOCK with CABLOCK of type "look-up")
			4 = input variables of function (see MCD-2 MC [1] keyword IN_MEASUREMENT) All referenced channels must be measurement objects ("calibration" flag (bit 7) not set in cn_flags)
			5 = output variables of function (see MCD-2 MC [1] keyword OUT_MEASUREMENT) All referenced channels must be measurement objects ("calibration" flag (bit 7) not set in cn_flags)
			6 = local variables of function (see MCD-2 MC [1] keyword LOC_MEASUREMENT) All referenced channels must be measurement objects ("calibration" flag (bit 7) not set in cn_flags)
			7 = calibration objects defined in function (see MCD-2 MC [1] keyword DEF_CHARACTERISTIC) All referenced channels must be calibration objects ("calibration" flag (bit 7) set in cn_flags)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			8 = calibration objects referenced in function (see MCD-2 MC [1] keyword REF_CHARACTERISTIC) All referenced channel must be calibration objects ("calibration" flag (bit 7) set in cn_flags)
ch_reserved	BYTE	3	**Reserved**

The following table shows how many times a hierarchy level type (row) can appear as direct child of the parent hierarchy level type (column):
[-] cannot appear
[1] can appear at most one time (0..1 times)
[x] can appear arbitrary often (0..x times)

Example: a group (type 0) cannot appear as child of a function (type 1).

Table 23 Table of allowed child types for hierarchy levels

child type \ parent type	0 = group	1 = function	2 = structure	3 = map list	4 = input variables	5 = output variables	6 = local variables	7 = defined cal. objects	8 = referenced cal. objects
0 = group	x	-	-	-	-	-	-	-	-
1 = function	x	x	-	-	-	-	-	-	-
2 = structure	x	-	x	-	x	x	x	x	x
3 = map list	x	-	x	-	x	x	x	x	x
4 = input variables	-	1	-	-	-	-	-	-	-
5 = output variables	-	1	-	-	-	-	-	-	-
6 = local variables	-	1	-	-	-	-	-	-	-
7 = defined cal. objects	-	1	-	-	-	-	-	-	-
8 = referenced cal. objects	-	1	-	-	-	-	-	-	-

4.10.2 Contents of the Meta Data Block for the Hierarchy Comment

The content of the MDBLOCK for the comment of the hierarchy level is defined by the XML schema ch_comment.xsd. In addition to the standard tags it includes the following tags specific to the CHBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 24 Tags for MDBLOCK ch_md_comment

Tag name	Description
<TX>	Comment/description text for hierarchy level in plain text (default tag, required)
<names>	Alternative names and description as defined in 5.4 Alternative Names. (optional)

4.11 The Attachment Block ATBLOCK

The ATBLOCK specifies attached data, either by referencing an external file or by embedding the data in the MDF file. Embedding the data, the data stream optionally can be compressed using the Deflate zip algorithm (see [12] and [24]).

The ATBLOCK also holds information about the content of the attached data using a MIME content-type string (e.g. "video/avi" or "application/text", see [17] and [18]). For rules how to define custom MIME types please use the rules defined in ASAM ODS Version 5.1.1, chapter 8. In MDF, only the long form of the ASAM ODS MIME type string may be used, e.g. "application/x-asam.aolog".

To check the integrity of an external file, a 128-bit check sum ("fingerprint") according to the Message-Digest algorithm 5 can be calculated for it and stored in the ATBLOCK. This also is recommended when compressing embedded data to verify that the decompression returns the original data (see MD5 [15] and [16]).

Each ATBLOCK has a reference to the next ATBLOCK, thus allowing an arbitrary number of files to be attached (global linked list starting from HDBLOCK). Note: all ATBLOCKs of the file must be in the global linked list.

4.11.1 Block Structure of ATBLOCK

Table 25 Structure of ATBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##AT”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
at_at_next	LINK	1	Link to next ATBLOCK (linked list) (can be NIL)
at_tx_filename	LINK	1	Link to TXBLOCK with the path and file name of the embedded or referenced file (can only be NIL if data is embedded). The path of the file can be relative or absolute. If relative, it is relative to the directory of the MDF file. If no path is given, the file must be in the same directory as the MDF file.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
at_tx_mimetype	LINK	1	LINK to TXBLOCK with MIME content-type text that gives information about the attached data. Can be NIL if the content-type is unknown, but should be specified whenever possible. The MIME content-type string must be written in lowercase.
at_md_comment	LINK	1	Link to MDBLOCK with comment and additional information about the attachment (can be NIL).
Data section			
at_flags	UINT16	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: Embedded data flag If set, the attachment data is embedded, otherwise it is contained in an external file referenced by file path and name in at_tx_filename.
			Bit 1: Compressed embedded data flag If set, the stream for the embedded data is compressed using the Defalte zip algorithm (see [12] and [24]). Can only be set if "embedded data" flag (bit 0) is set.
			Bit 2: MD5 check sum valid flag If set, the at_md5_checksum field contains the MD5 check sum of the data.
at_creator_index	UINT16	1	Creator index, i.e. zero-based index of FHBLOCK in global list of FHBLOCKs that specifies which application has created this attachment, or changed it most recently.
at_reserved	BYTE	4	Reserved
at_md5_checksum	BYTE	16	128-bit value for MD5 check sum (of the uncompressed data if data is embedded and compressed). Only valid if "**MD5 check sum valid" flag** (bit 2) is set.
at_original_size	UINT64	1	Original data size in Bytes, i.e. either for external file or for compressed data.
at_embedded_size	UINT64	1	Embedded data size N, i.e. number of Bytes for binary embedded data following this element. Must be 0 if external file is referenced.
at_embedded_data	BYTE	N	Contains binary embedded data (possibly compressed).

4.11.2 Contents of the Meta Data Block for the Attachment Comment

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

The content of the MDBLOCK for the comment of the attachment is defined by the XML schema at_comment.xsd. In addition to the standard tags it includes the following tags specific to the ATBLOCK.

Table 26 Tags for MDBLOCK at_md_comment

Tag name	Description
<TX>	Comment text for attachment (default tag, required)

4.12 The Event Block EVBLOCK

The EVBLOCK serves to describe an event. Each EVBLOCK stores a synchronization value to specify when the event occurred. Usually this will be a time stamp, but the event also can be synchronized with some other master channel type or the record index of a channel group (see 4.4.6 Synchronization Domains). Generally, an event defines a "point" in time or some other synchronization domain. For some event types, two "points" can be used to define a "range".

All events are stored in a forward linked list starting from the HDBLOCK, i.e. each EVBLOCK has a reference to its next EVBLOCK. Note that an ordering of the events in the list, e.g. with respect to their synchronization values, is not required.

Each event can have a name and a comment, e.g. the user comment text or a description of the cause of the event. Additional information like a trigger condition, formatted text or custom information about the event can be added using an MDBLOCK for the comment.

The EVBLOCK can be used to express events which influenced the recording (e.g. begin, end or interruption), conditional events (i.e. triggers) or markers (e.g. comment related to a point or range). An event can be caused by an error, by a user interaction, by a scripting command or by a tool-internal condition. An overview of common use cases can be found in Table 28.

Each event has a "scope", i.e. a definition to which channels or channel groups it applies. The scope can be categorized in three levels: an event can have a global scope, i.e. it applies to the complete file (HD level), or it can apply to all channels in one or more channel groups (CG level), or only to one or more individual channels (CN level). The scope is expressed by a variable length list of links to the related CGBLOCKs or CNBLOCKs (mixture allowed, but usually an event either applies to single channels or complete channel groups). If the event applies to the complete file, this will be indicated by an empty list.

If two events define a range in time or some other synchronization domain, both events must have the same scope. To avoid redundancy, only the first event (beginning of range) defines its scope (i.e. stores the links of the "scope list"), and the second event must use the same scope as the first one. The only exception is if the event for the beginning of the range is missing, in which case the "end range" event must define its scope by itself.

Independent if an event defines a point or a range, it can be related to some other event, called its "parent" event. The parent relationship allows a hierarchical grouping of events. Two events defining a range must have the same parent event. Furthermore, the scope of the parent event must be larger than or equal to the scope of its child, i.e. when breaking down the scope to channels (i.e. listing all channels of the channel groups for CG level or all channels in the file for a global scope), the scope of the child must be a subset of the scope of the parent.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.12.1 Block structure of EVBLOCK

Table 27 Structure EVBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##EV”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
ev_ev_next	LINK	1	Link to next EVBLOCK (linked list) (can be NIL)
ev_parent (previously ev_ev_parent)	LINK	1	Referencing link to EVBLOCK with parent event (can be NIL). The parent relationship must not contain circular references. The scope of the parent event must be larger than or equal to the scope of the child event. If the child does not define its own scope, then the scope of the parent is used (see ev_scope). Note: in contrast to ev_ev_range, there is no restriction on the type of the parent event. Common use cases are that trigger or interrupt events have a recording (begin) event as parent. For a template EVBLOCK of an event signal group it is possible to reference a CGBLOCK containing all the parent signals. (see chapter 4.12.5 Event Signals for details)
ev_ev_range	LINK	1	Referencing link to EVBLOCK with event that defines the beginning of a range (can be NIL, must be NIL if ev_range_type ≠ 2). The event referenced by ev_ev_range and the current event define the borders of a range. ev_ev_range must define the beginning of the range (i.e. ev_range_type = 1) and the current event its end. This implies the following restrictions: ev_ev_range must have occurred prior to the current event, i.e. the (calculated) synchronization value for ev_ev_range must be smaller than for the current event. Furthermore, both events must have the same event type and sync type and the same parent, i.e. the values of ev_type, ev_sync_type and ev_parent must be

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			equal. In addition, both events must have the same scope, which is achieved by the rule, that the current event must re-use the scope list of ev_ev_range (see explanation for ev_scope). This avoids a duplication of the scope list.
ev_tx_name	LINK	1	Pointer to TXBLOCK with event name (can be NIL) Name must be according to naming rules stated in Naming Rules. If available, the name of a named trigger condition should be used as event name. Other event types may have individual names or no names.
ev_md_comment	LINK	1	Pointer to TX/MDBLOCK with event comment and additional information, e.g. trigger condition or formatted user comment text (can be NIL)
ev_scope	LINK	M	List of links to channels and channel groups to which the event applies (referencing links to CNBLOCKs or CGBLOCKs). This defines the "scope" of the event. The length of the list is given by ev_scope_count. It can be empty (ev_scope_count = 0). If the record index is used for synchronization (ev_sync_type = 1), then the scope must be less than or equal to one channel group, i.e. all affected channels must be in one channel group. If this event defines the end of a range (ev_range_type = 2) and references the event for the beginning of the range (ev_ev_range ≠ NIL), then the scope list must be empty and this event must use the scope list of the event referenced by ev_ev_range. Thus, both events have the same scope. If this event has a parent event (ev_parent ≠ NIL), and if its scope list is empty (and also the one of ev_ev_range ≠ NIL), then the scope list of ev_parent must be used. For all other cases, an empty scope list means that the event has a global scope, i.e. the event applies to the whole file.
ev_at_reference	LINK	N	List of attachments for this event (references to ATBLOCKs in global linked list of ATBLOCKs).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			The length of the list is given by ev_attachment_count. It can be empty (ev_attachment_count = 0), i.e. there are no attachments for this event.
ev_tx_group_name	LINK	1	**Only present if the "Group name present" (bit 1) flag is set.** Pointer to TXBLOCK with and arbitrary group name for the event. (can be NIL) Events with the same group name are for the same use case. For template EVBLOCKs of event signal groups, the group name must not be specified since the name of the event structure already defines the use case. (see chapter 4.12.5 Event Signals) *valid since MDF 4.2.0*
Data section			
ev_type	UINT8	1	Event type
			0 = Recording This event type specifies a recording period, i.e. the first and last time a signal value could theoretically be recorded. Recording events must always define a range, i.e. 1 ≤ ev_range_type ≤ 2. All recording events in the MDF file must only occur on exactly one scope level: either all have a global scope, e.g. if file has been created by a single recorder; or all of them have a scope on CG level (ev_scope list must contain links to one or more CGBLOCKs), e.g. after combining two files. Recording events which apply to single channels only (CN level scope) generally is not possible. Within one scope, there can be several recording periods, e.g. if the recording was paused between these periods (although this should be modeled by a recording interrupt event, see below). However, these periods must not overlap.
			1 = Recording Interrupt This event type indicates that the recording has been interrupted. It can only occur within the range of a matching recording period. Like the recording event type, it can only have a global scope or a CG level scope. It should be either on the same scope level as the recording event, or on a lower level, i.e. if the recording events have

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			a global scope, the recording interrupt event can either have a global or a CG level scope. A recording interrupt can be defined as point single event) or as range (pair of events as explained for ev_ev_range). If it defines a point, the interruption should be considered to automatically have finished when the next sample for a signal value for a channel in its scope level occurred. Since a recording interrupt event is closely related to a recording period, its ev_parent should point to the respective "range begin" recording event.
			2 = Acquisition Interrupt This event type indicates that not only the recording, but already the acquisition of the signal values has been interrupted. Except of this, the same rules apply as for the recording interrupt event type.
			3 = Start Recording Trigger This event type specifies an event which started the recording of signal values due to some condition (including user interaction). The trigger condition can be specified in the ev_md_comment MDBLOCK, (see Table 30). Here also a pre and post trigger interval can be specified. A start recording trigger event can only occur as point (ev_range_type = 0), not as range. It usually is closely related to a recording period, i.e. it should have the same scope as the respective recording events (note that due to the pre-trigger interval, the matching "range begin" recording event can be before this event). Here ev_parent may be used to point to the "range begin" recording event. For some other use case, ev_parent instead might point to another trigger event, e.g. if this trigger activated the condition for the start recording trigger.
			4 = Stop Recording Trigger Symmetrically to the "start recording trigger" event type, this event type specifies an event which stopped the recording of signal values due to some condition. The same rules apply as for the start recording trigger event type. Note that the two event types may occur in

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			pairs, but they do not have to.
			5 = Trigger This event type generally specifies an event that occurred due to some condition (except of the special start and stop recording trigger event types). The trigger condition can be specified in the ev_md_comment MDBLOCK, (see Table 30). A trigger event can only occur as point (ev_range_type = 0), not as range.
			6 = Marker This event type specifies a marker for a point or a range. As examples, a marker could be a user-generated comment or an automatically generated bookmark.
ev_sync_type	UINT8	1	Sync type 1 = calculated synchronization value represents time in seconds 2 = calculated synchronization value represents angle in radians 3 = calculated synchronization value represents distance in meter 4 = calculated synchronization value represents zero-based record index For ev_sync_type < 4, the scope of the event must either be global or it must only contain channel groups (or channels from channel groups) which are in the same synchronization domain, i.e. which contain a (virtual) master channel with matching cn_sync_type. In case of a global scope, the scope automatically is restricted to channel groups which are in the same synchronization domain. For ev_sync_type = 4, the scope of the event must be less than or equal to a single channel group, i.e. the channel group whose record index is used for synchronization. See also 4.4.6 Synchronization Domains.
ev_range_type	UINT8	1	Range type: 0 = event defines a point 1 = event defines the beginning of a range 2 = event defines the end of a range Point and range are defined in the synchronization domain defined by ev_sync_type.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

<table>
<tr><th>Name</th><th>Type</th><th>Count</th><th>Description</th></tr>
<tr><td></td><td></td><td></td><td>The following table shows the allowed combinations of ev_type and ev_range_type
<table>
<tr><th>ev_range_type / ev_type</th><th>0 = Point</th><th>1, 2 = Range</th></tr>
<tr><td>0 = Recording</td><td></td><td>x</td></tr>
<tr><td>1 = Recording Interrupt</td><td>x</td><td>x</td></tr>
<tr><td>2 = Acquisition Interrupt</td><td>x</td><td>x</td></tr>
<tr><td>3 = Recording Start Trigger</td><td>x</td><td></td></tr>
<tr><td>4 = Recording Stop Trigger</td><td>x</td><td></td></tr>
<tr><td>5 = Trigger</td><td>x</td><td></td></tr>
<tr><td>6 = Marker</td><td>x</td><td>x</td></tr>
</table></td></tr>
<tr><td rowspan="6">ev_cause</td><td rowspan="6">UINT8</td><td rowspan="6">1</td><td>Cause of event</td></tr>
<tr><td>0 = OTHER
cause of event is not known or does not fit into given categories.</td></tr>
<tr><td>1 = ERROR
event was caused by some error.</td></tr>
<tr><td>2 = TOOL
event was caused by tool-internal condition, e.g. trigger condition or re-configuration.</td></tr>
<tr><td>3 = SCRIPT
event was caused by a scripting command.</td></tr>
<tr><td>4 = USER
event was caused directly by user, e.g. user input or some other interaction with GUI.</td></tr>
<tr><td rowspan="3">ev_flags</td><td rowspan="3">UINT8</td><td rowspan="3">1</td><td>Flags
The value contains the following bit flags (Bit 0 = LSB):</td></tr>
<tr><td>Bit 0: Post processing flag
If set, the event has been generated during post processing of the file.</td></tr>
<tr><td>Bit 1: Group name present flag
If set, this indicates that an event group name is specified by means of the ev_tx_group_name link.
valid since MDF 4.2.0, should not be set for</td></tr>
</table>

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			earlier versions.
ev_reserved	BYTE	3	Reserved
ev_scope_count	UINT32	1	Length M of ev_scope list. Can be zero.
ev_attachment_count	UINT16	1	Length N of ev_at_reference list, i.e. number of attachments for this event. Can be zero.
ev_creator_index	UINT16	1	Creator index, i.e. zero-based index of FHBLOCK in global list of FHBLOCKs that specifies which application has created or changed this event (e.g. when generating event offline).
ev_sync_base_value	INT64	1	Base value for synchronization value. The synchronization value of the event is the product of base value and factor (ev_sync_base_value x ev_sync_factor). See also remark for ev_sync_factor. The calculated synchronization value depends on ev_sync_type: For ev_sync_type < 4, the synchronization value is a time/angle/distance value relative to the respective start value in HDBLOCK. Negative synchronization values can be used for events that occurred before the start value. For ev_sync_type = 4, the synchronization value is the absolute record index in the channel group specified by the scope. The event thus occurred at the same time as the record indicated by the index. In this case, the value must be rounded to an Integer value ≥ 0 and < cg_cycle_count, i.e. it must be less than the number of cycles specified for the channel group. See also 4.4.6 Synchronization Domains.
ev_sync_factor	REAL	1	Factor for event synchronization value. The event synchronization value is the product of base value and factor (ev_sync_base_value x ev_sync_factor). Generally, base value and factor can be chosen arbitrarily, but as recommendation they should be used to reflect the raw value and conversion factor of the master channel to be synchronized with. For instance, assume a time master channel using a 64-bit Integer channel data type to store the raw time value in nanoseconds using a linear conversion with offset 0 and factor 1e-9. In this case, ev_sync_factor could be set to 1e-9 and ev_sync_base_value could

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			store the nanosecond time stamp value. Thus, a higher precision is available than when just specifying the time value in seconds as REAL value. For ev_sync_type = 4, ev_sync_factor generally could be set to 1.0, so that the record index will be given by ev_sync_base_value.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.12.2 Common Use Cases for Events

Table 28 Common use cases for events

Event Type	cause	range(r) / point (p)	can have trigger formula	interpolation relevant	post processing flag (bit 0)	Scope level			used for
						HD	CG	CN	
Recording	tool	r		x	0	x	(1)		recording ranges, based on triggers
Recording	user/ script	r		x	0	x	(1)		recording ranges, started directly by user / script
AcqInterrupt	tool	(3)		x	0	x	x		interruption due to re-configuration
RecInterrupt	user/ script	(3)		x	0	(2)	x		user initiated pause of recording
AcqInterrupt	user/ script	(3)		x	0	(2)	x		user initiated pause of acquisition
AcqInterrupt	error	(3)		x	0	(2)	x		interruption due to device / transmission error
StartRecTrig	tool	p	x		0	x	(1)		recorder start trigger due to trigger condition
StartRecTrig	user/ script	p	x (4)		0	x	(1)		manual recorder start trigger
StopRecTrig	tool	p	x		0	x	(1)		recorder stop trigger due to trigger condition
StopRecTrig	user/ script	p	x (4)		0	x	(1)		manual recorder stop trigger
Trigger	tool	p	x		0	x	x	x	trigger for recorder, alarm, scripts due to condition
Trigger	user/ script	p	x (4)		0	x	x	x	manual trigger for recorder, alarm, scripts
Trigger	tool	p	x		1	x	x	x	find results (bookmarks based on trigger)
Marker	user	r/p			0	x			user generated recorded comments
Marker	user	r/p			1	x	x	x	user defined bookmarks

(1) Only if more than one recorder
(2) interruption is usually local to a group but can also be global
(3) range if end can be determined by tool, point if end is implicitly given by next sample
(4) manual triggers usually do not have a formula

"interpolation relevant" means that this event influences calculation of raster quality (see 4.24.1 Block Structure of SRBLOCK) and possibly the display of signal values as a graph.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.12.3 Example For Events

We assume a time-based measurement for which the recording has been started by a trigger condition and stopped by some other trigger condition. During the recording there has been an acquisition interrupt due to an error and a recording interrupt due to a pause initiated by the user.

The events are illustrated in Figure 6 as positions on the time axis. Red circles indicate samples for the measurement data (i.e. records in one or more channel groups).

The start recording trigger event (2) caused the beginning of the recording. Due to a pre-trigger interval, the event for the beginning of the recording range (1) is before the start recording trigger. Equally, the event for the end of the recording range (7) is after the stop recording trigger event (6) because of a post trigger interval. The events (1) and (7) define the recording range, and event (7) refers to event (1) using the ev_ev_range link.

During the recording, there first has been an acquisition interrupt (3) due to some error. The acquisition interrupt automatically is assumed to have finished with the next sample that occurred.

Later then a recording interrupt has occurred. Since the end of the interrupt is known, it has been modeled by a range defined by two recording interrupt events (4) and (5). Again, event (5) refers to event (4) using the ev_ev_range link.

Since all events are related to the recording range, events (2) – (6) all refer to event (1) using the ev_parent link (not indicated in picture).
Table 29 shows some properties of the events.

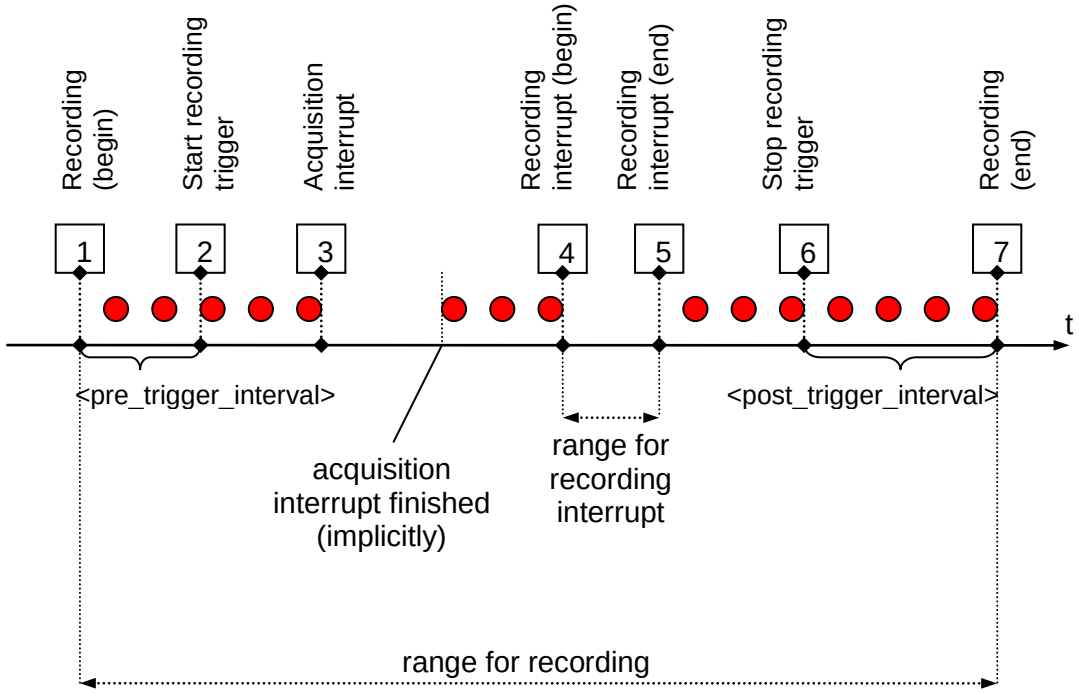


Figure 6 Example for events

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 29 Properties of events for example

event number	ev_type	ev_cause	ev_range_type	ev_parent points to	ev_ev_range points to	description
(1)	0 = recording	2 = tool	1 = range begin	NIL	NIL	begin of recording range initiated by tool due to start recording trigger and pre-trigger interval
(2)	3 = start recording trigger	3 = tool	0 = point	(1)	NIL	start recording trigger initiated by a condition, e.g. a certain signal value exceeds a specified limit (rising edge).
(3)	2 = acquisition interrupt	1 = error	0 = point	(1)	NIL	acquisition interrupt (as single point) due to an error (e.g. transmission error)
(4)	1 = recording interrupt	4 = user	1 = range begin	(1)	NIL	begin of a recording interrupt range initiated by user interaction (e.g. user clicks a button)
(5)	1 = recording interrupt	4 = user	2 = range end	(1)	(4)	end of a recording interrupt range initiated by user interaction (e.g. user clicks button once more)
(6)	4 = stop recording trigger	2 = tool	0 = point	(1)	NIL	stop recording trigger initiated by a condition, e.g. limit not exceeded by signal value any longer (falling edge).
(7)	0 = recording	2 = tool	2 = range end	NIL	(1)	end of recording range initiated by tool due to stop recording trigger and post-trigger interval.

4.12.4 Contents of the Meta Data Block for the Event Comment

The content of the MDBLOCK for the comment of the event is defined by the XML schema ev_comment.xsd. In addition to the standard tags it includes the following tags specific to the EVBLOCK.

Table 30 Tags for MDBLOCK ev_md_comment

Tag name	Description
<TX>	Comment text for event (default tag, required)
<pre_trigger_interval>	Pre-trigger interval for the event as configured in tool (optional) Unit depends on ev_sync_type. Only to be used for 3 ≤ ev_type ≤ 5.
<post_trigger_interval>	Post-trigger interval for the event as configured in tool (optional)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Tag name	Description	
	Unit depends on ev_sync_type. Only to be used for 3 ≤ ev_type ≤ 5.	
<formula>	Trigger condition in a formalized syntax (optional). Only to be used for 3 ≤ ev_type ≤ 5. The condition must evaluate to a Boolean value (true fires the trigger). For details see 5.5 Formula Specifications.	
<timeout>	Value used for a timeout condition (optional) Unit depends on ev_sync_type. Only to be used for 3 ≤ ev_type ≤ 5. ***Valid since MDF 4.1.0**, should not be used for earlier versions.* As the <formula> tag the <timeout> tag can be used to describe an alternative condition to invoke the event. The following attribute can be used to give additional information:	
	triggered	Boolean flag. If true, then the timeout condition "triggered", i.e. the timeout occurred and led to this event. Optional attribute, default value: "false"

4.12.5 Event Signals

Event signals present an alternative way for storing EVBLOCKs. They can be transformed to EVBLOCKs without any loss of information. The motivation is to increase the performance when accessing the file.

Event signals were introduced in MDF version 4.2.0 and must not be used in earlier MDF versions.

As described before, EVBLOCKs are stored in a single linked list. If a large number of events is written, this list becomes longer and spreads all over the file. Therefore, searching for an EVBLOCK located at the end of the file can take almost as much time as reading the file completely if no access information is cached.

The idea of event signals is to group events according to their use case and store them to a separate channel group called ‘event signal group’. One record described by this channel group corresponds to one event, i.e., one EVBLOCK.

The channels in this group describe all mutable values. The product of the synchronization values (ev_sync_base_value and ev_sync_factor) is stored to the master channel of the event signal group. Other mutable values are stored to component channels of a structure in the event signal group called ’event structure’. This way of storing events makes it possible to use the same index-based access as for signals – thus increasing the read performance.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

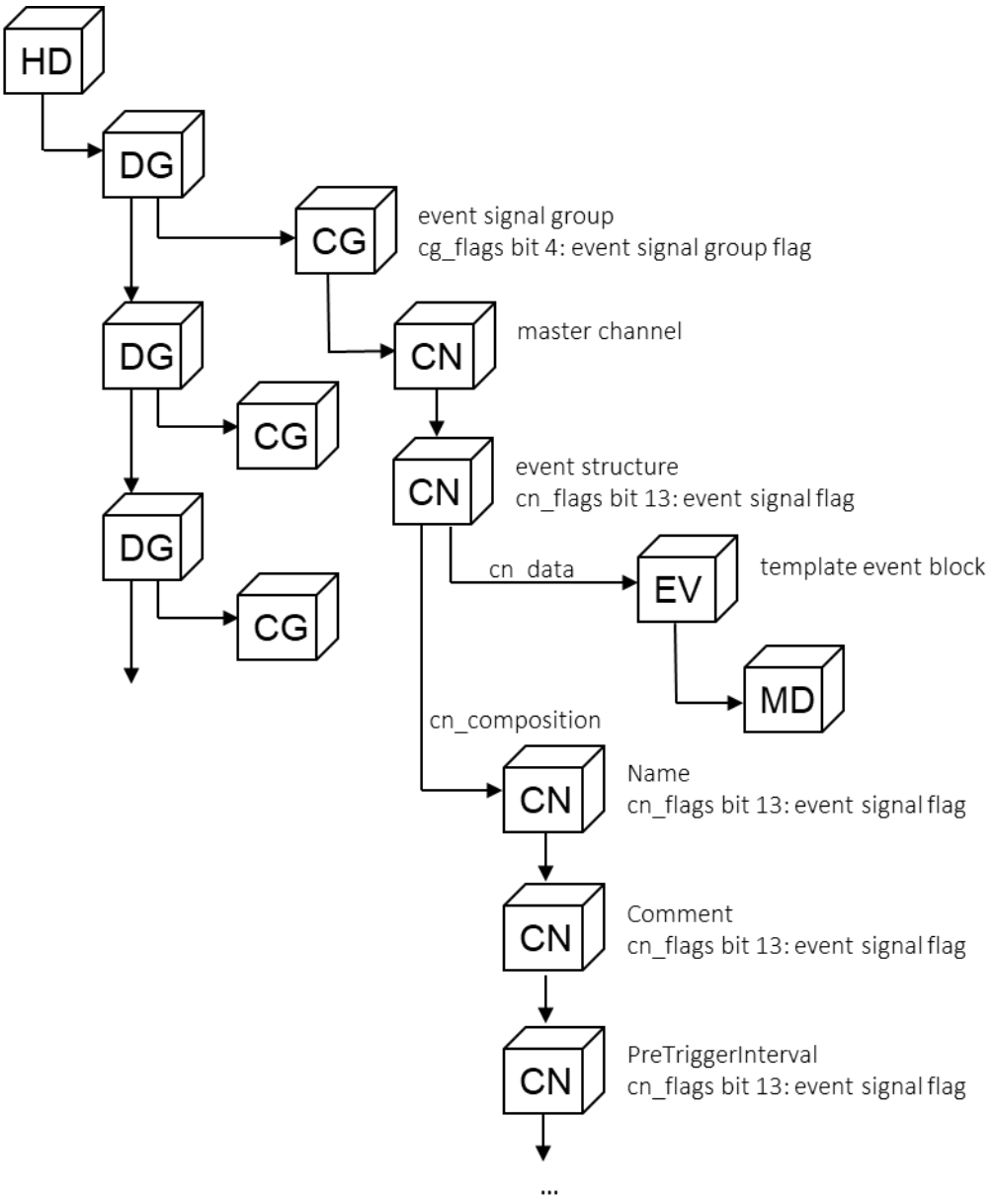


Figure 7 Structure of an event signal group

Event signal groups are marked by the event signal group flag. (cg_flags bit 4) They usually consist of a master channel and the event structure. The event structure and its component channels must be marked with the event signal flag. (cn_flags bit 13)

An event signal group can contain additional channels without event signal flag. These are ordinary channels that have the same raster as the event signals. (e.g., if an event is created if a key is pressed and this starts the measurement of an event-driven channel).

Via its cn_data link, the event structure references a single EVBLOCK which is not contained in the global linked list of EVBLOCKs. This EVBLOCK serves as a template for all constant values shared by all events in this event signal group. The mutable values are then defined by means of component channels of the event structure using a naming convention. The component channels are predefined in Table 31. All of them are optional. A value from the template EVBLOCK or its MDBLOCK must be ignored if the event structure contains a member channel for this value.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

The links in the template EVBLOCK are shared by all events of the event signal group and cannot be overwritten. However, there are the following exceptions:

- The ev_tx_name link can be overwritten by using a channel with a string data type.
- Most tags of the EVBLOCK's MDBLOCK referenced by the ev_md_comment link can be expressed by component channels.
- The ev_ev_range link can be described by the RangeIndex channel. It references the corresponding begin event by means of a relative index.
- The ev_parent link can be described by the ParentIndex channel. It references the corresponding event by means of a relative index.

The usage of ParentIndex and RangeIndex is described in more detail later in the chapter.

To identify which use case the event signal group is for, the event signal structure must be given a name by means of cn_tx_name which must be unique for all event signal structures in the file. If the event signals are transformed to EVBLOCKs, ev_tx_group_name should be used for this name instead.

The synchronization values ev_sync_base_value and ev_sync_factor are not sub-signals of the event signal structure because they are expressed by the master channel. The raw value of the master channel and the conversion can be used to achieve the same precision as expressed by ev_sync_base_value and ev_sync_factor.

For the sync type 4 (= Index), master channels are not allowed. Therefore, events with this sync type cannot be stored by means of event signals.

The following table describes all possible component channels of the event structure. The second column contains the data type supported for the channels' physical values.

Table 31 Component channels of an event signal structure

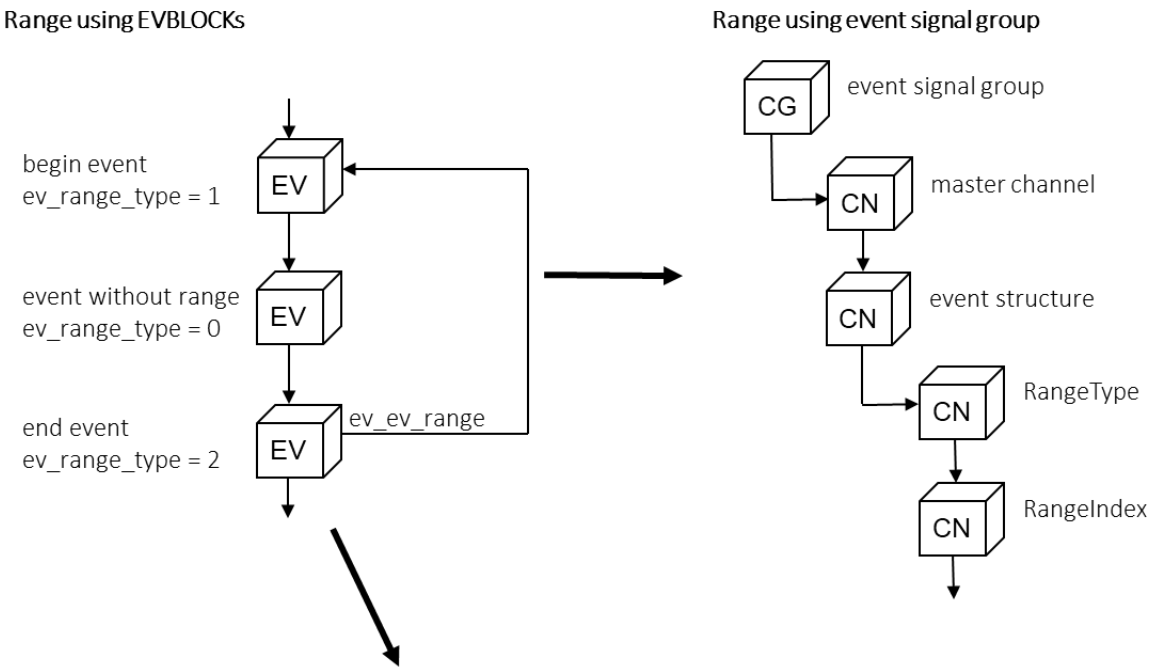
Name	Phys Data Type	Description
Name	string	Event name as described for ev_tx_name in the link section.
Type	unsigned integer	Event type as described for ev_type in the data section.
ParentIndex	signed integer	This value contains the relative index for the record with the corresponding parent event. If ev_parent of the template EVBLOCK references a CGBLOCK, this value contains the absolute index to the record in the event signal group referenced by ev_parent. Details are described below.
RangeType	unsigned integer	Event range type as described for ev_range_type in the data section.
RangeIndex	signed integer	Relative index referencing the corresponding event of a range. Details are described below.
Cause	unsigned integer	Event cause as described for ev_cause in the data section.
Flags	unsigned integer	Event flags as described for ev_flags in the data section.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Phys Data Type	Description	
CreatorIndex	unsigned integer	Creator index as described for ev_creator_index in the data section.	
Comment	string	Event comment as described for the <TX> tag of the event's MDBLOCK.	
PreTriggerInterval	floating-point	Pre-trigger interval as described for the <pre_trigger_interval> tag of the event's MDBLOCK.	
PostTriggerInterval	floating-point	Post-trigger interval as described for the <post_trigger_interval> tag of the event's MDBLOCK.	
Formula	string	Formula as described for the <formula> tag of the event's MDBLOCK.	
Timeout	floating-point	Timeout as described for the <timeout> tag of the event's MDBLOCK.	
TimeoutTriggered	unsigned integer	Boolean flag indicating whether the timeout occurred as described for the 'triggered' attribute at the <timeout> tag of the event's MDBLOCK.	
CommonProperties	Structure	Structure for <e> tags contained in the <common_properties> tag (5.2) of the event's MDBLOCK. If a component channel of this structure is a structure, it is considered as a <tree> tag. Scalar channels are considered as <e> tags. <elist> tags can be expressed by a channel linking a CABLOCK describing an array. The name of the member channel is equal to the 'name' attribute of the corresponding <e> tag, <elist> tag, or <tree> tag. <list> tags and <extension> tags (5.3) can be used only in the MDBLOCK of the template EVBLOCK.	
		The tags' attributes are expressed by members of the component channel as follows:	
		name	cn_tx_name
		type	cn_data_type
		desc	cn_md_comment → <TX>
		ci	cn_md_comment → <TX ci="1">
		xml:lang	cn_md_comment → <TX xml:lang="en-US">
		unit	cn_md_unit → <TX>
		unit_ref	cn_md_unit → <ho_unit>

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Figure 8 illustrates how event ranges are described using an event signal group.



Example for records described by event signal group

begin event	master channel	0.1 s
	RangeType	1 (begin)
	RangeIndex	0 (or 2)
event without range	master channel	0.2 s
	RangeType	0 (point)
	RangeIndex	0
end event	master channel	0.3 s
	RangeType	2 (end)
	RangeIndex	-2

Figure 8 Range described by an event signal group

If a record describes an end event (range type 2), the RangeIndex contains the relative index for the record where the respective begin event occurs. This implies that begin events and their respective end events must be stored to the same event signal group. A zero value for the end event indicates a missing begin event.

If a record describes a begin event (range type 1), the RangeIndex can reference the respective end event in the same way. This value is not mandatory for begin events since the relation is mainly described by the end event, i.e., a zero value can also be written if a respective end event exists.

It is possible that a relative range index references a record outside the valid range, i.e. the following condition is *not* met:

0 <= record index + range index < cg_cycle_count

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

This can be the case if an MDF file is split into multiple files without resetting the indices. Such cases are explicitly supported and must be treated as if the RangeIndex was 0, i.e., as if the event is missing.

The parent event can be measured in the same way as the range events by using the ParentIndex. This also implies that the parent event must be stored to the same event signal group. A zero value indicates a missing parent event.

It is possible to collect the parent events in a separate event signal group. This can be done by linking ev_parent of the template EVBLOCK to the CGBLOCK of this event signal group. In this special case, the ParentIndex is interpreted as an absolute index in the separate event signal group.

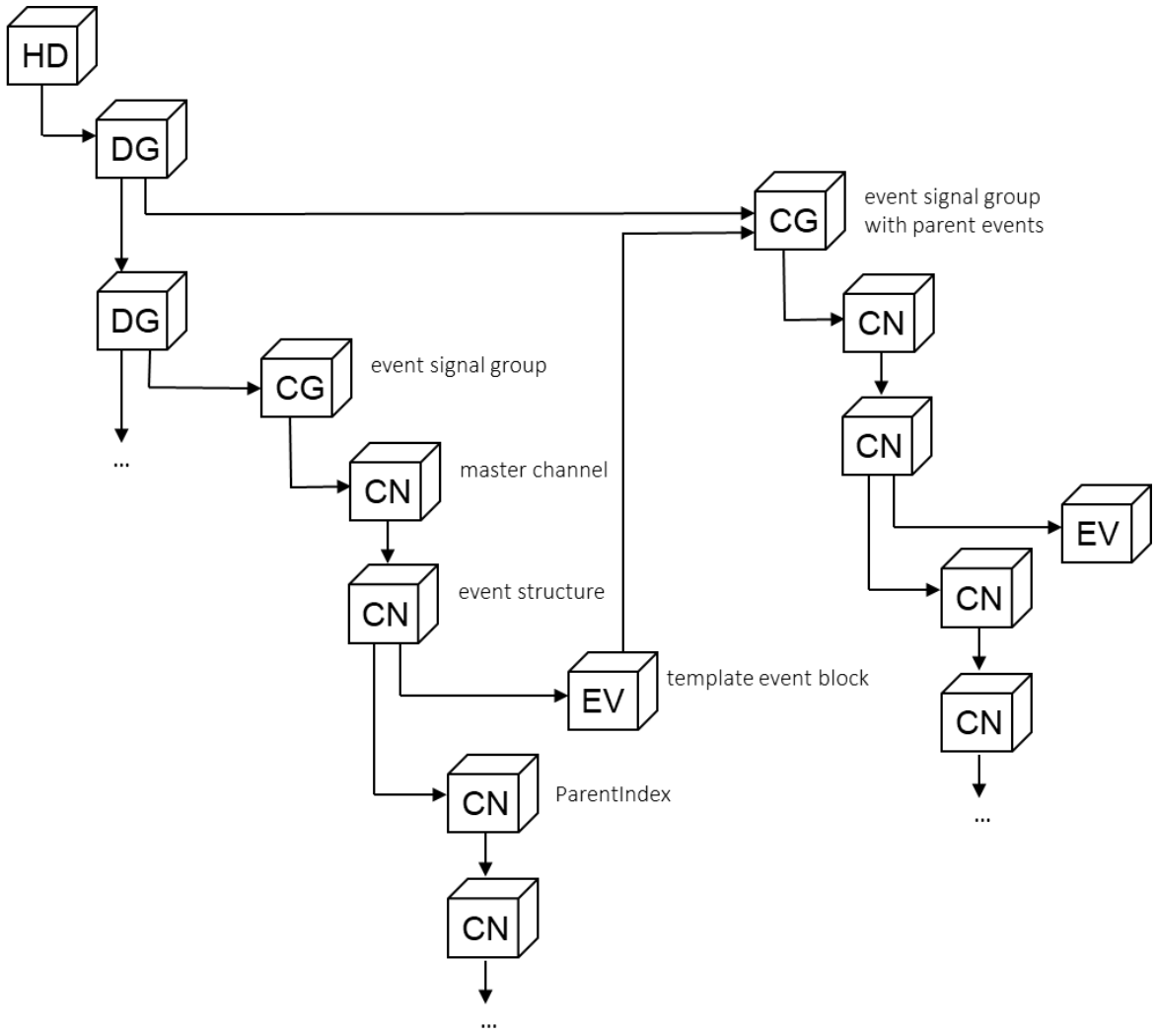


Figure 9 Referencing the parent from a separate event signal group

4.12.6 Documentation of Lost Time Synchronization

Using a time synchronization with an external or even global time source (hd_time_class > 0, see HDBLOCK), the synchronization source (e.g. GPS) may be lost temporarily or permanently, for instance when driving through a tunnel.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Typical implementations of the synchronization will try to continue the time recording in a smooth way, but cannot avoid a drift compared to the external synchronization source. After a re-established connection and switching back to the synchronization source, it is essential to avoid reverse time stamps which are not allowed in MDF. A smooth correction back to synchronized time stamps is desirable, but this may take a while until the time stamp quality is back within the tolerance of the measurement system.

This section does not try to show how to handle and correct a lost synchronization, but just wants to suggest a common practice how to document this loss and the time range where time stamps are "out of sync".

For this, we suggest using "range" events (pair of events, see ev_ev_range and ev_range_type) of type "marker" (ev_type = 6). As convention, a fixed group name "TimeSync Interrupt" is suggested (see ev_tx_group_name).

Using a naming convention instead of an event type has the advantage that this suggestion can also be applied to previous MDF 4.x versions where the event name (ev_tx_name) must be used instead of the group name (ev_tx_group_name) introduced in MDF 4.2.0.

The start of a TimeSync Interrupt (TSI) range should mark the instant when synchronization was lost, the end of the range the instant when the time stamps are in synchronization again. All time stamps outside such TSI ranges are required to be synchronous to the sync source (within tolerance of the measurement system).

As for all event ranges, either begin or end event may be missing because it occurred outside the recorded period. If all time stamps of the MDF file are out of sync (e.g. "TSI begin" occurred before start of, and "TSI end" after end of period recorded in file), this should be marked by using hd_time_class = 0.

4.13 The Data Group Block DGBLOCK

The DGBLOCK gathers information and links related to its data block. Thus the branch in the tree of MDF blocks that is opened by the DGBLOCK contains all information necessary to understand and decode the data block referenced by the DGBLOCK.
The DGBLOCK can contain several channel groups. In this case the data group (and thus the MDF file) is "unsorted". If there is only one channel group in the DGBLOCK, the data group is "sorted" (for details please refer to 4.21.3 Sorted and Unsorted Data).

4.13.1 Block Structure of DGBLOCK

Table 32 Structure of DGBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##DG”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
dg_dg_next	LINK	1	Pointer to next data group block (DGBLOCK)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			(can be NIL)
dg_cg_first	LINK	1	Pointer to first channel group block (CGBLOCK) (can be NIL)
dg_data	LINK	1	Pointer to data block (DT-/DVBLOCK or DZBLOCK for this block type) or data list block (DL-/LDBLOCK or HLBLOCK if required) (can be NIL) Note: if the child CGBLOCK references a remote master group (cg_cg_master), then only DV- or LDBLOCK is allowed.
dg_md_comment	LINK	1	Pointer to comment and additional information (TXBLOCK or MDBLOCK) (can be NIL)
Data section			
dg_rec_id_size	UINT8	1	Number of Bytes used for record IDs in the data block. 0 = data records without record ID (only possible for sorted data group) 1 = record ID (UINT8) before each data record 2 = record ID (UINT16, LE Byte order) before each data record 4 = record ID (UINT32, LE Byte order) before each data record 8 = record ID (UINT64, LE Byte order) before each data record Must be 0 in case dg_data references a LDBLOCK or a DVBLOCK.
dg_reserved	BYTE	7	Reserved

4.13.2 Contents of the Meta Data Block for the Data Group Comment

The content of the MDBLOCK for the comment of the data group is defined by the XML schema dg_comment.xsd. In addition to the standard tags it includes the following tag specific to the DGBLOCK.

Table 33 Tags for MDBLOCK dg_md_comment

Tag name	Description
<TX>	Comment for data group in plain text (default tag, required)

4.14 The Channel Group Block CGBLOCK

The CGBLOCK contains a collection of channels which are stored in one record, i.e. which have equal sampling.
The only exception is a channel group for variable length signal data (VLSD), called "VLSD channel group". It has no channel collection and can only occur in an unsorted data group. It describes signal values of variable length which are stored as variable length records in a normal DTBLOCK. For details of please refer to chapter 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.14.1 Block Structure of CGBLOCK

Table 34 Structure of CGBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##CG”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
cg_cg_next	LINK	1	Pointer to next channel group block (CGBLOCK) (can be NIL)
cg_cn_first	LINK	1	Pointer to first channel block (CNBLOCK) (can be NIL, must be NIL for VLSD CGBLOCK, i.e. if "VLSD channel group" flag (bit 0) is set)
cg_tx_acq_name	LINK	1	Pointer to acquisition name (TXBLOCK) (can be NIL, must be NIL for VLSD CGBLOCK)
cg_si_acq_source	LINK	1	Pointer to acquisition source (SIBLOCK) (can be NIL, must be NIL for VLSD CGBLOCK) See also rules for uniqueness explained in 4.4.3 Identification of Channels.
cg_sr_first	LINK	1	Pointer to first sample reduction block (SRBLOCK) (can be NIL, must be NIL for VLSD CGBLOCK)
cg_md_comment	LINK	1	Pointer to comment and additional information (TXBLOCK or MDBLOCK) (can be NIL, must be NIL for VLSD CGBLOCK)
cg_cg_master	LINK	1	**Only present if the “remote master” (bit 3) flag is set.** This link points to another cg that must contain a master channel and must have the same cg_cycle_count. The channels in this channel group are to be treated as if they were in the channel group linked by this link. This can be used to optimize for reading by splitting up the record into multiple smaller records. If a client then read one of the channels of this group, not every signal value needs to be read. If each CGBLOCK contains only 1 channel, that group is said to be in column-oriented storage.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			The second use case for using this link is to add additional signals to the group (e.g. calculated signals during post-processing). Groups using that link must be stored either using LDBLOCKs or using a single DVBLOCK. They can only be sorted. Further details see: 4.14.3 Remote Master Link *Valid since MDF 4.2.0.*
Data section			
cg_record_id	UINT64	1	Record ID, value must be less than maximum unsigned integer value allowed by dg_rec_id_size in parent DGBLOCK. Record ID must be unique within linked list of CGBLOCKs.
cg_cycle_count	UINT64	1	Number of cycles, i.e. number of samples for this channel group. This specifies the number of records of this type in the data block.
cg_flags	UINT16	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: VLSD channel group flag. If set, this is a "variable length signal data" (VLSD) channel group. See explanation in 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK.
			Bit 1: Bus event channel group flag If set, this channel group contains information about a bus event, i.e. it contains a structure channel with bit 10 (bus event falg) set in cn_flags. For details please refer to MDF Bus Logging [7]. *valid since MDF 4.1.0, should not be set for earlier versions*

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Bit 2: Plain bus event channel group flag Only relevant if "bus event channel group" flag (bit 1) is set. If set, this indicates that only the plain bus event is stored in this channel group, but no channels describing the signals transported in the payload of the bus event. If not set, at least one channel for a signal transported in the payload of the bus event (data frame/PDU) must be present. For details please refer to MDF Bus Logging [7]. *valid since MDF 4.1.0, should not be set for earlier versions*
			Bit 3: Remote master flag If set, this indicates that the channel group uses the master values of another channel group. That remote master group is linked by the cg_cg_master link. A group designated with this flag must be stored using LDBLOCKs or a single DVBLOCK. *valid since MDF 4.2.0, should not be set for earlier versions.*
			Bit 4: Event signal group flag If set, this indicates that the channel group is for storing events, not for storing measurement signals. See chapter 4.12.5 **Event Signals** for details. *valid since MDF 4.2.0, should not be set for earlier versions*
cg_path_separator	UINT16	1	Value of character to be used as path separator, 0 if no path separator specified. The specified value is the UTF-16 Little Endian encoding of the character (no zero termination). Note: UTF-16 is used instead of UTF-8 to restrict the size to two Bytes. Commonly used characters are: - Dot (.): 0x002E (dec: 46) - Slash (/): 0x002F (dec: 47) - Backslash(\): 0x005c (dec: 92) The path separator character can be specified if one of the following strings (see

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Table 9) is composed of several parts which are separated by the given character: • group name (gn), group source (gs), and group path (gp) for this channel group • channel name (cn), channel source (cs), and channel path (cp) for a channel in this channel group *valid since MDF 4.1.0, should be 0 for earlier versions*
cg_reserved	BYTE	4	Reserved.
cg_data_bytes	UINT32	1	Normal CGBLOCK: Number of data Bytes (after record ID) used for signal values in record, i.e. size of plain data for each recorded sample of this channel group. VLSD CGBLOCK: Low part of a UINT64 value that specifies the total size in Bytes of all variable length signal values for the recorded samples of this channel group. See explanation for cg_inval_bytes.
cg_inval_bytes	UINT32	1	Normal CGBLOCK: Number of additional Bytes for record used for invalidation bits. Can be zero if no invalidation bits are used at all. Invalidation bits may only occur in the specified number of Bytes after the data Bytes, not within the data Bytes that contain the signal values. VLSD CGBLOCK: High part of UINT64 value that specifies the total size in Bytes of all variable length signal values for the recorded samples of this channel group, i.e. the total size in Bytes can be calculated by cg_data_bytes + (cg_inval_bytes << 32) Note: this value does not include the Bytes used to specify the length of each VLSD value!

4.14.2 Contents of the Meta Data Block for the Channel Group Comment

The content of the MDBLOCK for the comment of the channel group is defined by the XML schema cg_comment.xsd. In addition to the standard tags it includes the following tags specific to the CGBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 35 Tags for MDBLOCK cg_md_comment

Tag name	Description
<TX>	Comment for the channel group (default tag, required).
<names>	Alternative acquisition names and description as defined in 5.4 Alternative Names (optional).

4.14.3 Remote Master Link

The remote master link is intended for two main use-cases:

- Adding new (calculated) signals to an existing group without rewriting the file
- Improving read-access to single signal / small signal groups that are part of larger groups of signals. The case where each CGBLOCK contains only a single signal is called “column-oriented storage”.

To achieve this the remote master link allows splitting groups into multiple smaller groups that still all belong to the same logical group but do not necessarily share the same storage location anymore. The logical group is formed around a main CGBLOCK. That CGBLOCK contains the main master channel. All other CGBLOCKs reference this main CGBLOCKs using the remote master link.

These are the pre-requisites when using the remote master links:

- The group that has the remote master link set, must use LDBLOCKs or a single DVBLOCK to store the data.
- Due to this, split records are not allowed.
- The logical group must not contain multiple master channels of the same domain.
- The following fields have to be synchronized across all CGBLOCKs participating in the logical group:
  - cg_tx_acq_name: all CGBLOCKs participating in the same logical group must point to the same block.
  - cg_si_acq_source: all CGBLOCKs participating in the same logical group must point to the same block.
  - cg_md_comment: all CGBLOCKs participating in the same logical group must point to the same block.
  - cg_cycle_count: must be the same across all CGBLOCKs participating in the logical group
  - cg_path_separator: must be the same across all CGBLOCKs participating in the logical group

It is also a good practice to have the master channel as first channel in the linked group.

4.14.4 Variable Length Signal Data (VLSD) CGBLOCK

A variable length data channel (cn_type = 1, see 4.16 The Channel Block CNBLOCK) can save signal values with variable length like strings. Normally, these are stored in a signal data block (SDBLOCK) as explained in section 4.28 The Signal Data Block SDBLOCK.

However, an alternative way for storing signal values with variable length has been devised in order to allow an easier implementation and a better performance when writing an MDF file in streaming mode. It allows appending of signal values with variable length to a data block (DTBLOCK) like ordinary data records. This simplifies the streaming of data, especially for logger devices with limited capabilities to buffer the data.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

As we will see later, this way of storing variable length signal data (VLSD) values by definition results in an unsorted data group (and thus an unsorted MDF file). Sorting the data group for faster read access, the VLSD records must be converted into a normal signal data block (SDBLOCK).

In contrast to a normal data record in a DTBLOCK, a VLSD record does not have a fixed length. After the record ID, a 32-bit unsigned integer value (LE/Intel Byte order) specifies the length of the subsequently stored variable length signal value. A VLSD record contains exactly one signal value and has no invalidation Bytes.

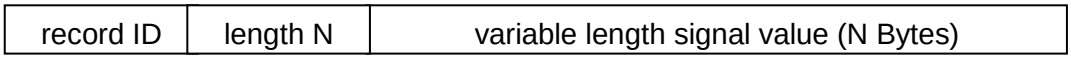


Figure 10 Layout of a VLSD record

A unique record ID is required to identify the VLSD records in the DTBLOCK of the unsorted data group. This record ID will be provided by a so-called "VLSD CGBLOCK".

A VLSD CGBLOCK is indicated by the respective flag (bit 0) in cg_flags. Note that this channel group has no child channels, i.e. cg_cn_first must be NIL. The connection between the VLSD CGBLOCK and the channel of the variable length signal data is established by a reference to the VLSD CGBLOCK using the cn_data link of the CNBLOCK (see Figure 10). The parent CGBLOCK of the CNBLOCK will also be denoted as "parent CGBLOCK" of the VLSD CGBLOCK.

The VLSD CGBLOCK and its parent CGBLOCK must be siblings of the same data group. Hence, a data group containing a VLSD CGBLOCK is unsorted by definition. When sorting the data group, the VLSD CGBLOCK must be replaced by an SDBLOCK.

For the VLSD CGBLOCK, the following rules apply:

- the "VLSD channel group" flag bit(0) must be set in cg_flags.
- all links must be NIL except of cg_cg_next.
- cg_cycle_count must be equal for the VLSD CGBLOCK and its parent CGBLOCK.
- the members that normally specifies the fixed number of data Bytes and invalidation Bytes (cg_data_bytes and cg_inval_bytes) are considered as low and high part of a UINT64 value which can be calculated by (cg_data_bytes + cg_inval_bytes<<32). This value holds the total sum of Bytes of all VLSD values for this channel in the recorded samples (not including the Bytes for the UINT32 length value).

A VLSD record does not contain a time stamp or any other synchronization value. It occurs simultaneous to the respective record for the parent CGBLOCK with the same record index, i.e. the Nth VLSD record corresponds to the Nth record of the parent CGBLOCK. This means that the VLSD CGBLOCK and its parent CGBLOCK are synchronized using their record index. Since the data group is unsorted, the variable length records of the VLSD CGBLOCK and the fixed length records of the parent CGBLOCK will be stored in the same data block (order is not relevant).

Note: Regarding the content of the fixed length records of the parent CGBLOCK, there is no difference between storing the VLSD values in an SDBLOCK or in VLSD records, i.e. the offset value must be filled in correctly for them. The benefit is that the fixed-length records do not have to be changed when sorting the data group.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.14.5 Example using a VLSD CGBLOCK

Figure 11 shows the usage of a VLSD CGBLOCK in an unsorted data group. Sorting this data group, the variable length part (i.e. the VLSD value) of the records with ID 2 will be collected and added as data section of an SDBLOCK as shown in Figure 12:

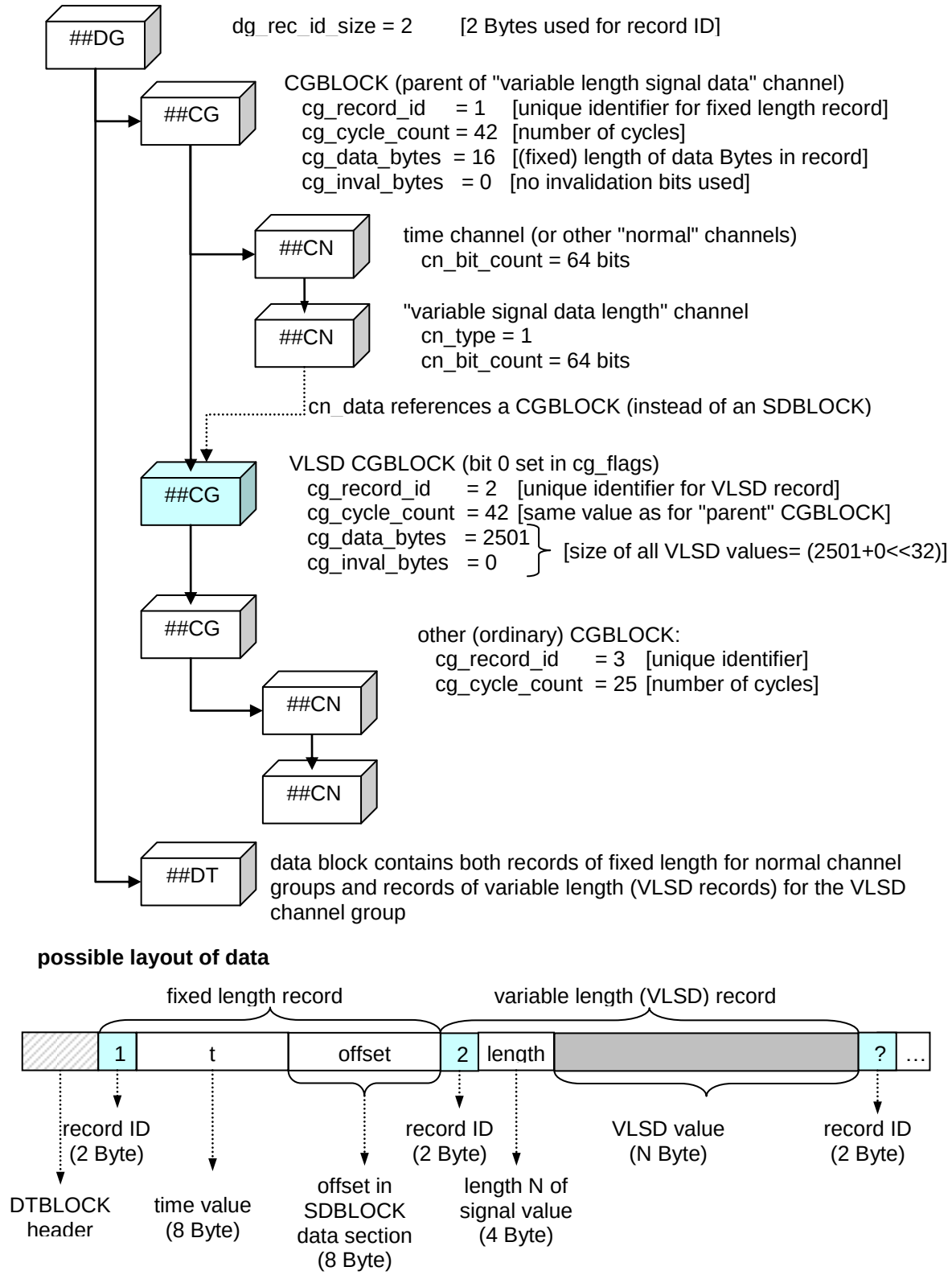


Figure 11 Example for usage of VLSD CGBLOCK

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

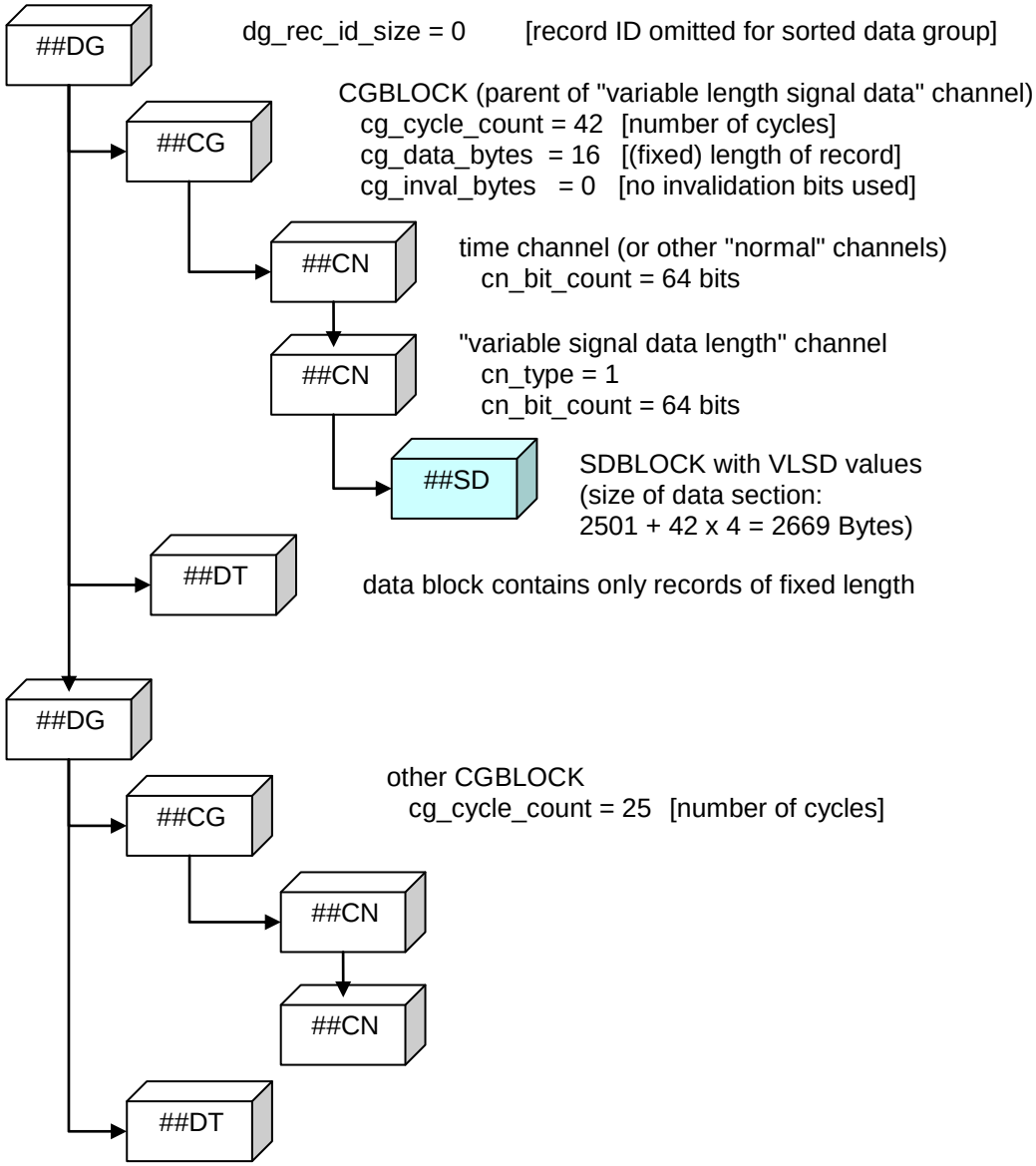


Figure 12 SDBLOCK replacing the VLSD CGBLOCK after sorting the data group

4.15 The Source Information Block SIBLOCK

The SIBLOCK describes the source of an acquisition mode or of a signal. The source information is also used to ensure a unique identification of a channel as explained in section 4.4.3 Identification of Channels.

Note that several channels may have the same source and thus might share the same SIBLOCK. The source of the acquisition mode and of a channel can be equal, too.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.15.1 Block Structure of SIBLOCK

Table 36 Structure of SIBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##SI”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
si_tx_name	LINK	1	Pointer to TXBLOCK with name (identification) of source (can be NIL). The source name must be according to naming rules stated in Naming Rules.
si_tx_path	LINK	1	Pointer to TXBLOCK with (tool-specific) path of source (can be NIL). The path string must be according to naming rules stated in Naming Rules. Each tool may generate a different path string. The only purpose is to ensure uniqueness as explained in section 4.4.3 Identification of Channels. As a recommendation, the path should be a human readable string containing additional information about the source. However, the path string should not be used to store this information in order to retrieve it later by parsing the string. Instead, additional source information should be stored in generic or custom XML fields in the comment MDBLOCK si_md_comment.
si_md_comment	LINK	1	Pointer to source comment and additional information (TXBLOCK or MDBLOCK) (can be NIL)
Data section			
si_type	UINT8	1	Source type additional classification of source: 0 = OTHER source type does not fit into given categories or is unknown 1 = ECU source is an ECU 2 = BUS source is a bus (e.g. for bus monitoring) 3 = I/O source is an I/O device (e.g. analog I/O) 4 = TOOL source is a software tool (e.g. for tool generated signals/events) 5 = USER source is a user interaction/input (e.g. for user generated events)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
si_bus_type	UINT8	1	Bus type additional classification of used bus (should be 0 for si_type ≥ 3): 0 = NONE no bus 1 = OTHER bus type does not fit into given categories or is unknown 2 = CAN 3 = LIN 4 = MOST 5 = FLEXRAY 6 = K_LINE 7 = ETHERNET 8 = USB Vendor defined bus types can be added starting with value 128.
si_flags	UINT8	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: simulated source Source is only a simulation (can be hardware or software simulated) Cannot be set for si_type = 4 (TOOL).
si_reserved	BYTE	5	reserved

4.15.2 Contents of the Meta Data Block for the Source Comment

The content of the MDBLOCK for the comment of the source is defined by the XML schema si_comment.xsd. In addition to the standard tags it includes the following tags specific to the SIBLOCK. Information about the measurement environment can be added in <common_properties> as described in [9].

Table 37 Tags for MDBLOCK si_md_comment

Tag name	Description
<TX>	Comment for source in plain text (default tag, required)
<names>	Alternative names and description for the source name (si_tx_name) as defined in 5.4 Alternative Names. (optional)
<path>	Alternative names and description for the path string (si_tx_path) as defined in 5.4 Alternative Names. (optional)
<bus>	Alternative names and description of the bus (e.g. CAN1) as defined in 5.4 Alternative Names. The actual bus name is given by attribute "name" of <bus> tag. (optional)
<protocol>	Name of protocol (e.g. XCP) (optional)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.16 The Channel Block CNBLOCK

The CNBLOCK describes a channel, i.e. it contains information about the recorded signal and how its signal values are stored in the MDF file.

4.16.1 Block Structure of CNBLOCK

Table 38 Structure of CNBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##CN”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
cn_cn_next	LINK	1	Pointer to next channel block (CNBLOCK) (can be NIL)
cn_composition	LINK	1	Composition of channels: Pointer to channel array block (CABLOCK) or channel block (CNBLOCK) (can be NIL). Details see 4.18 Composition of Channels .
cn_tx_name	LINK	1	Pointer to TXBLOCK with name (identification) of channel. Name must be according to naming rules stated in Naming Rules. The combination of name and source name and path (both from SIBLOCK) must be unique within all channels of this channel group. See also 4.4.3 Identification of Channels. Note: Alternative names (e.g. display name) can be stored in MDBLOCK of cn_md_comment, see Table 41.
cn_si_source	LINK	1	Pointer to channel source (SIBLOCK) (can be NIL) Must be NIL for component channels (members of a structure or array elements) because they all must have the same source and thus simply use the SIBLOCK of their parent CNBLOCK (direct child of CGBLOCK). See also 4.4.3 Identification of Channels.
cn_cc_conversion	LINK	1	Pointer to the conversion formula (CCBLOCK) (can be NIL, must be NIL for complex channel data types, i.e. for cn_data_type ≥ 10). If the pointer is NIL, this means that a 1:1 conversion is used (phys = int).
cn_data	LINK	1	Pointer to channel type specific signal data

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			• For variable length data channel (cn_type = 1): unique link to signal data block (SDBLOCK or DZBLOCK for this block type) or data list block (DLBLOCK or HLBLOCK if required) or, only for unsorted data groups, referencing link to a VLSD channel group block (CGBLOCK). Can only be NIL if SDBLOCK would be empty. • For synchronization channel (cn_type = 4): referencing link to attachment block (ATBLOCK) in global linked list of ATBLOCKs starting at hd_at_first. Cannot be NIL. • For maximum length data channel (cn_type = 5): referencing link to channel block (CNBLOCK) in same channel group. Cannot be NIL. • For the event signal structure referencing link to the template event block (EVBLOCK). Cannot be NIL. See chapter 4.12.5 Event Signals for details. Must be NIL in all other cases. See respective channel type for further details.
cn_md_unit	LINK	1	Pointer to TXBLOCK/MDBLOCK with designation for physical unit of signal data (after conversion) or (only for channel data types "MIME sample" and "MIME stream") to MIME context-type text. (can be NIL). • The unit can be used if no conversion rule is specified or to overwrite the unit specified for the conversion rule (e.g. if a conversion rule is shared between channels). If the link is NIL, then the unit from the conversion rule must be used. If the content is an empty string, no unit should be displayed. If an MDBLOCK is used, in addition the HDO unit definition can be stored, see Table 42. Note: for (virtual) master and synchronization channels the HDO definition should be omitted to avoid redundancy. Here the unit is already specified by cn_sync_type of the channel. • In case of channel data types "MIME sample" and "MIME stream", the text of the unit must be the content-type text of a MIME type which specifies the content of the values of the channel (either fixed length in record

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			or variable length in SDBLOCK). The MIME content-type string must be written in lowercase, and it must apply to the same rules as defined for at_tx_mimetype in 4.11 The Attachment Block ATBLOCK.
cn_md_comment	LINK	1	Pointer to TXBLOCK/MDBLOCK with comment and additional information about the channel, see Table 41. (can be NIL)
cn_at_reference	LINK	N	List of attachments for this channel (references to ATBLOCKs in global linked list of ATBLOCKs). The length of the list is given by cn_attachment_count. It can be empty (cn_attachment_count = 0), i.e. there are no attachments for this channel. *valid since MDF 4.1.0*
cn_default_x	LINK	3 or empty	**Only present if "default X" flag (bit 12) is set.** Reference to channel to be preferably used as X axis. The reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel (none of them must be NIL). The referenced channel does not need to have the same raster nor monotonously increasing values. It can be a master channel, e.g. in case several master channels are present. In case of different rasters, visualization may depend on the interpolation method used by the tool. In case no default X channel is specified, the tool is free to choose the X axis; usually a master channels would be used. *valid since MDF 4.1.0*
Data section			
cn_type	UINT8	1	Channel type
			0 = fixed length data channel channel value is contained in record.
			1 = variable length data channel also denoted as "variable length signal data" (VLSD) channel The channel value in the record is an unsigned integer value (LE/Intel Byte order) of the specified number of bits. This value is the offset to a variable length signal value in the data section of the SDBLOCK referenced by cn_data. Channel data type and CC rule refer to data in SDBLOCK, not to record data.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Signal data must point to an SDBLOCK or a DLBLOCK with list of SDBLOCKs. For unsorted data groups, alternatively it can point to a VLSD CGBLOCK, see explanation in 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK.
			2 = master channel for all signals of this group Must be combined with one of the following sync types: 1,2,3 (see table line for cn_sync_type). In each channel group, not more than one channel can be defined as master or virtual master channel for each possible sync type. Value decoding is equal to a fixed length data channel. Physical values of this channel must return the SI unit (see [19]) of the respective sync type, with or without CCBLOCK, i.e. seconds for a time master channel. Values of this channel listed over the record index must be monotonously increasing[4]. They are always relative to the respective start value in the HDBLOCK. See also section 4.4.6 Synchronization Domains.
			3 = virtual master channel Like a master channel, except that the channel value is not contained in the record (cn_bit_count must be zero). Instead the physical value must be calculated by feeding the zero-based record index to the conversion rule. The data type of the virtual master channel must be unsigned integer with Little Endian byte order (cn_data_type = 0). Except of this, the same rules apply as for a master channel (cn_type = 2).
			4 = synchronization channel Must be combined with one of the following sync types: 1,2,3,4 (see table line for cn_sync_type). A synchronization channel is used to synchronize the records of this channel group with samples in some other stream, e.g. AVI frames. Physical values of this channel must return the unit of the respective synchronization domain, i.e. seconds for cn_sync_type = 1, or an index value for cn_sync_type = 4. These values are used for synchronization with the

[4] A non-strictly monotonously increasing master channel is an indicator for an error in the measurement setup in most cases though.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			stream. Hence, the stream must use the same synchronization domain, i.e. a data row with synchronization values or index-based samples (only for cn_sync_type = 4). The signal data link (cn_data) must refer to an ATBLOCK in the global list of attachments. The ATBLOCK contains the stream data or a reference to a stream file. See also section 4.4.6 Synchronization Domains.
			5 = maximum length data channel also denoted as "maximum length signal data" (MLSD) channel Record contains range with maximum number of Bytes for channel value like for a fixed length data channel (cn_type = 0), but the number of currently valid Bytes is given by the value of a channel referenced by cn_data (denoted as "size signal"). Since the size signal defines the number of Bytes to use, its physical values must be Integer values ≥ 0 and ≤ (cn_bit_count>>3), i.e. its values must not exceed the number of Bytes reserved in the record for this channel. Usually, the size signal should have some Integer data type (cn_data_type ≤ 3) without conversion rule and without unit. The value of the size signal must be contained in the same record as the current channel, i.e. both channels must be in the same channel group. Note: this channel type must not be used with numeric or CANopen data types, i.e. it can occur only for (6 ≤ cn_data_type ≤ 12). *valid since MDF 4.1.0, should not occur for earlier versions*
			6 = virtual data channel Similar to a virtual master channel the channel value is not contained in the record (cn_bit_count must be zero). Instead the physical value must be calculated by feeding the zero-based record index to the conversion rule. The data type of the virtual master channel must be unsigned integer with Little Endian byte order (cn_data_type = 0). Except of this, the same rules apply as for a fixed length data channel (cn_type = 0). A virtual data channel may be used to specify a

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

<table>
<tr><th>Name</th><th>Type</th><th>Count</th><th>Description</th></tr>
<tr><td></td><td></td><td></td><td>channel with constant values without consuming any space in the record. The constant value can be given by the offset of a linear conversion rule with factor equal to zero.

valid since MDF 4.1.0, should not occur for earlier versions</td></tr>
<tr><td>cn_sync_type</td><td>UINT8</td><td>1</td><td>Sync type:
0 = None (to be used for normal data channels)
1 = Time (physical values must be seconds)
2 = Angle (physical values must be radians)
3 = Distance (physical values must be meters)
4 = Index (physical values must be zero-based index values)
See also section 4.4.6 Synchronization Domains.
The possible combinations of cn_sync_type with cn_type are listed by the following table:

<table>
<tr><th>cn_type \ cn_sync_type</th><th>0 = None</th><th>1 = Time</th><th>2 = Angle</th><th>3 = Distance</th><th>4 = Index</th></tr>
<tr><td>0 = fixed length data channel</td><td>x</td><td></td><td></td><td></td><td></td></tr>
<tr><td>1 = variable length data channel</td><td>x</td><td></td><td></td><td></td><td></td></tr>
<tr><td>2 = master channel</td><td></td><td>x</td><td>x</td><td>x</td><td></td></tr>
<tr><td>3 = virtual master channel</td><td></td><td>x</td><td>x</td><td>x</td><td></td></tr>
<tr><td>4 = synchronization channel</td><td></td><td>x</td><td>x</td><td>x</td><td>x</td></tr>
<tr><td>5 = maximum length data channel</td><td>x</td><td></td><td></td><td></td><td></td></tr>
<tr><td>6 = virtual data channel</td><td>x</td><td></td><td></td><td></td><td></td></tr>
</table>
</td></tr>
<tr><td>cn_data_type</td><td>UINT8</td><td>1</td><td>Channel data type of raw signal value
Integer data types:
0 = unsigned integer (LE Byte order)
1 = unsigned integer (BE Byte order)
2 = signed integer (two’s complement) (LE Byte order)
3 = signed integer (two’s complement) (BE Byte order)
Floating-point data types:
4 = IEEE 754 floating-point format (LE Byte order)
5 = IEEE 754 floating-point format (BE Byte order)</td></tr>
</table>

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			String data types: 6 = string (SBC, standard ISO-8859-1 encoded (Latin), NULL terminated) 7 = string (UTF-8 encoded, NULL terminated) 8 = string (UTF-16 encoded LE Byte order, NULL terminated) 9 = string (UTF-16 encoded BE Byte order, NULL terminated) **Complex data types:** 10 = byte array with unknown content (e.g. structure) 11 = MIME sample (sample is Byte Array with MIME content-type specified in cn_md_unit) 12 = MIME stream (all samples of channel represent a stream with MIME content-type specified in cn_md_unit) 13 = CANopen date (Based on 7 Byte CANopen Date data structure, see Table 39) 14 = CANopen time (Based on 6 Byte CANopen Time data structure, see Table 40) 15 = complex number (real part followed by imaginary part, stored as two floating-point data, both with 2, 4 or 8 Byte, LE Byte order) *valid since MDF 4.2.0* 16 = complex number (real part followed by imaginary part, stored as two floating-point data, both with 2, 4 or 8 Byte, BE Byte order) *valid since MDF 4.2.0*
cn_bit_offset	UINT8	1	Bit offset (0-7): first bit (=LSB) of signal value after Byte offset has been applied (see 4.21.5.2 Reading the Signal Value). If zero, the signal value is 1-Byte aligned. A value different to zero is only allowed for Integer data types (cn_data_type ≤ 3) and if the Integer signal value fits into 8 contiguous Bytes (cn_bit_count + cn_bit_offset ≤ 64). For all other cases, cn_bit_offset must be zero.
cn_byte_offset	UINT32	1	Offset to first Byte in the data record that contains bits of the signal value. The offset is applied to the plain record data, i.e. skipping the record ID.
cn_bit_count	UINT32	1	Number of bits for signal value in record
cn_flags	UINT32	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: All values invalid flag If set, all values of this channel are invalid. If in addition an invalidation bit is used (bit 1 set), then the value of the invalidation bit must be set

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			(high) for every value of this channel. Must not be set for a master channel (channel types 2 and 3).
			Bit 1: Invalidation bit valid flag If set, this channel uses an invalidation bit (position specified by cn_inval_bit_pos). Must not be set if cg_inval_bytes is zero. Must not be set for a master channel (channel types 2 and 3).
			Bit 2: Precision valid flag If set, the precision value for display of floating-point values specified in cn_precision is valid and overrules a possibly specified precision value of the conversion rule (cc_precision).
			Bit 3: Value range valid flag If set, both the minimum and the maximum raw value that occurred for this signal within the samples recorded in this file are known and stored in cn_val_range_min and cn_val_range_max. Otherwise the two fields are not valid. Note: the raw value range can only be expressed for numeric channel data types (cn_data_type ≤ 5). For all other data types, the flag must not be set.
			Bit 4: Limit range valid flag If set, the limits of the signal value are known and stored in cn_limit_min and cn_limit_max. Otherwise the two fields are not valid. (see MCD-2 MC [1] keywords LowerLimit/UpperLimit for MEASUREMENT and CHARACTERISTIC) This limit range defines the range of plausible values for this channel and may be used to display a warning. Note: the (extended) limit range can only be expressed for a channel whose conversion rule returns a numeric value (REAL) or which has a numeric channel data type (cn_data_type ≤ 5). In all other cases, the flag must not be set. Depending on the type of conversion, the limit values are interpreted as physical or raw values. If the conversion rule results in numeric values, the limits must be interpreted as physical values, otherwise (e.g. for verbal / scale conversion) as raw values.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Bit 5: Extended limit range valid flag If set, the extended limits of the signal value are known and stored in cn_limit_ext_min and cn_limit_ext_max. Otherwise the two fields are not valid. (see MCD-2 MC [1] keyword EXTENDED_LIMITS) The extended limit range must be larger or equal to the limit range (if valid). Values outside the extended limit range indicate an error during measurement acquisition. The extended limit range can be used for any limits, e.g. physical ranges from analog sensors. See also remarks for "limit range valid" flag (bit 4).
			Bit 6: Discrete value flag If set, the signal values of this channel are discrete and must not be interpolated. (see MCD-2 MC [1] keyword DISCRETE)
			Bit 7: Calibration flag If set, the signal values of this channel correspond to a calibration object, otherwise to a measurement object (see MCD-2 MC [1] keywords MEASUREMENT and CHARACTERISTIC)
			Bit 8: Calculated flag If set, the values of this channel have been calculated from other channel inputs. (see MCD-2 MC [1] keywords VIRTUAL and DEPENDENT_CHARACTERISTIC) In MDBLOCK for cn_md_comment the used input signals and the calculation formula can be documented, see Table 41.
			Bit 9: Virtual flag If set, this channel is virtual, i.e. it is simulated by the recording tool. (see MCD-2 MC [1] keywords VIRTUAL and VIRTUAL_CHARACTERISTIC) Note: for a virtual measurement according to MCD-2 MC both the "Virtual" flag (bit 9) and the "Calculated" flag (bit 8) should be set.
			Bit 10: Bus event flag If set, this channel contains information about a bus event. For details please refer to MDF Bus Logging [7]. *valid since MDF 4.1.0, should not be set for earlier versions*
			Bit 11: Strictly monotonous flag

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			If set, this channel contains only strictly monotonously increasing/decreasing values. The flag is optional. *valid since MDF 4.1.0, should not be set for earlier versions*
			Bit 12: Default X axis flag If set, a channel to be preferably used as X axis is specified by cn_default_x. This is only a recommendation; a tool may choose to use a different X axis. *valid since MDF 4.1.0, should not be set for earlier versions*
			Bit 13: Event signal flag If set, a channel is used for the description of events in an event signal group. See chapter 4.12.5 **Event Signals** for details. *valid since MDF 4.2.0, should not be set for earlier versions*
			Bit 14: VLSD data stream flag Can only be set for a variable length data channel (cn_type = 1) in a sorted data group. If set, the SDBLOCK referenced by cn_data (or its equivalent in case of distributed or compressed data blocks) contains a stream of VLSD values, i.e. all VLSD values (each consisting of 4 Byte length and N Bytes data) must be stored in correct ordering and without gaps. In this case, for reading all VLSD values, the offset stored in fixed-length record for VLSD channel can be ignored because the VLSD values can be read one-by-one from the data "stream". The flag is optional. *valid since MDF 4.2.0, should not be set for earlier versions*
cn_inval_bit_pos	UINT32	1	Position of invalidation bit. The invalidation bit can be used to specify if the signal value in the current record is valid or not. Note: the invalidation bit is optional and can only be used if the "invalidation bit valid" flag (bit 1) is set. cn_inval_bit_pos contains both bit and Byte offset for the (single) invalidation bit and specifies its position within the invalidation Bytes of the record. This means that the record ID (if present) and the data Bytes must be skipped before applying the Byte offset (cn_inval_bit_pos >> 3). Within this Byte, the number of the invalidation bit (starting from LSB

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			= 0) is specified by (cn_inval_bit_pos & 0x07). If an invalidation bit is used for a channel, and if the respective bit in the invalidation Bytes of the record is set (value is high), then the signal value of this channel in the record is invalid. In this case, as best practice, the record should contain the most recent valid signal value for this channel, or zero if no valid signal value has occurred yet. For an example please refer to 4.21.5.1 Reading the Invalidation Bit.
cn_precision	UINT8	1	Precision for display of floating-point values. 0xFF means unrestricted precision (infinite). Any other value specifies the number of decimal places to use for display of floating-point values. Only valid if "precision valid" flag (bit 2) is set
cn_reserved	BYTE	1	Reserved
cn_attachment_ count	UINT16	1	Length N of cn_at_reference list, i.e. number of attachments for this channel. Can be zero. *valid since MDF 4.1.0, should be zero for earlier versions*
cn_val_range_min	REAL	1	**Minimum signal value** that occurred for this signal (raw value) Only valid if "value range valid" flag (bit 3) is set.
cn_val_range_max	REAL	1	**Maximum signal value** that occurred for this signal (raw value) Only valid if "value range valid" flag (bit 3) is set.
cn_limit_min	REAL	1	**Lower limit** for this signal (physical value for numeric conversion rule, otherwise raw value) Only valid if "limit range valid" flag (bit 4) is set.
cn_limit_max	REAL	1	**Upper limit** for this signal (physical value for numeric conversion rule, otherwise raw value) Only valid if "limit range valid" flag (bit 4) is set.
cn_limit_ext_min	REAL	1	**Lower extended limit** for this signal (physical value for numeric conversion rule, otherwise raw value) Only valid if "extended limit range valid" flag (bit 5) is set. If cn_limit_min is valid, cn_limit_min must be larger or equal to cn_limit_ext_min.
cn_limit_ext_max	REAL	1	**Upper extended limit** for this signal (physical value for numeric conversion rule, otherwise raw value) Only valid if "extended limit range valid" flag (bit 5) is set. If cn_limit_max is valid, cn_limit_max must be less or equal to cn_limit_ext_max.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Note: The combinations of cn_data_type and cn_bit_count are limited as follows:

- For Integer channel data types (cn_data_type ≤ 3), the number of bits cn_bit_count can be arbitrary, i.e. you may encode a signal value with a range of [0-7] in 3 bits only. However, the Integer signal must fit into 8 contiguous Bytes (cn_bit_offset + cn_bit_count ≤ 64).
  Note: some applications may only accept up to 32 bits for an Integer value (unsigned int) or expect that an Integer signal fits into 4 contiguous Bytes.
- For virtual master channels (cn_type = 3) and virtual data channels (cn_type = 6), no bits are used (cn_bit_count = 0) and the data type must be unsigned integer with Little Endian byte order (cn_data_type = 0).
- For floating-point channel data types (3 < cn_data_type ≤ 5), cn_bit_count must be either 16, 32 or 64 (2, 4 or 8 Bytes, i.e. half, single or double precision floating-point according to IEEE 754, see [13]), and cn_bit_offset must be zero (1-Byte alignment).
  Note that half precision floating-point (16 bit) has been introduced with MDF 4.2.0 and should not be used in earlier versions.
- For complex number channel data types (14 < cn_data_type ≤ 16), cn_bit_count must be either 32, 64 or 128 (4, 8 or 16 Bytes, i.e. two consecutive half, single or double precision floating-points according to IEEE 754, see [13]), and cn_bit_offset must be zero (1-Byte alignment).
  Note that complex number channel data type has been introduced with MDF 4.2.0 and should not be used in earlier versions.
- For all other channel data types, cn_bit_count must be a multiple of 8 (only whole Bytes) and cn_bit_offset must be zero (1-Byte alignment).
- In addition, for the special data types "CANopen date" and "CANopen time" (see [11]), cn_bit_count must be equal to the size of the respective bit structure (56 or 48 bits). See following sections for a definition of these structures.

4.16.2 Data Structure for Channel Data Type "CANopen Date"

For the channel data type "CANopen date" (cn_data_type = 13), the number of bits must be 56 (i.e. 7 Byte). A conversion rule is not necessary. The record value represents a bit structure with date information (see CANopen [11]).

Table 39 Data structure for channel data type "CANopen date"

Type	Identifier	Description
UINT16	ms	Bit 0 .. Bit 15: Milliseconds (0 .. 59999)
UINT8	min	Bit 0 .. Bit 5: Minutes (0 .. 59) Bit 6 .. Bit 7: Reserved
UINT8	hour	Bit 0 .. Bit 4: Hours (0 .. 23) Bit 5 .. Bit 6: Reserved Bit 7: 0 = Standard time, 1 = Summer time
UINT8	day	Bit 0 .. Bit 4: Day (1 .. 31) Bit 5 .. Bit 7: Day of week (1 = Monday ... 7 = Sunday)
UINT8	month	Bit 0 .. Bit 5: Month (1 = January .. 12 = December) Bit 6 .. Bit 7: Reserved
UINT8	year	Bit 0 .. Bit 6: Year (0 .. 99) Bit 7: Reserved

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.16.3 Data Structure for Channel Data Type "CANopen Time"

For the channel data type "CANopen time" (cn_data_type = 14), the number of bits must be 48 (i.e. 6 Byte). A conversion rule is not necessary. The record value represents a bit structure with time information (see CANopen [11]).

Table 40 Data structure for channel data type "CANopen time"

Type	Identifier	Description
UINT32	ms	Bit 0 .. Bit 27: Number of Milliseconds since midnight of 01. Jan.1984 Bit 28 .. Bit 31: Reserved
UINT16	days	Bit 0 .. Bit 15: Number of days since 01. Jan.1984 (Can be 0)

4.16.4 Contents of the Meta Data Block for the Channel Comment

The content of the MDBLOCK for the channel comment is defined by a special XML schema (cn_comment.xsd). It defines the following XML tags, but can be extended to store user-defined information.

Table 41 Tags for MDBLOCK cn_md_comment

Tag name	Description	
<TX>	Description of the channel as plain text (default tag, required)	
<names>	Alternative names and description as defined in 5.4 Alternative Names (optional)	
<linker_name>	Name of linker map variable for the channel (optional) see "SymbolName" for MCD-2 MC [1] keyword SYMBOL_LINK. The following attribute can be used to give additional information:	
	offset	Offset of variable from start address specified in linker map file (i.e. in <linker_address> tag), see "Offset" for MCD-2 MC [1] keyword SYMBOL_LINK (optional, default value: 0)
<linker_address>	Address information for channel data in ECU (optional) Tag contains hexadecimal start address in ECU as stated in the corresponding linker map file. Note that this may not be the address used for reading the channel data. If specified, the offset from the respective attribute of <linker_name> must be added to obtain the read address. In addition, the actually used read address may be specified in tag <address>. In case both tags are present, <address> should be considered as read address. The following attributes can be used to give additional information:	
	byte_count	number of Bytes to read (optional, default value: 8) byte_count = 0 means that the number of Bytes is not specified in linker map file.
	bit_mask	hexadecimal value for bit mask to apply to the Bytes (optional, default value: 0xFFFFFFFFFFFFFFFF, i.e. all bits are used)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Tag name	Description	
	byte_order	Byte order, either "BE" for Big Endian (Motorola) Byte order or "LE" for Little Endian (Intel) Byte order. (optional, default value: "LE")
<address>	Address information from where the channel data has been read (optional) Tag contains hexadecimal start address in ECU used for reading the channel data. The following attributes can be used to give additional information:	
	byte_count	number of Bytes to read (optional, default value: 8)
	bit_mask	hexadecimal value for bit mask to apply to the Bytes (optional, default value: 0xFFFFFFFFFFFFFFFF, i.e. all bits are used)
	byte_order	Byte order, either "BE" for Big Endian (Motorola) Byte order or "LE" for Little Endian (Intel) Byte order. (optional, default value: "LE")
<axis_monotony>	Monotony of axis (optional). Must only be used if CNBLOCK references a CABLOCK of type "scaling axis". Can contain one of the following strings (see MCD-2 MC [1] keyword MONOTONY).If not specified, the monotony is undefined. Note: Monotony is defined with respect to the raw values and not the physical values.	
	MON_DECREASE	monotonously decreasing
	MON_INCREASE	monotonously increasing
	STRICT_DECREASE	strict monotonously increasing
	STRICT_INCREASE	strict monotonously increasing
	MONOTONOUS	monotonously in- or decreasing
	STRICT_MON	strict monotonously in- or decreasing
	NOT_MON	no monotony required

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Tag name	Description
<raster>	Target raster value for complete channel (optional) Must only be used for (virtual) master channel types (i.e. 2 ≤ ca_type ≤ 3) If the raster information is known, it should always be added. The target raster value is the recommended raster value to be used when transforming to equidistant time/angle/distance values. This may be the expected raster value or a value determined from real data (e.g. calculated average raster value). In addition, the optional attributes "min", "max" and "average" can be used to store the minimum, maximum and average raster value over all samples for this channel as a measure of the "raster quality" (definition of raster value see Table 68). All raster values are stored as physical value with the SI unit (see [19]) of the master channel type (i.e. seconds, radians or meters). The raster tag also can be left empty, e.g. to only document the statistics for the raster. Examples: <raster min=”0.09931” max=”0.1042”>0.100</raster> <raster min=”0.0532” max=”0.1142” average=”0.103”/>
<formula>	Formula for calculated channels (optional) Must only be used if the "calculated" flag (bit 8) is set in cn_flags. For details see 5.5 Formula Specifications.

4.16.5 Contents of the Meta Data Block for the Channel Unit

The content of the MDBLOCK for cn_md_unit is defined by a special XML schema (cn_unit.xsd). It defines the following XML tags, but can be extended to store user-defined information about the unit.

Table 42 Tags for MDBLOCK cn_md_unit

Tag name	Description
<TX>	Unit or MIME type as described for cn_md_unit (default tag, required)
<ho_unit>	Unit expressed as HDO syntax (optional) Tag content is empty, required attribute unit_ref contains the ID of an HDO <UNIT> tag globally defined in <ho:UNIT-SPEC> tag in MDBLOCK referenced by hd_md_comment. Example: <ho_unit unit_ref="ho_unit_kmh"/>

4.17 The Channel Conversion Block CCBLOCK

The data records can be used to store raw values (often also denoted as implementation values or internal values). The CCBLOCK serves to specify a conversion formula to convert the raw values to physical values with a physical unit. The result of a conversion always is either a floating-point value (REAL) or a character string (UTF-8).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.17.1 Block Structure of CCBLOCK

Table 43 Structure of CCBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##CC”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
cc_tx_name	LINK	1	Link to TXBLOCK with name (identifier) of conversion (can be NIL). Name must be according to naming rules stated in Naming Rules.
cc_md_unit	LINK	1	Link to TXBLOCK/MDBLOCK with physical unit of signal data (after conversion). (can be NIL) Unit only applies if no unit defined in CNBLOCK. Otherwise the unit of the channel overwrites the conversion unit. An MDBLOCK can be used to additionally reference the HDO unit definition (see Table 61). **Note:** for channels with cn_sync_type > 0, the unit is already defined, thus a reference to an HDO definition should be omitted to avoid redundancy.
cc_md_comment	LINK	1	Link to TXBLOCK/MDBLOCK with comment of conversion and additional information, see Table 60.(can be NIL)
cc_cc_inverse	LINK	1	Link to CCBLOCK for inverse formula (can be NIL, must be NIL for CCBLOCK of the inverse formula (no cyclic reference allowed).
cc_ref	LINK	M	List of additional links to TXBLOCKs with strings or to CCBLOCKs with partial conversion rules. Length of list is given by cc_ref_count. The list can be empty. Details are explained in formula-specific block supplement.
Data section			
cc_type	UINT8	1	**Conversion type** (formula identifier) 0 = 1:1 conversion (in this case, the CCBLOCK can be omitted) 1 = linear conversion 2 = rational conversion 3 = algebraic conversion (MCD-2 MC text

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			formula) 4 = value to value tabular look-up with interpolation 5 = value to value tabular look-up without interpolation 6 = value range to value tabular look-up 7 = value to text/scale conversion tabular look-up 8 = value range to text/scale conversion tabular look-up 9 = text to value tabular look-up 10 = text to text tabular look-up (translation) 11 = bitfield text table
cc_precision	UINT8	1	Precision for display of floating-point values. 0xFF means unrestricted precision (infinite) Any other value specifies the number of decimal places to use for display of floating-point values. Note: only valid if "precision valid" flag (bit 0) is set and if cn_precision of the parent CNBLOCK is invalid, otherwise cn_precision must be used.
cc_flags	UINT16	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: Precision valid flag If set, the precision value for display of floating-point values specified in cc_precision is valid
			Bit 1: Physical value range valid flag If set, both the minimum and the maximum physical value that occurred after conversion for this signal within the samples recorded in this file are known and stored in cc_phy_range_min and cc_phy_range_max. Otherwise the two fields are not valid. Note: the physical value range can only be expressed for conversions which return a numeric value (REAL). For conversions returning a string value or for the inverse conversion rule the flag must not be set.
			Bit 2: Status string flag This flag indicates for conversion types 7 and 8 (value/value range to text/scale conversion tabular look-up) that the normal table entries are status strings (only reference to TXBLOCK), and the actual conversion rule is given in CCBLOCK referenced by default

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			value. This also implies special handling of limits, see MCD-2 MC [1] keyword STATUS_STRING_REF. Can only be set for 7 ≤ cc_type ≤ 8.
cc_ref_count	UINT16	1	**Length M of cc_ref list with additional links.** See formula-specific block supplement for meaning of the links.
cc_val_count	UINT16	1	**Length N of cc_val list with additional parameters.** See formula-specific block supplement for meaning of the parameters.
cc_phy_range_min	REAL	1	**Minimum physical signal value** that occurred for this signal. Only valid if "physical value range valid" flag (bit 1) is set.
cc_phy_range_max	REAL	1	**Maximum physical signal value** that occurred for this signal. Only valid if "physical value range valid" flag (bit 1) is set.
cc_val	REAL or UINT64	N	**List of additional conversion parameters**. Length of list is given by cc_val_count. The list can be empty. Details are explained in formula-specific block supplement.

Table 44 shows the calculation of cc_ref_count and cc_val_count for each conversion type where n is the number of table entries for conversions using a look-up table.

Table 44 Calculation of cc_ref_count and cc_val_count

cc_type	cc_ref_count M	cc_val_count N
0 = 1:1 conversion	0	0
1 = parametric, linear	0	2
2 = rational conversion formula	0	6
3 = algebraic conversion (MCD-2 MC text formula)	1	0
4 = value to value tabular look-up with interpolation	0	2 x n
5 = value to value tabular look-up without interpolation	0	2 x n
6 = value range to value tabular look-up	0	3 x n + 1
7 = value to text/scale conversion tabular look-up	n + 1	n
8 = value range to text/scale conversion tabular look-up	n + 1	2 x n
9 = text to value tabular look-up	n	n + 1
10 = text to text tabular look-up (translation)	2 x n + 1	0
11 = bitfield text table	n	n

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

The following sections specify and explain the different conversion types and their formula-specific block supplements in detail. Int denotes the internal (implementation / raw) value stored in the MDF file. Phys denotes the converted physical value. Normal conversion rules define a conversion from Int (as input to the conversion) to Phys (as output), i.e. Phys = f(Int). For an inverse conversion rule, this is vice versa, i.e. Phys and Int in the formula must be switched.

4.17.2 CCBLOCK – 1:1 Conversion

This is the simplest conversion where the *Phys* value is equal to the *Int* value.

$$Phys = Int$$

Note: Unless the CCBLOCK is used to store additional information about the conversion, it simply can be omitted because without CCBLOCK generally a 1:1 conversion must be used.

4.17.3 CCBLOCK – Linear Conversion

The conversion uses a linear formula with two parameters:

$$Phys = Int * P2 + P1$$

Note: P2 may be 0 which maps all values to a constant.

The list of additional conversion parameters contains N = 2 elements:

Table 45 CCBLOCK supplement for linear conversion

Name	Type	Count	Description
cc_val[0]	REAL	1	parameter P1
cc_val[1]	REAL	1	parameter P2

4.17.4 CCBLOCK - Rational conversion

The conversion uses a rational formula with six parameters:

$$Phys = \frac{P1 * Int^2 + P2 * Int + P3}{P4 * Int^2 + P5 * Int + P6}$$

Attention: In contrast to the MCD-2 MC [1] keyword RAT_FUNC, in MDF the conversion is vice versa, i.e. from internal (raw) value to physical value!

The list of additional conversion parameters contains N = 6 elements:

Table 46 CCBLOCK supplement for rational conversion

Name	Type	Count	Description
cc_val[0]	REAL	1	parameter P1
cc_val[1]	REAL	1	parameter P2
cc_val[2]	REAL	1	parameter P3
cc_val[3]	REAL	1	parameter P4

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
cc_val[4]	REAL	1	parameter P5
cc_val[5]	REAL	1	parameter P6

4.17.5 CCBLOCK –Algebraic Conversion (MCD-2 MC Text formula)

With this conversion the *Phys* value is determined for a given *Int* value by using a calculation rule given as text formula. In the text formula the *Int* value is specified by variable X.

Example:
The text formula "X * X + 1" multiplies the Int value by itself and adds 1 to the result.
See MCD-2 MC [1] keyword FORMULA.

Note that for the example also a rational formula with P1 = P3 = P6 = 1 and P2 = P4 = P5 = 0 could be used. Since parsing of an algebraic formula needs more effort, using a linear or rational conversion should be preferred if possible.

The list of additional links contains M = 1 element:

Table 47 CCBLOCK supplement for algebraic conversion

Name	Type	Count	Description
cc_ref[0]	LINK	1	TXBLOCK with text formula in syntax defined for MCD-2 MC [1] keyword FORMULA (uses a subset of ASAM General Expression Syntax V1.0). An alternative syntax for the algebraic conversion optionally can be specified in MDBLOCK referenced by cc_md_comment using the tag <formula> (see Table 60). The alternative syntax is only intended for documentation, the conversion should use the formula of the TXBLOCK.

4.17.6 CCBLOCK – Value to Value Table With Interpolation

Basis for this conversion is a look-up table with n table entries (key-value pairs) for look-up of a *Phys* value for a given *Int* value.

If *Int* ≤ key[0], then *Phys* = value[0] must be returned.
If *Int* ≥ key[n-1], then *Phys* = value[n-1] must be returned.
If the *Int* value is located between two key values (key[i] ≤ *Int* < key[i+1]), then linear interpolation between value[i] and value[i+1] must be used to determine the *Phys* value.

Note: The key values must be sorted in strictly monotonously increasing order, i.e. key [i] > key[i-1] for all 0 < i < n.

The list of additional conversion parameters contains N = (n x 2) elements, where n is the number of table entries (key-value pairs).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 48 CCBLOCK supplement for value to value table with interpolation

Name	Type	Count	Description
cc_val[0]	REAL	1	key[0]
cc_val[1]	REAL	1	value[0]
…	...		
cc_val[2*n-2]	REAL	1	key[n-1]
cc_val[2*n-1]	REAL	1	value[n-1]

4.17.7 CCBLOCK – Value to Value Table Without Interpolation

Basis for this conversion is a look-up table with n table entries (key-value pairs) for look-up of a *Phys* value for a given *Int* value.

If *Int* = key[i], then *Phys* = value[i] must be returned.
If *Int* < key[0], then *Phys* = value[0] must be returned.
If *Int* > key[n-1], then *Phys* = value[n-1] must be returned.
If key[i] < *Int* < key[i+1], i.e. the *Int* value does not exactly match one of the key values, then the nearest key value must be returned as *Phys* value, i.e. if (*Int* - key[i]) > (key[i+1] - *Int*) then *Phys* = value[i+1], otherwise *Phys* = value[i].
This implies that if *Int* is exactly between two key values, i.e. (*Int* - key[i]) = (key[i+1] - *Int*), then the nearest lower key value value[i] must be used.

Note: The key values must be sorted in strictly monotonously increasing order, i.e. key [i] > key[i-1] for all 0 < i < n.

The list of additional conversion parameters contains N = (n x 2) elements, where n is the number of table entries (key-value pairs).

Table 49 CCBLOCK supplement for value to value table without interpolation

Name	Type	Count	Description
cc_val[0]	REAL	1	key[0]
cc_val[1]	REAL	1	value[0]
…	...		
cc_val[2*n-2]	REAL	1	key[n-1]
cc_val[2*n-1]	REAL	1	value[n-1]

4.17.8 CCBLOCK – Value Range to Value Table

Basis for this conversion is a look-up table with n table entries (key range-value pairs) for look-up of a *Phys* value for a given *Int* value. Instead of a single key value, a value range specified by a lower and upper bound (key_min[i] ≤ key_max[i]) is used as key. Thus, a *Phys* value is assigned to a range of *Int* values. In addition, a default value is specified in case none of the ranges fits.

- If *Int* is an Integer value (cn_data_type ≤ 3):
  - If key_min[i] ≤ *Int* ≤ key_max[i], then *Phys* = value[i] must be returned.
  - Otherwise (i.e. no range fits) the default value must be returned for *Phys*.
- If *Int* is a floating-point value (3 < cn_data_type ≤ 5) or for inverse conversion:

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

- If key_min[i] ≤ *Int* < key_max[i], then *Phys* = value[i] must be returned.
- Otherwise (i.e. no range fits) the default value must be returned for *Phys*.

Note:

- The ranges should be sorted in strictly monotonously increasing order, i.e. key_min[i] > key_min[i-1] for all 0 < i < n.
- The key ranges must not overlap, i.e. key_max[i-1] ≤ key_min[i] for all 0 < i < n.
- Positive or negative INFINITY (NaN values) can be used for the respective range limits key_max[n-1] or key_min[0] to express an open range towards positive or negative infinity.

The list of additional conversion parameters contains N = (n x 3 +1) elements, where n is the number of table entries (key range-value pairs).

Table 50 CCBLOCK supplement for value range to value table

Name	Type	Count	Description
cc_val[0]	REAL	1	key_min[0]
cc_val[1]	REAL	1	key_max[0]
cc_val[2]	REAL	1	value[0]
…	...		
cc_val[3*n-3]	REAL	1	key_min[n-1]
cc_val[3*n-2]	REAL	1	key_max[n-1]
cc_val[3*n-1]	REAL	1	value[n-1]
cc_val[3*n]	REAL	1	default value

4.17.9 CCBLOCK – Value to Text / Scale Conversion Table

Basis for this conversion is a look-up table with n table entries (key-link pairs) for look-up of a link to determine the *Phys* value for a given *Int* value. In addition, a default link is specified in case none of the key values fits.

If *Int* = key[i], then link[i] must be used to determine the *Phys* value.

Otherwise, the default link must be used.

The result of the look-up is a link either to a TXBLOCK or to a CCBLOCK.

- In case of a TXBLOCK, the character string in the TXBLOCK must be returned as result of the conversion. This can be used to assign a single bit value or an Integer value to verbal descriptions (for example "Gear 1", "Gear 2", "Gear 3", "On", "Off"). See also MCD-2 MC [1] keyword COMPU_VTAB.
- In case of a CCBLOCK, this CCBLOCK is called a "scale conversion" and specifies a specific conversion for this Int value similar to HDO CompuMethods "Scale-xxx".
  In this case, the Int value must be fed to the scale conversion and the scale conversion result (either a character string or a floating-point value) must be returned. Note that there may be more than one step in a conversion chain, i.e. the CCBLOCK for the scale conversion may again use a scale conversion itself. However, circular references of the scale conversions are not allowed.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

- If the link returned by the look-up is NIL, this means that the output value for the given *Int* value is undefined (invalid). A NIL link must not be interpreted as 1:1 conversion!

If the "status string" flag (bit 2) in cc_flags is set, all links in the look-up table must point to TXBLOCKs and only the default link must point to a CCBLOCK. Thus, for a number of selected values from the *Int* value range an individual status string is assigned to whereas the actual conversion rule is given by the default link. See MCD-2 MC [1] keyword STATUS_STRING_REF.

Note:

- The key values must be sorted in strictly monotonously increasing order, i.e. key [i] > key[i-1] for all 0 < i < n.
- Text and scale conversion assignments can be mixed in the table.

The list of additional conversion parameters contains N = n elements, where n is the number of table entries (key-link pairs).

Table 51 CCBLOCK parameter supplement for value to text / scale conversion table

Name	Type	Count	Description
cc_val[0]	REAL	1	key[0]
…	...		
cc_val[n-1]	REAL	1	key[n-1]

The additional conversion link list contains M = (n+1) elements, where n is the number of table entries (key-link pairs).

Table 52 CCBLOCK link supplement for value to text / scale conversion table

Name	Type	Count	Description
cc_ref[0]	LINK	1	link[0] to TXBLOCK with output string or CCBLOCK with scale conversion for key[0]
…	...		
cc_ref[n-1]	LINK	1	link[n-1] to TXBLOCK with output string or CCBLOCK with scale conversion for key[n-1]
cc_ref[n]	LINK	1	default link, i.e. link to TXBLOCK with default output string or CCBLOCK with default scale conversion (can be NIL)

4.17.10 CCBLOCK – Value Range to Text / Scale Conversion Table

Basis for this conversion is a look-up table with n table entries (key range-link pairs) for look-up of a link to determine the *Phys* value for a given *Int* value. Instead of a single key value, a value range specified by a lower and upper bound (key_min[i] ≤ key_max[i]) is used as key. Thus, a *Phys* value is assigned to a range of *Int* values. In addition, a default link is specified in case none of the ranges fits.

The result of the look-up is a link either to a TXBLOCK or to a CCBLOCK. Please refer to 4.17.9 CCBLOCK – Value to Text / Scale Conversion Table for explanation how the Phys value must be determined from this link.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

The only difference to section 4.17.9 CCBLOCK – Value to Text / Scale Conversion Table is that a range of Int values is mapped to a link in order to determine the Phys value. With TXBLOCKs as target of the links, this can be used to assign float ranges to verbal descriptions (for example "low", "regular", "high"). See also MCD-2 MC [1] keyword COMPU_VTAB_RANGE. With CCBLOCKs as target of the links, each float range can have its own scale conversion similar to HDO CompuMethods "Scale-xxx".

- If Int is an Integer value (cn_data_type ≤ 3):
  - If key_min[i] ≤ *Int* ≤ key_max[i], then link[i] must be used to determine *Phys*.
  - Otherwise (i.e. no range fits) the default link must be used to determine *Phys*.
- If *Int* is a floating-point value (3 < cn_data_type ≤ 5) or for inverse conversion:
  - If key_min[i] ≤ *Int* < key_max[i], then link[i] must be used to determine *Phys*.
  - Otherwise (i.e. no range fits) the default link must be used to determine *Phys*.

If the "status string" flag (bit 2) in cc_flags is set, all links in the look-up table must point to TXBLOCKs and only the default link must point to a CCBLOCK. See MCD-2 MC [1] keyword STATUS_STRING_REF.

Note:

- The ranges should be sorted in strictly monotonically increasing order, i.e. key_min[i] > key_min[i-1] for all 0 < i < n.
- The key ranges must not overlap, i.e. key_max[i-1] ≤ key_min[i] for all 0 < i < n.
- Positive or negative INFINITY (NaN values) can be used for the respective range limits key_max[n-1] or key_min[0] to express an open range towards positive or negative infinity.
- Text and scale conversion assignments can be mixed in the table.

The list of additional conversion parameters contains N = (n x 2) elements, where n is the number of table entries (key range-link pairs).

Table 53 CCBLOCK parameter supplement for value range to text / scale conversion table

Name	Type	Count	Description
cc_val[0]	REAL	1	key_min[0]
cc_val[1]	REAL	1	key_max[0]
…	...		
cc_val[2*n-2]	REAL	1	key_min[n-1]
cc_val[2*n-1]	REAL	1	key_max[n-1]

The list of additional links contains M = (n+1) elements, where n is the number of table entries (key range-link pairs).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 54 CCBLOCK link supplement for value range to text / scale conversion table

Name	Type	Count	Description
cc_ref[0]	LINK	1	link[0] to TXBLOCK with output string or CCBLOCK with scale conversion for range from key_min[0] to key_max[0]
…	...		
cc_ref[n-1]	LINK	1	link[n-1] to TXBLOCK with output string or CCBLOCK with scale conversion for range from key_min[n-1] to key_max[n-1]
cc_ref[n]	LINK	1	default link, i.e. link to TXBLOCK with default output string or CCBLOCK with default scale conversion (can be NIL)

4.17.11 CCBLOCK – Text To Value Table

Basis for this conversion is a look-up table with n table entries (key-value pairs) for look-up of a *Phys* value for a given input string (*Int* value). In addition, a default value is specified in case the input string is not found in the list of keys.

If the input string matches the string for key[i], then value[i] must be returned for Phys. Otherwise, the default value must be returned for Phys.

Note:

- The key for look-up is a string instead of a value.
- String comparison for matching the input string to some key must be case sensitive.
- Each input string should only be specified once in the table.

The list of additional links contains M = n elements, where n is the number of table entries (key-value pairs).

Table 55 CCBLOCK link supplement for text to value table

Name	Type	Count	Description
cc_ref[0]	LINK	1	link to TXBLOCK with string for key[0] (must not be NIL)
…	...		
cc_ref[n-1]	LINK	1	link to TXBLOCK with string for key[n-1] (must not be NIL)

The list of additional conversion parameters contains N = (n + 1) elements, where n is the number of table entries (key-value pairs).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 56 CCBLOCK parameter supplement for text to value table

Name	Type	Count	Description
cc_val[0]	REAL	1	value[0]
…	...		
cc_val[n-1]	REAL	1	value[n-1]
cc_val[n]	REAL	1	default value

4.17.12 CCBLOCK – Text To Text Table

Basis for this conversion is a look-up table with n table entries (key-link pairs) for look-up of a link to determine the *Phys* value for a given input string (*Int* value). In addition, a default link is specified in case the input string is not found in the list of keys.

If the input string matches the string for key[i], then link[i] must be used to determine the output string for Phys. Otherwise, the default link must be used.
If the link to be used points to a TXBLOCK, the string of the TXBLOCK must be returned for *Phys*. Otherwise the input string must be returned for *Phys*.

Generally speaking, this conversion "translates" a given string to some other string. If the link for an output string is NIL, the input string must be returned as output. If the input string is not in the list, a default output must be used.

Note:

- The key for look-up is a string instead of a value.
- String comparison for matching the input string to some key must be case sensitive.
- Each input string should only be specified once in the table.
- If the link for the default output string is NIL, the input string must be returned as output (similar to 1:1 conversion).

The list of additional links contains M = (n x 2 +1) elements, where n is the number of table entries (key-link pairs).

Table 57 CCBLOCK link supplement for text to text table

Name	Type	Count	Description
cc_ref[0]	LINK	1	link to TXBLOCK with string for key[0] (lmust not be NIL)
cc_ref[1]	LINK	1	link[0], i.e. link to TXBLOCK with output string for key[0] (can be NIL)
…	…	...	..
cc_ref[2*n-2]	LINK	1	link to TXBLOCK with string for key[n-1] (must not be NIL)
cc_ref[2*n-1]	LINK	1	link[n-1], i.e. link to TXBLOCK with output string for key[n-1] (can be NIL)
cc_ref[2*n]	LINK	1	default link, i.e. link to TXBLOCK with default output string (can be NIL)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.17.13 CCBLOCK – Bitfield Text Table

Basis for this conversion is a table with n table entries (value-link pairs) to determine the *Phys* value (assembled string) for a given *Int* value:

- The value is a bitmask (UINT64) to be applied to the given *Int* value resulting in a "masked *Int* value of the same type and bit count of the input signal". Therefore, the *Int* value should be an Integer type.
- The link references a conversion rule (CCBLOCK) that must be applied to the masked *Int* value and which must either return a text (link to TXBLOCK) or "nothing" (NIL link) as result. Therefore, the link must reference a CCBLOCK with either a "value to text/scale conversion tabular look-up" (cc_type = 7) or a "value range to text/scale conversion tabular look-up" (cc_type = 8). Note if a name is specified in cc_tx_name for the referenced CCBLOCK, this name will be used, too.

The result of the look-up is an assembled string.

To create the output string of the bitfield text table, for each table entry, the bit mask value is applied to *Int* value (bitwise AND). The result of this operation is used as input value to the respective conversion rule.

- If the output of the conversion rule is "nothing", the table entry will be ignored.
- If the output of the conversion is a text, and if a name is specified for the rule, the partial result for this table entry is "<name> = <result text>".
- If no name is specified, the partial result is simply the result text of the conversion.

All valid partial results then are concatenated in order of the table entries using the pipe symbol '|' as separator. Therefore, the pipe symbol must neither occur in the names nor in the result texts of the referenced conversion rules.

Note:

- The bitmasks in the table are not required to be ordered and can overlap.
- Links to value to text and value range to text conversions can be mixed in the table.

The list of additional conversion parameters contains N = n elements, where n is the number of table entries (key-link pairs).

Table 58 CCBLOCK parameter supplement for value to text / scale conversion table

Name	Type	Count	Description
cc_val[0]	UINT64	1	bitmask[0]
…	...		
cc_val[n-1]	UINT64	1	bitmask[n-1]

The additional conversion link list contains M = n elements, where n is the number of table entries (key-link pairs).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 59 CCBLOCK link supplement for bitfield text table

Name	Type	Count	Description
cc_ref[0]	LINK	1	link[0] to CCBLOCK with conversion to be applied to bitmask[0] | *Int*
…	...		
cc_ref[n-1]	LINK	1	link[n-1] to CCBLOCK with conversion to be applied to bitmask[n-1] | *Int*

4.17.14 Contents of the Meta Data Block for the Conversion Comment

The content of the MDBLOCK for the comment of the conversion is defined by the XML schema cc_comment.xsd. In addition to the standard tags it includes the following tags specific to the CCBLOCK.

Table 60 Tags for MDBLOCK cc_md_comment

Tag name	Description
<TX>	Comment for the conversion (default tag, required).
<names>	Alternative names and description as defined in 5.4 Alternative Names (optional).

<ho:COMPU_METHOD>	Conversion rule expressed in HDO syntax (optional). References to units defined in <ho:UNIT-SPEC> tag in MDBLOCK of hd_md_comment possible by <ho:UNIT-REF>. The namespace for ASAM harmonized objects (see [4]) must be used.
	Example:
	`<ho:COMPU-METHOD xmlns:ho="http://www.asam.net/xml">`
	`  <ho:SHORT-NAME>Linear7</ho:SHORT-NAME>`
	`  <ho:CATEGORY>LINEAR</ho:CATEGORY>`
	`  <ho:UNIT-REF ID-REF="unitSiBase_meter"/>`
	`  <ho:COMPU-INTERNAL-TO-PHYS>`
	`  <ho:COMPU-SCALES>`
	`  <ho:COMPU-SCALE>`
	`  <ho:COMPU-RATIONAL-COEFFS>`
	`  <ho:COMPU-NUMERATOR>`
	`  <ho:V>0</ho:V>`
	`  <ho:V>7</ho:V>`
	`  </ho:COMPU-NUMERATOR>`
	`  </ho:COMPU-RATIONAL-COEFFS>`
	`  </ho:COMPU-SCALE>`
	`  </ho:COMPU-SCALES>`
	`  </ho:COMPU-INTERNAL-TO-PHYS>`
	`</ho:COMPU-METHOD>`

<formula>	Conversion rule expressed in alternative syntax (optional). For details see 5.5 Formula Specifications. As an exception to the general form of the <formula> tag only the sub-tag <custom_syntax> is allowed here. The tag <syntax> is not allowed because the GES formula is already specified by the TXBLOCK of the Algebraic Conversion The tag <variables> is not allowed since a conversion always applies to the channel described by the parent CNBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.17.15 Contents of the Meta Data Block for the Conversion Unit

The content of the MDBLOCK for the comment of the event is defined by the XML schema cc_unit.xsd. In addition to the standard tags it includes the following tags specific to the CCBLOCK.

Table 61 Tags for MDBLOCK cc_md_unit

Tag name	Description
<TX>	Unit as described for cc_md_unit (default tag, required)
<ho_unit>	Unit expressed as HDO syntax (optional) Tag content is empty, required attribute unit_ref contains the ID of an HDO <UNIT> tag globally defined in <ho:UNIT-SPEC> tag in MDBLOCK referenced by hd_md_comment. Example: <ho_unit unit_ref="ho_unit_kmh"/>

4.18 Composition of Channels

A signal may be a composition of other signals, e.g. for a map/curve or for a structure. When saving a composed signal, this can be modeled in MDF by a composition of channels. MDF supports two native composition types that are known in most programming languages: arrays and structures.

The composed channel is said to have components (component channels). For an array, the components are the array elements, for a structure, the components are the members of the structure.

Note that a component channel again may be a composition of channels. This can be used to model a nested hierarchy, i.e. if a member of a structure or an element of an array again is a structure or an array itself. Details about this possibility will be explained in 4.20 Nested Composition of Channels.

4.18.1 Structures

A structure can be seen as one-dimensional array of elements, where each element may have a different data type.

There are two ways to model a structure in MDF: If all components (members) of a structure are acquired simultaneously, i.e. all component channels are in one channel group, then we can model a structure by an own CNBLOCK representing the complete structure and a CNBLOCK for each component. We will denote this as "compact" structure because all Bytes of the structure are stored "en block" in the same record.

The opposite case, in which components of the structure are acquired at different times or rates, will be denoted as "fragmented" structure. In this case, the complete structure can not be modeled by an own CNBLOCK anymore. Instead it must be modeled using a CHBLOCK of type "structure". Each component referenced by the CHBLOCK either again is a CHBLOCK (of type "structure" or "map list"), or simply a CNBLOCK. The CNBLOCKs for the components are individual channels; like if they were not part of the structure. The CHBLOCK of type structure simply is used to combine them and to save the information that they originally belonged to a structure. Figure 13 shows an example.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

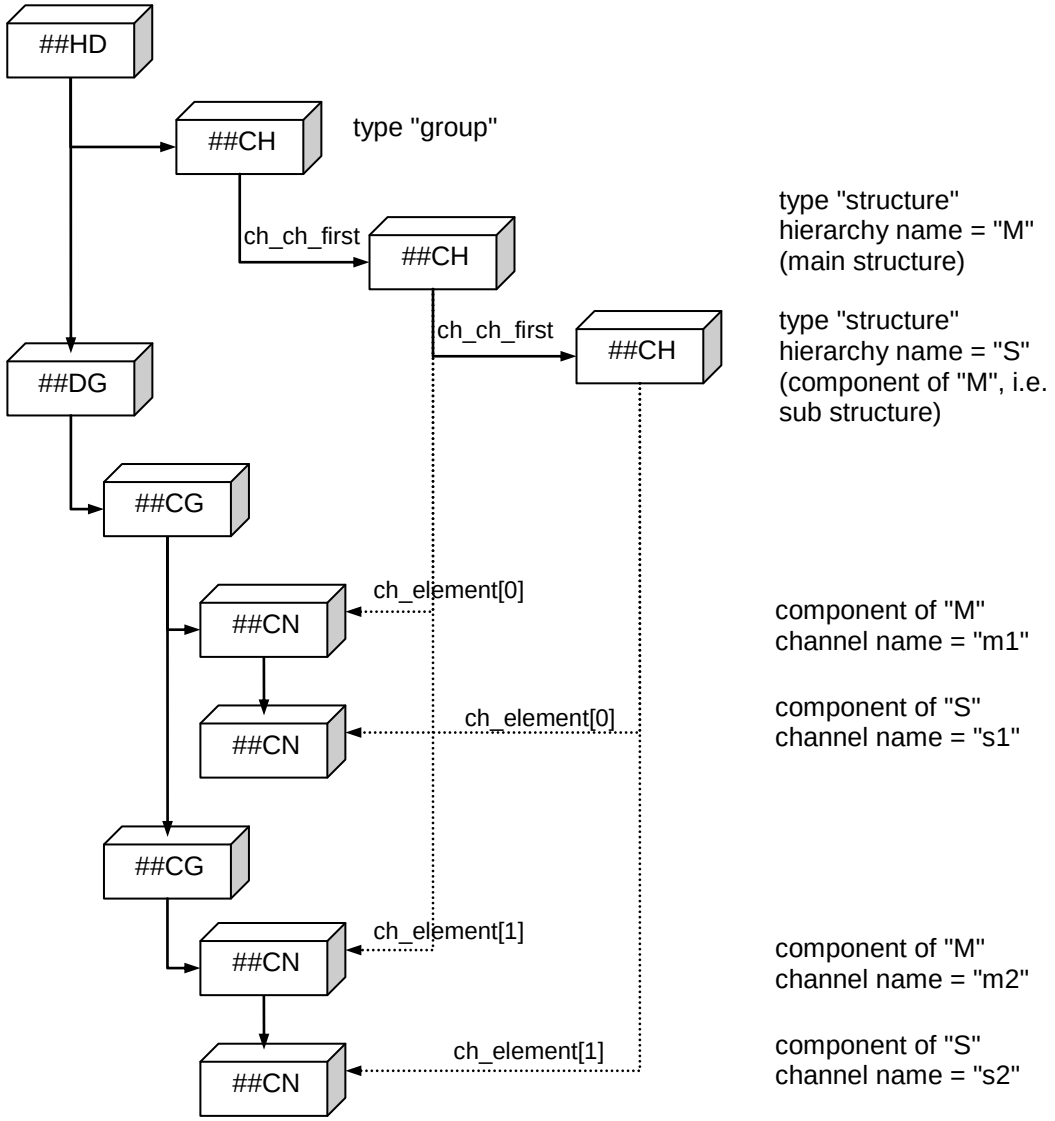


Figure 13 Example of block structure modeling a fragmented structure

In case of a compact structure, the parent CNBLOCK represents the complete structure, i.e. a range of contiguous Bytes that must contain all component values of the structure. Channel data type "Byte Array" must be used for the parent CNBLOCK. Since a structure has a fixed size, the CNBLOCK must be a fixed length data channel.

The indication that a CNBLOCK represents a compact structure is that it has a cn_composition link which points to a CNBLOCK. This CNBLOCK is the start of a linked list (by cn_cn_next links) of the component channels of the structure.

The component CNBLOCKs must not be direct children of a channel group, i.e. they must not be in the forward linked list of CNBLOCKs starting at cg_cn_first. Also, as we will see later, if a component CNBLOCK is an array, it must use the "CN template" storage type, because all values for the array elements must be stored in the same record as the compact structure.

The Bytes used for the signal value of a component channel in the data record must be contained completely in the range of Bytes defined by the parent channel (channel data type "Byte Array"). This generally implies that the complete structure will be stored "en block" in

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

the record. The only exception is in case a VLSD channel is contained in the structure. In this case, instead of the signal value of the component channel, the range of Bytes of the structure only contains the offset referring to the VLSD value stored in a SDBLOCK (see 4.28 The Signal Data Block SDBLOCK). To draw an analogy to structures used in programming languages, this can be seen as if using a pointer as member of the structure: the structure itself only contains the pointer value which refers to the actual value.

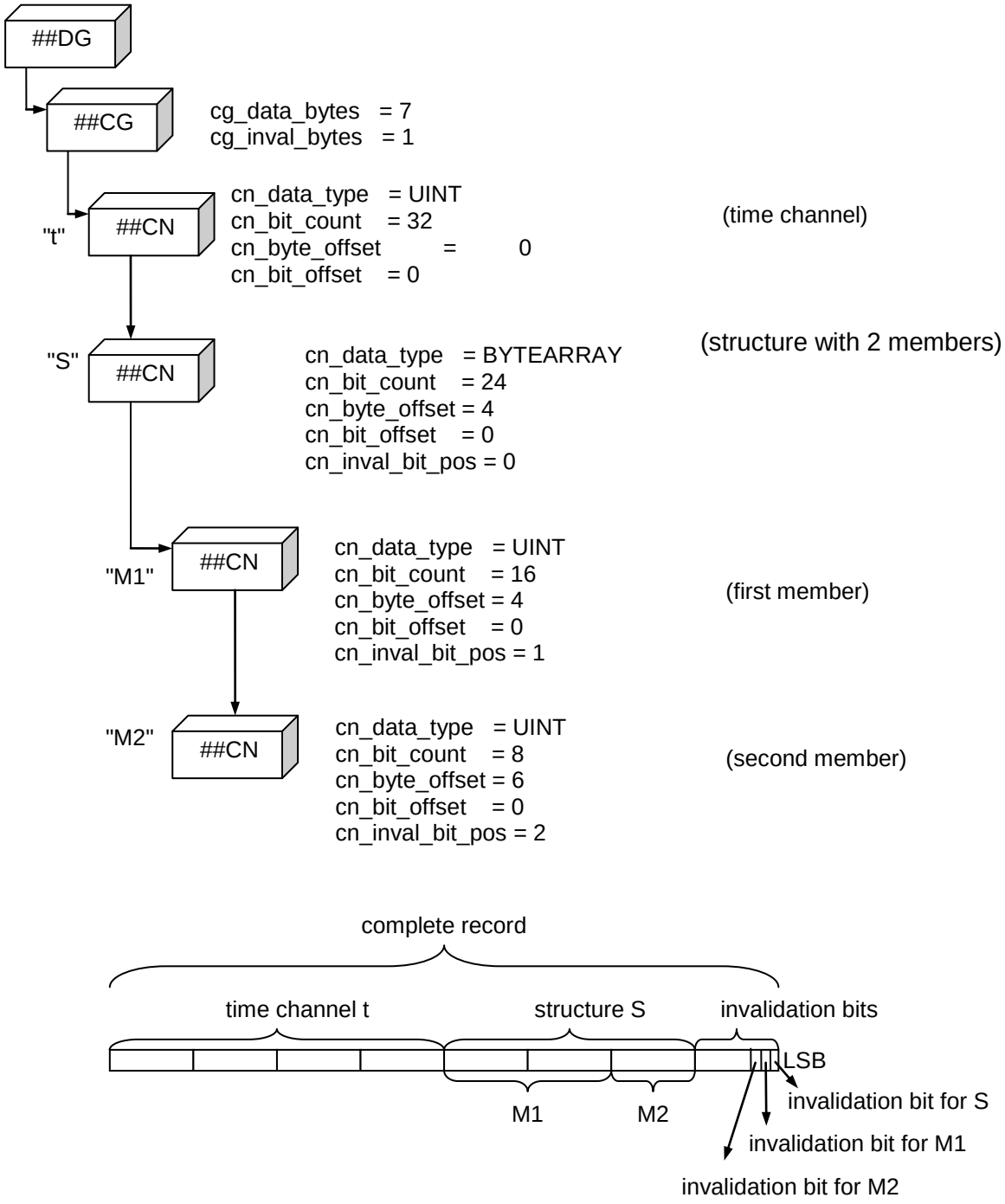


Figure 14 Example of block structure modeling a compact structure

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.18.2 Arrays

An array can be seen as N-dimensional matrix of elements (components) with equal data types. Optionally, additional information can be provided for an array, e.g. an axis or a working point (input quantity) signal for each dimension. These additional features are used when modeling maps/curves from MCD-2 MC [1] or classification results.

Generally, an array is modeled using a CNBLOCK which references a CABLOCK using the cn_composition link. This CNBLOCK will be denoted as "parent" CNBLOCK. It represents the complete array. However, unlike to a compact structure the parent CNBLOCK does not describe the range of contiguous Bytes for the complete array. It describes the first array element instead and must be used as "template" for all other elements. Depending on the storage type, the CNBLOCKs for the components are derived from the template by replacing the part of information which is individual for each element. The only exception here is the value range stored in the parent CNBLOCK (cn_val_range_min / cn_val_range_max) respectively in the associated CCBLOCK (cc_phy_range_min / cc_phy_range_max): in case of an array, the stored value range must be calculated over all elements of the array, i.e. the minimum value, for instance, is the smallest value that occurred for all array elements over the complete time range.

Like for structures, we can distinguish between "compact" and "fragmented" arrays. However, in contrast to structures, the fragmented array is not modeled using a CHBLOCK but a special storage type of the CABLOCK (see also ca_storage):

- Storage type "CN template" for "compact" arrays: all components of an array are stored in the same record, i.e. they are always recorded together.
- Storage type "CG/DG template" for "fragmented" arrays: the components of an array are distributed to several records, i.e. each element can be recorded individually and usually has its own master channel value (e.g. a time stamp). This can be used to record only a selection of elements of the array (partial storage). Another use case is to record elements only if their values change which may be more efficient with regard to data size if typically only one element or few elements of the array are changed at the same time. "CG template" storage is for unsorted data, "DG template" storage for sorted data. The latter storage even allows to distinguish if an element was not recorded at all (due to partial storage), or if it was recorded but no sample occurred (see ca_data).

The CABLOCK will be explained in the next section.

4.19 The Channel Array Block CABLOCK

The CABLOCK is used to model an array.

4.19.1 Block structure of CABLOCK

D = number of dimensions
d = dimension number $1 \le i \le D$
N(d) = number of elements for dimension d.
$\sum N(d)$ = sum of all N(d) for d = 1, …, D
$\prod N(d)$ = product of all N(d) for d = 1, …, D
k = element index $0 \le k < \prod N(d)$ (elements are stored line-oriented, see below).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 62 Structure of CABLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##CA”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
ca_composition	LINK	1	Array of composed elements: Pointer to a CNBLOCK for array of structures, or to a CABLOCK for array of arrays (can be NIL). If a CABLOCK is referenced, it must use the "CN template" storage type (ca_storage = 0). Details are explained in 4.20.2 Arrays of Composed Signals.
ca_data	LINK	Π N(d) or empty	**Only present for storage type "DG template" (ca_storage = 2).** List of links to either a data block (DT-/DVBLOCK or DZBLOCK for this block type) or a data list block (DL-/LDBLOCK or HLBLOCK if required) for each element. A link in this list may only be NIL if the cycle count of the respective element is 0: ca_data[k] = NIL => ca_cycle_count[k] = 0 If ca_data[k] is NIL, this should be interpreted that the respective array element was not recorded. If the element was recorded, but no single sample occurred for it, this should be indicated by ca_cycle_count[k] = 0, but ca_data[k] referencing an "empty" DTBLOCK (i.e. only DTBLOCK header with 24 bytes). Thus a simple distinction between "not recorded elements" and "recorded elements without samples" is possible. The links are stored line-oriented, i.e. element k uses ca_data[k] (see explanation below). The size of the list must be equal to Π N(d), i.e. to the product of the number of elements per dimension N(d) over all dimensions D. Note: link ca_data[0] must be equal to dg_data link of the parent DGBLOCK.
ca_dynamic_size	LINK	D x 3 or empty	**Only present if "dynamic size" flag (bit 0) is set.** References to channels for size signal of each dimension (can be NIL). Each reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel (either all three

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			links are assigned or NIL). Thus the links have the following order: DGBLOCK for size signal of dimension 1 CGBLOCK for size signal of dimension 1 CNBLOCK for size signal of dimension 1 … DGBLOCK for size signal of dimension D CGBLOCK for size signal of dimension D CNBLOCK for size signal of dimension D The size signal can be used to model arrays whose number of elements per dimension can vary over time. If a size signal is specified for a dimension, the number of elements for this dimension at some point in time is equal to the value of the size signal at this time (i.e. for time-synchronized signals, the size signal value with highest time stamp less or equal to current time stamp). If the size signal has no recorded signal value for this time (yet), assume 0 as size. Since each referenced channel defines the current size of a dimension, its physical values must be Integer values ≥ 0 and ≤ N(d), i.e. the values must not exceed the maximum number of elements for the respective dimension. Usually, a size signal should have some Integer data type (cn_data_type ≤ 3) without conversion rule and without unit. Since the size signal and the array signal must be synchronized, their channel groups must contain at least one common master channel type. Note: the "positions" of the array elements are fixed as defined for the maximum allowed array size. They will not change when the array is reduced by a size signal value less than N(d). However, the "removed" array elements must not be considered. For example, assume a max 2x3 matrix as in the example below. For row-oriented CN template storage, the elements would be stored in the record in following order: $a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{23}$ Assume that the size of the second dimension is reduced from 3 to 2. Then the values of elements a_{13} and a_{23} will be undefined and should not be used. However, the other elements stay at the same position (Byte offset). $a_{11}, a_{12}, [a_{13}], a_{21}, a_{22}, [a_{23}]$

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
ca_input_quantity	LINK	D x 3 or empty	**Only present if "input quantity" flag (bit 1) is set.** Reference to channels for input quantity signal for each dimension (can be NIL). Each reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel (either all three links are assigned or NIL). Thus, the links have the following order: DGBLOCK for input quantity of dimension 1 CGBLOCK for input quantity of dimension 1 CNBLOCK for input quantity of dimension 1 … DGBLOCK for input quantity of dimension D CGBLOCK for input quantity of dimension D CNBLOCK for input quantity of dimension D Since the input quantity signal and the array signal must be synchronized, their channel groups must contain at least one common master channel type. The input quantity often is denoted as "working point" because it is used for look-up of a value in the array. See MCD-2 MC [1] keyword "InputQuantity". For array type "look-up", the value of the input quantity will be used for looking up a value in the scaling axis of the respective dimension (either axis channel or fixed axis values listed in ca_axis_value list). For array type "scaling axis", the value of the input quantity will be used for looking up a value in the axis itself. The input quantity should have the same physical unit as the axis it is applied to.
ca_output_ quantity	LINK	3 or empty	**Only present if "output quantity" flag (bit 2) is set.** Reference to channel for output quantity (can be NIL). The reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel (either all three links are assigned or NIL). Since the output quantity signal and the array signal must be synchronized, their channel groups must contain at least one common master channel type. For array type "look-up", the output quantity is the result of the complete look-up (see MCD-2

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			MC [1] keyword RIP_ADDR_W). The output quantity should have the same physical unit as the array elements of the array that references it.
ca_comparison_ quantity	LINK	3 or empty	**Only present if "comparison quantity" flag (bit 3) is set.** Reference to channel for comparison quantity (can be NIL). The reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel (either all three links are assigned or NIL). Since the comparison quantity signal and the array signal must be synchronized, their channel groups must contain at least one common master channel type. The comparison quantity should have the same physical unit as the array elements. See MCD-2 MC [1] keyword COMPARISON_QUANTITY.
ca_cc_axis_ conversion	LINK	D or empty	**Only present if "axis" flag (bit 4) is set.** Pointer to a conversion rule (CCBLOCK) for the axis of each dimension. If a link NIL a 1:1 conversion must be used for this axis. If the "fixed axis" flag (Bit 5) is set, the conversion must be applied to the fixed axis values of the respective axis/dimension (ca_axis_value list stores the raw values as REAL). If the link to the CCBLOCK is NIL already the physical values are stored in the ca_axis_value list. If the "fixed axes" flag (Bit 5) is not set, the conversion must be applied to the raw values of the respective axis channel, i.e. it overrules the conversion specified for the axis channel, even if the ca_axis_conversion link is NIL! Note: ca_axis_conversion may reference the same CCBLOCK as referenced by the respective axis channel ("sharing" of CCBLOCK).
ca_axis	LINK	D x 3 or empty	**Only present if "axis" flag (bit 4) is set and "fixed axes flag" (bit 5) is not set.** References to channels for axis of respective dimension (can be NIL). Each reference is a link triple with pointer to parent DGBLOCK, parent CGBLOCK and CNBLOCK for the channel (either all three links are assigned or NIL). Thus the links have the following order:

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			DGBLOCK for axis of dimension 1 CGBLOCK for axis of dimension 1 CNBLOCK for axis of dimension 1 … DGBLOCK for axis of dimension D CGBLOCK for axis of dimension D CNBLOCK for axis of dimension D Each referenced channel must be an array of type "scaling axis" or "interval axis". For a "scaling axis" the maximum number of elements (ca_dim_size[0] in axis) must not be less than the maximum number of elements of the respective dimension d in the array which references it (ca_dim_size[d-1]). For an "interval axis" the number of elements must be exactly one more than respective dimension d in the "classification result" array (ca_dim_size[d-1]+1). If an axis signal must be synchronized with the array signal (e.g. for dynamic size axis), the channel groups of the axis and the array signal must contain at least one common master channel type. If a fixed axis is stored by an axis channel, its channel group should only contain one cycle. If a reference to an axis is NIL, there is no axis and a zero-based index should be used for axis values.
Data section			
ca_type	UINT8	1	Array type (defines semantic of the array) Attention: for some array types certain features are not allowed (see ca_flags)
			0 = Array The array is a simple D-dimensional value array (value block) without axes and without input/output/comparison quantities. (see MCD-2 MC [1] keywords VAL_BLK for CHARACTERISTIC or MATRIX_DIM/ARRAY_SIZE for MEASUREMENT)
			1 = Scaling axis The array is a scaling axis (1-dimensional vector), possibly referenced by one or more arrays. If referenced by an array of type "look-up" (see MCD-2 MC [1] keywords AXIS_PTS/AXIS_DESCR), the axis itself may have a scaling axis (e.g. CURVE_AXIS) and

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			an own input quantity.
			2 = Look-up The array is a D-dimensional array with axes. It can have input/output/comparison quantities. (see MCD-2 MC [1] keywords CURVE/MAP/CUBOID/CUBE_n)
			3 = Interval axis The array is an axis (1-dimensional vector) defining interval ranges as "axis points". It can be referenced by one or more arrays of type "classification result" (ca_type = 4). The interval axis must have at least two elements (ca_dim_size[0] = N(0) > 1) and the physical values of the elements must be strictly monotonous increasing over the element index, i.e. $a_i < a_{i+1}$ for $1 \le i < N(0)$ In contrast to a scaling axis (ca_type = 1), an interval axis always has one element more than the number of elements for the respective dimension of the classification result array which references it. The elements of the class interval axis define the borders of interval ranges that are seen as axis points. Depending on the "left-open interval" flag (bit 7 in ca_flags), the intervals are defined as $]a_o,a_1],]a_1,a_2], \ldots,]a_{N(0)-1},a_{N(0)}]$ *(flag set)* $[a_o,a_1[, [a_1,a_2[, \ldots, [a_{N(0)-1},a_{N(0)}[$ *(flag not set)* *valid since MDF 4.1.0, should not occur for earlier versions*
			4 = Classification result The array is a D-dimensional array containing classification results. It can have scaling axes (ca_type = 1) or interval axes (ca_type = 3), even mixed. Details about the classification method and parameters can be given as XML tags in cn_md_comment of the parent CNBLOCK and the CNBLOCKs of the axes. *valid since MDF 4.1.0, should not occur for earlier versions*
ca_storage	UINT8	1	Storage type (defines how the element values are stored)
			0 = CN template Values of all elements of the array are stored in the same record (i.e. all elements are measured together).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			The parent CNBLOCK defines the first element in the record (k = 0). All other elements are defined by the same CNBLOCK except that the values for cn_byte_offset and cn_inval_bit_pos change for each component (see explanation below).
			1 = CG template Value for each element of the array is stored in a separate record (i.e. elements are stored independently of each other). All records are stored in the same data block referenced by the parent DGBLOCK. As a consequence, the data group (and thus the MDF file) is unsorted. All elements have exactly the same record layout specified by the parent CGBLOCK. However, each element uses a different cycle count (given by ca_cycle_count[k]) and a different record ID which must be calculated by "auto-increment" of the record ID of the parent CGBLOCK: cg_record_id + k. Since ca_cycle_count[0] must be equal to cg_cycle_count of the parent CGBLOCK, the parent CNBLOCK of the CABLOCK automatically describes the first array element (k = 0). When sorting a data group, a CABLOCK with "CG template" storage will be converted to a CABLOCK with "DG template" storage. If there are no samples for an element (i.e. ca_cycle_count[k] = 0), then this conversion should set the ca_data[k] link to NIL for the "DG template" storage unless it has further knowledge whether the element was recorded or not (see also description for ca_data). This means that for storage type "CG template" a simple distinction between "not recorded elements" and "recorded elements without samples" like for "DG template" storage is not available.
			2 = DG template Similar to CG template, the value of each element of the array is stored in a separate record (i.e. elements are stored independently of each other). However, the records for each element are stored in separate data blocks referenced by the list of links in ca_data. Similar to "CG template" storage, all elements have exactly the same record layout (defined by the parent CGBLOCK) but a different cycle

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			count (specified by ca_cycle_count[k], see below). Since ca_cycle_count[0] must be equal to cg_cycle_count of the parent CGBLOCK, and ca_data[0] must be equal to dg_data of the parent DGBLOCK, the parent CNBLOCK of the CABLOCK automatically describes the first array element (k = 0).
ca_ndim	UINT16	1	Number of dimensions D > 0 For array type "scaling axis" or "interval axis", D must be 1.
ca_flags	UINT32	1	Flags The value contains the following bit flags (Bit 0 = LSB):
			Bit 0: dynamic size flag If set, the number of scaling points for the array is not fixed but can vary over time. Can only be set for array types "look-up" and "scaling axis".
			Bit 1: input quantity flag If set, a channel for the input quantity is specified for each dimension by ca_input_quantity. Can only be set for array types "look-up" and "scaling axis".
			Bit 2: output quantity flag If set, a channel for the output quantity is specified by ca_output_quantity. Can only be set for array type "look-up".
			Bit 3: comparison quantity flag If set, a channel for the comparison quantity is specified by ca_comparison_quantity. Can only be set for array type "look-up".
			Bit 4: axis flag If set, a scaling axis is given for each dimension of the array, either as fixed or as dynamic axis, depending on "fixed axis" flag (bit 5). Can only be set for array types "look-up", "scaling axis" and "classification result".
			Bit 5: fixed axis flag If set, the scaling axis is fixed and the axis points are stored as raw values in ca_axis_value list. If not set, the scaling axis may vary over time and the axis points are stored as channel referenced in ca_axis for each dimension. Only relevant if "axis" flag (bit 4) is set.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Can only be set for array types "look-up" and "scaling axis".
			Bit 6: inverse layout flag If set, the record layout is "column oriented" instead of "row oriented". See MCD-2 MC [1] keywords ROW_DIR and COLUMN_DIR. Only relevant for "CN template" storage type and for number of dimensions > 1.
			Bit 7: left-open interval flag If set, the interval ranges for the class interval axes are left-open and right-closed, i.e.]a,b] = {x | a < x ≤ b}. If not set, the interval ranges for the class interval axes are left-closed and right-open, i.e. [a,b[= {x | a ≤ x < b}. Note that opening or closing is irrelevant for infinite endpoints (there can only be -INF for the left border of the very first interval, or +INF for right border of the very last interval). Only relevant for array type "interval axes". *valid since MDF 4.1.0, should not be set for earlier versions*
			Bit 8: standard axis flag If set, the axis described by the CABLOCK is a standard axis (see MCD-2 MC [1] STD_AXIS for keyword AXIS_DESCR). Unlike common axes – standard axes are not shared between look-up channels (curves, maps, cuboids). They are part of the look-up channel they belong to. Can only be set for array type "scaling axis". *valid since MDF 4.2.0, should not be set for earlier versions*

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

<table>
<tr><th>Name</th><th>Type</th><th>Count</th><th>Description</th></tr>
<tr><td></td><td></td><td></td><td>The following table summarizes the allowed bit flags for each array type:
<table>
<tr><th>ca_type / bit flag</th><th>0 = array</th><th>1 = scaling axis</th><th>2 = look-up</th><th>3 = interval axis</th><th>4 = classification</th></tr>
<tr><td>bit 0 = dynamic size</td><td></td><td>x</td><td>x</td><td></td><td></td></tr>
<tr><td>bit 1 = input quantity</td><td></td><td>x</td><td>x</td><td></td><td></td></tr>
<tr><td>bit 2 = output quantity</td><td></td><td></td><td>x</td><td></td><td></td></tr>
<tr><td>bit 3 = comparison quantity</td><td></td><td></td><td>x</td><td></td><td></td></tr>
<tr><td>bit 4 = axis</td><td></td><td>x</td><td>x</td><td></td><td>x</td></tr>
<tr><td>bit 5 (and bit 4) = fixed axis</td><td></td><td>x</td><td>x</td><td></td><td></td></tr>
<tr><td>bit 6 = inverse layout</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr>
<tr><td>bit 7 = interval left-open</td><td></td><td></td><td></td><td>x</td><td></td></tr>
<tr><td>bit 8 = standard axis flag</td><td></td><td>x</td><td></td><td></td><td></td></tr>
</table></td></tr>
<tr><td>ca_byte_offset_ base</td><td>INT32</td><td>1</td><td>Base factor for calculation of Byte offsets for "CN template" storage type.
The absolute value of ca_byte_offset_base should be larger than or equal to the size of Bytes required to store a component channel value in the record (all must have the same size). If the values are equal, then the component values are stored contiguously, i.e. next to each other without gaps.
If ca_byte_offset_base is negative, the component values are stored with decreasing index, see MCD-2 MC [1] keyword INDEX_DECR.
Exact formula for calculation of Byte offset for each component channel see below.</td></tr>
<tr><td>ca_inval_bit_pos_ base</td><td>UINT32</td><td>1</td><td>Base factor for calculation of invalidation bit positions for CN template storage type.
For a simple array which is not part of a nested composition, ca_inval_bit_pos_base =</td></tr>
</table>

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			1 means that the invalidation bits for the component channels are stored without gaps and ca_inval_bit_pos_base = 0 means that all component channels use the same invalidation bit in the record. Exact formula for calculation of invalidation bit position for each component channel see below.
ca_dim_size	UINT64	D	Maximum number of elements for each dimension N(d). N(d) > 0 for all d. For array type "interval axis", there must be at least two elements, i.e. N(0) > 1.

ca_axis_value	REAL	∑ N(d) or empty	**Only present if "fixed axes" flag (bit 5) is set.**
			List of raw values for (fixed) axis points of each axis.
			The axis points are stored in a linear sequence for each dimension:
			`A1(1), A1(2), ..., A1(N(1)),`
			`A2(1), A2(2), ..., A2(N(2)),`
			`...`
			`AD(1), AD(2), ..., AD(N(D))`
			where $A_d(j)$ is axis point `j` of dimension `d` and `N(d)` is the number of elements for dimension `d`.
			
			Example for D = 2 dimensions (as explained later Y axis for A_1 and X axis for A_2):
			`Y(1), Y(2), ..., Y(N(1)),`
			`X(1), X(2), ..., X(N(2))`
			The size of the list must be equal to ∑ N(d) = the sum of the number of elements per dimension N(d) over all dimensions D.
			To convert the raw axis values to physical values, the conversion rule of the respective dimension must be applied (CCBLOCK referenced by ca_cc_axis_conversion[i]). If the CCBLOCK link of a dimension is NIL, the raw values are equal to the physical values (1:1 conversion).

ca_cycle_count	UINT64	Π N(d) or empty	**Only present for storage types "CG/DG template".** List of cycle counts for each element in case of "CG/DG template" storage. The offsets are stored line-oriented, i.e. element k uses ca_cycle_count[k] (see explanation below)

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			The size of the list must be equal to Π N(d) = the product of the number of elements per dimension N(d) over all dimensions D. Note: ca_cycle_count[0] must be equal to cg_cycle_count of parent CGBLOCK!

The CABLOCK defines the "elements" of the array signal. Either the values of all array elements are stored in the same record (CN template, all elements measured together), or the value for each element is stored in its own record (CG/DG template, elements measured independent of each other). A mixture is not possible.

Index k for reference to element-specific information

To store or reference the information that is individual for each element, the elements of the array are considered to be listed in a simple "flat" list (1-dimensional array). The length of this list will be equal to the product of the element count per dimension over all dimensions, denoted as Π N(d). This list must be interpreted line-oriented, see example below. The zero-based index k for the position in this list is used to refer to an array element, e.g. when calculating the record ID for CG template, or when referencing the link to the data block in ca_data list for DG template.

Example for line-oriented interpretation:

A 2 x 3 matrix M = $\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$ would be stored as follows: a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{23}

k = 0 thus denotes element a_{11}, k = 1 element a_{12}, etc.

Replacing the elements with their index in the matrix, we get $\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$

More generally, an $i \times j \times n$ Matrix would be listed as $a_{111}, \ldots a_{11n}, a_{121}, \ldots a_{1jn}, a_{211}, \ldots a_{ijn}$
(see also MCD-2 MC [1] keyword ROW_DIR).
In this case,

ca_ndim	= D	= 3
ca_dim_size[0]	= N(1)	= i
ca_dim_size[1]	= N(2)	= j
ca_dim_size[2]	= N(3)	= n.

Note that k also can be calculated by a formula equal to the calculation of offset **$O(i_1, \ldots, i_D)$ below when assuming row-orientation and a base value of 1.**

Please keep in mind that an XY map usually will be modeled by N(1) = Y and N(2) = X, and an XYZ cube accordingly by N(1) = Z, N(2) = Y and N(3) = X.

Storage Types

- Each element of the array is an individual channel, but only the first element has a representation as CNBLOCK, i.e. the parent CNBLOCK of the CABLOCK. The channel descriptions of all other elements are derived from this CNBLOCK and, depending on the storage type, from its parent CGBLOCK/DGBLOCK: for "CN template" storage, the parent CNBLOCK is used as template. The channel

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

description of an individual array element is derived by replacing cn_byte_offset and (if invalidation bit is used) cn_inval_bit_pos. Details about how to calculate the values for replacement see below.

- for "CG template" storage the parent CGBLOCK (including all child CNBLOCKs) is used as template. The channel group of an individual array element is derived by replacing cg_record_id and cg_cycle_count. Figure 15 shows an example.
- for "DG template" storage the parent DGBLOCK (including its single child CGBLOCK and all child CNBLOCKs) is used as template. The data group of an individual array element is derived by replacing dg_data for DGBLOCK, and cg_cycle_count for CGBLOCK. Figure 16 shows a example.

Both for CG and DG template, the CNBLOCKs are not changed, i.e. each record has the same layout. The channel group should only contain the CNBLOCK of the array (i.e. the parent of the CABLOCK) and CNBLOCKs with master channel type. Since for both CG and DG template the CGBLOCK is used as template, these storage types are only possible if the CABLOCK is a direct child of a CNBLOCK, and if this CNBLOCK is a direct child of a CGBLOCK. This means that CG/DG template storage is not possible for a component of a compact structure or for the element of an array (see also 4.20 Nested Composition of Channels).

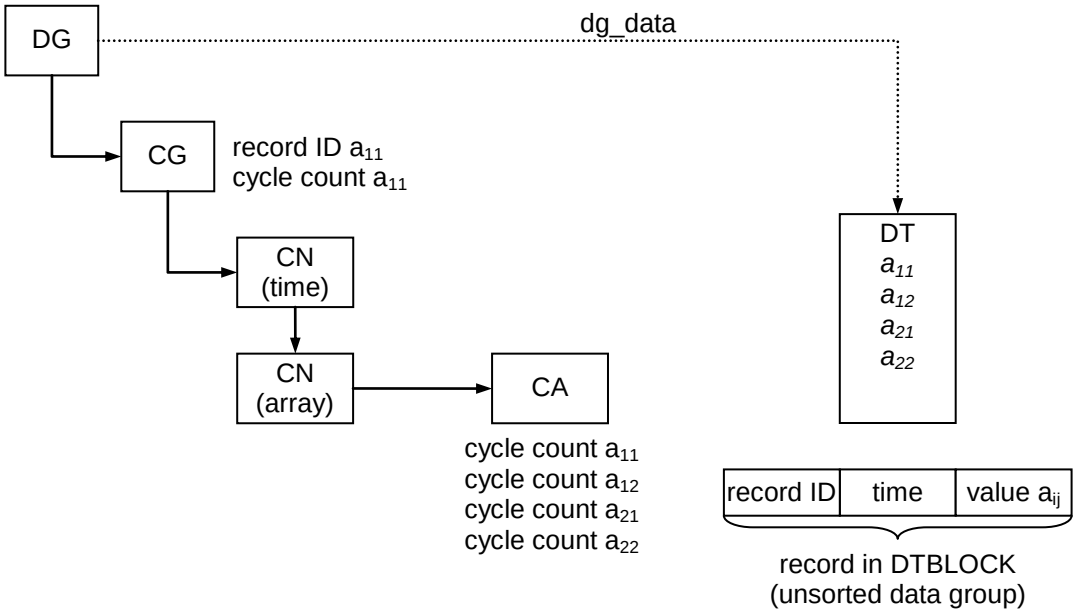


Figure 15 Example for block structure using CG template storage

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

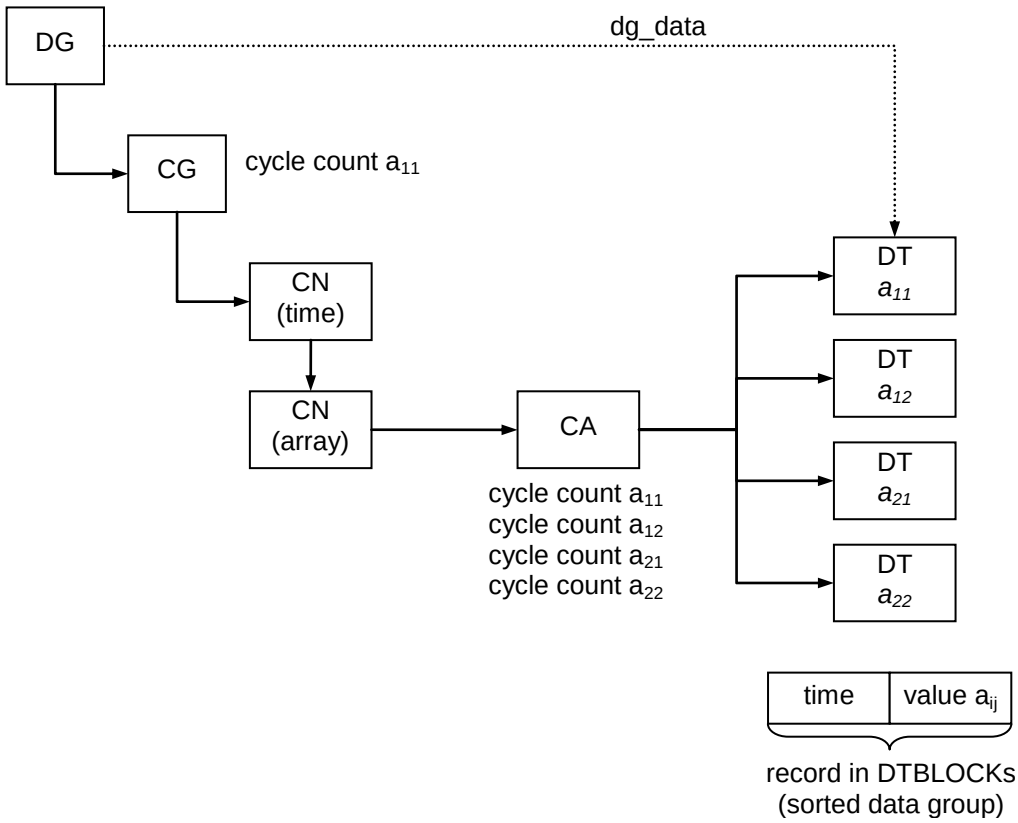


Figure 16 Example for block structure using DG template storage

Note: One use case for CG/DG template is to save only those element values of the array that really have changed. If typically only one or only few elements of the array are changed, then saving always all of the element values would be inefficient with regard to the file size. Another use case is to save only a part of the elements of the array (partial storage).

Calculation of Byte offsets and invalidation bit position for "CN template"

In case of "CN template" storage type, the values of all array elements are stored in the same record. The description of each element is derived from the parent CNBLOCK which servers as template by calculating a new value for cn_byte_offset and for cn_inval_bit_pos.

D = ca_ndim = number of dimensions
d = dimension number 1 ≤ d ≤ D
N(d) = ca_dim_size[d-1] = number of axis points for dimension d
id = zero-based index of dimension d
O(i1, …,iD) = calculated offset for individual array element specified by indices i1, …,iD.
f(d) = factor calculated for dimension d (used in formula for calculation of O(i1, …,iD))
B = ca_byte_offset_base = base value

If the record layout is "row oriented", i.e. the "inverse layout" flag (bit 6) is not set (low), then the factors f(d) are calculated as follows:

f(d) = B for d = D
f(d) = f(d+1) x N(d+1) for 1 ≤ d < D

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

If the record layout is "column oriented", i.e. the "inverse layout" flag (bit 6) is set (high), then the factors f(d) are calculated as follows:
f(d) = B for d = 1
f(d) = f(d-1) x N(d-1) for 1 < d ≤ D

Table 63 below lists the most common cases for D = 1, D = 2 and D = 3. Note that in case of D = 1, row and column orientation does not make a difference.

Table 63 Factors f(d) for common use cases

D = 1	**row oriented layout**	**column oriented layout**
f(1)	B	B
D = 2	**row oriented layout**	**column oriented layout**
f(1)	f(2) x N(2) = B x N(2)	B
f(2)	B	f(1) x N(1) = B x N(1)
D = 3	**row oriented layout**	**column oriented layout**
f(1)	f(2) x N(2) = B x N(3) x N(2)	B
f(2)	f(3) x N(3) = B x N(3)	f(1) x N(1) = B x N(1)
f(3)	B	f(2) x N(2) = B x N(1) x N(2)

These factors then are used to calculate the offset to be added to cn_byte_offset of the template CNBLOCK to get the Byte offset for the component channel specified by indices i1, …,iD :
O(i1, …,iD) = i1 x f(1) + … + id x f(d) = ∑ in x f(n) for all 0 ≤ n < D.

Note that B may be negative which results in negative factors f(d). In this case, the component with all indices i_d = 0 (represented by the parent CNBLOCK) is not at the beginning but at the end of the **range of contiguous Bytes** in the record. In MCD-2 MC [1] this special case will be denoted by keyword INDEX_DECR.

The calculation of the offset to be added to cn_inval_bit_pos is similar, just that here B = ca_inval_bit_base must be used. In contrast to ca_byte_offset_base, ca_inval_bit_base cannot be negative.

Note that for cn_byte_offset a Byte value is changed, whereas for cn_inval_bit_pos a bit value is changed.

Reference of scaling axes

For array types "look-up", "classification result" and "scaling axis" a scaling axis can be specified for each dimension. A scaling axis can be "fixed", i.e. its axis points cannot change, or it can be "variable", i.e. its axis points may vary over time.

If each scaling axis is fixed, the axis values simply can be stored in the data section of the CABLOCK (ca_axis_value). Otherwise, for each axis a channel with array type "scaling axis" can be referenced. The axis channel then contains the "measured" axis points.

Note: An axis channel may also be used for a fixed axis because it can store further information about the axis. In this case, the axis channel for the fixed axis should use "CN template" storage. Its channel group should only have one sample

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

(cg_cycle_count = 1) and it should not contain a master channel (to indicate that the axis values are valid all the time).

Reference of interval axes

For array type "classification result" alternative to a "scaling axis" an "interval axis" can be specified for any dimension. The interval axis defines a discrete set of interval ranges as axis points. Usually an "interval axis" is "fixed", i.e. its axis points (the interval ranges) do not change over time. Nevertheless, an interval axis can only be specified by using an axis channel (see note above).

4.20 Nested Composition of Channels

4.20.1 Structures of Composed Signals

The components (members) of a structure can be composed signals, too.

For a compact structure, one or more of its component CNBLOCK may have a cn_composition link to either a CNBLOCK (structure member is a structure itself) or to CABLOCK (structure member is an array). The component CNBLOCK of the main structure thus is the parent CNBLOCK for its own components and the same rules apply as if it were a direct child of a CGBLOCK.

In both cases, all element values of the composed signal for each member must be stored in the same record and within the range of contiguous Bytes of the main (compact) structure. This implies that the CABLOCK for an array element must use "CN template" storage (all array elements stored in the same record).

For a fragmented structure modeled by a CHBLOCK of type "structure", a CNBLOCK referenced by the CHBLOCK also may have a cn_composition link to a CNBLOCK or a CABLOCK. However, for the CABLOCK in this case also CG/DG template storage is possible. In addition, the CHBLOCK of the main structure even may reference as component some other CHBLOCK of type "structure" (an example of a sub structure CHBLOCK is illustrated in Figure 13).

4.20.2 Arrays of Composed Signals

The components (elements) of an array can be composed signals, too. In this case, the CABLOCK of the main array then can contain a ca_composition link either to a CNBLOCK (array of structures) or to a CABLOCK (array of arrays).

Array of structures

If a CNBLOCK is referenced in ca_composition, this CNBLOCK is the start of a forward linked list (via the cn_cn_next links) of the component channels which describe the layout of one element of the array. This is very similar to when a component channel is referenced by cn_composition from a CNBLOCK, except that here the component channels serve as template for all array elements.
The parent CNBLOCK of the CABLOCK (used as template for the complete structure) must have data type "Byte Array" and describes the range of contiguous Bytes for the first array element (which, as all others, is a structure).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

- For "CG/DG template" storage of the main array, the referenced component CNBLOCKs for the structure members describe how to decode the structure members within each record. The rules to derive the current data and channel group from the templates are unchanged.
- For "CN template" storage of the main array, the referenced component CNBLOCKs for the structure members describe how to decode the structure members for the first array element. For the other array elements, the component CNBLOCKs serve as CN templates, like the parent (root) CNBLOCK of the CABLOCK. Deriving the description for the current array element, an offset will be added to cn_byte_offset and cn_inval_bit_pos of the component CNBLOCKs. The calculation of this offset is the same as the calculation for arrays without structures.

Array of Arrays

If a CABLOCK is referenced in ca_composition, this CABLOCK defines the sub elements of the array element. The CNBLOCK parent of the main array describes the first sub element. The CABLOCK of the sub array must always use "CN template" storage, i.e. the sub elements can only be measured together.

- For "CG/DG template" storage of the main array, the CABLOCK for the sub array defines how the sub elements are stored in the record of the main element (each main element has its own record). The (grand-)parent CNBLOCK of the CABLOCK for the sub array is used as CN template just as if the CABLOCK with the "CG/DG template" storage was not there. The only difference is that the respective record individual for each main array element must be used (according to the rules for "CG/DG template").
- For "CN template" storage of the main array, the (grand-)parent CNBLOCK here defines how the very first element of the sub array is stored in the record containing all elements. In order to derive the description for a current sub array element, an offset must be added to cn_byte_offset and cn_inval_bit_pos of the template CNBLOCK. The calculation of this offset is similarly to an ordinary "CN template", but must consider all CABLOCKs with "CN template" between the parent CNBLOCK (direct child of a CGBLOCK) and the current CABLOCK. Descending the composition link chain from the root CNBLOCK, we enumerate the CABLOCKs using "CN template" storage form 1 to n. Parameters that belongs to a CABLOCK will be marked by the respective enumeration value in subscript, and parameters used for the nested array by an asterisk (*) in superscript. The total number of dimensions D* of the nested array then is the sum of the dimensions over all CABLOCKs, i.e. $D^* = \sum D_i$. The dimension sizes of the nested array are

  $N^*(1) = N1(1)$
  $N^*(2) = N1(2)$
  …
  $N^*(D1) = N1(D1)$
  $N^*(D1+1) = N2(1)$
  …
  $N^*(D^*) = Nn(Dn)$

- Equally we use the factors $f_i(d)$ as calculated for each CABLOCK using the respective ca_byte_offset_base$_i$ as base value:
  $f^*(1) = f_1(1)$
  $f^*(2) = f_1(2)$
  …
  $f^*(D^*) = f_n(D_n)$

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

- Using the factors f* and the new enumeration of the dimensions 1 ≤ d ≤ D* in the nested array, we can calculate the offset O(i*1, …,i*D*) to be added to cn_byte_offset of the template CNBLOCK similar to a normal "CN template". For the invalidation bit offset, the same approach is used, just that calculating the factors fi(d) we must use ca_inval_bit_basei as base value. See also example below.

Note: A chain of CABLOCKs with CN template can usually be modelled by one single CABLOCK which combines all dimensions except for the following use cases:

- there are different axes types for the dimensions, i.e. different states for fixed axis flag (Bit 5) in ca_flags
- there are different layouts (row/column orientation) for the dimensions, i.e. different states for inverse layout flag (Bit 6) in ca_flags
- there are extra Bytes between the elements of one of the outer dimensions. For an example assume a 2 x 3 matrix with row oriented layout as shown above where each element uses one Byte, but there is an extra "padding" Byte after each row: a_{11}, a_{12}, a_{13}, [extra Byte], a_{21}, a_{22}, $a_{23,}$ [extra Byte] This can be modeled only by two CABLOCKs where ca_byte_offset_base = 1 for the inner CABLOCK and ca_byte_offset_base = 4 for the outer CABLOCK.

If possible, the usage of a single CABLOCK should be preferred, unless the chain of CABLOCKs is used to illustrate the "grouping" of the dimensions, e.g. to model a vector of 2D arrays (in contrast to a 3D array).

Example

Consider the MDF block hierarchy illustrated in Figure 17. It shows a channel group with two channels. The first one is the time channel "t", the second is a channel with name "A" that describes a composed signal that is an array of nested structures. The first "component level" is a matrix with m = 2 rows and n = 3 columns (for simplicity denoted as CA1). Since the CABLOCK uses "CG template" storage, the data group is unsorted.

For simplicity, we will denote the components of a specific CABLOCK by *<CAx>.<member name>* where CAx is the "name" of the CABLOCK, e.g. CA1 for the first one.
For the array indices, we use the notation of C/C++, i.e. CA1[m][n] would be the array element with index m for the first dimension (rows) and index n for the second dimension (columns).

Let us consider element CA1[1][0], i.e. the first element of the second row (each row has 3 entries, i.e. 3 columns). It is described by element index k1 = 3. Due to CG template storage, the values for CA1[1][0] will be stored in all records in the DTBLOCK with record ID calculated by base record ID specified in CGBLOCK + element index k1 for CA1 = cg_record_id + k1 = 6 + 3 = 9.

Within the records with record ID 9, the layout is specified by the parent CGBLOCK, i.e. after the record ID there is the time stamp and then the elements of the composed signal.

The next component levels (CA2 and CA3) are 1-dimenstional arrays defined by CABLOCKs using "CN template" storage with 3 resp. 2 elements. The "base" element is a structure that requires 3 Bytes (CNBLOCK "A") and has two members denoted as "M1" and "M2", of which the second one is a structure, too.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

To determine the Byte offset for a component channel of CA3, we have to combine the factors of CA2 and CA3 (CA1 does not use "CN template", thus it will not be considered here). The number of dimensions is the sum of all dimensions of the CABLOCKs with "CN template", i.e. $D_{CA2}+D_{CA3} =$ CA2.ca_ndim+CA3.ca_ndim = 1+1 = 2. The enumeration of the combined dimension starts as usually with 1.

f*(1) = f_{CA2}(1) = CA2.ca_byte_offset_base = 6
f*(2) = f_{CA3}(1) = CA3.ca_byte_offset_base = 3

For array element CA2[1].CA3[1], we thus have an offset of
O(i*$_1$,i*$_2$) = O(1,1) = f*(1) x i*$_1$ + f*(2) x i*$_2$ = 6 x 1 + 3 x 1 = 9.

For struct member M2.M4, we calculate the offset M4.cn_byte_offset + 9 = 6 + 9 = 15.

For the invalidation bit position, we use
f*(1) = f_{CA2}(1) = CA2.ca_inval_bit_base = 8
f*(2) = f_{CA3}(1) = CA3.ca_inval_bit_base = 4
and obtain for struct member M2.M4 the bit position M4.cn_inval_bit_pos + 1 x 8 + 1 x 4 = 3 + 8 + 4 = 15, i.e. the invalidation bit has a bit offset of (15 mod 8) = 7 and a Byte offset of (15 div 8) = 1 within the invalidation Bytes.

Note: Since both CA2 and CA3 only have one dimension each (i.e. both are vectors), row or column orientation is not relevant here. As stated above, the two CABLOCKs could be combined to one single 2-dimensional CABLOCK unless they have different axes types (e.g. CA2 stores the fixed axis values whereas CA3 references an axis channel).

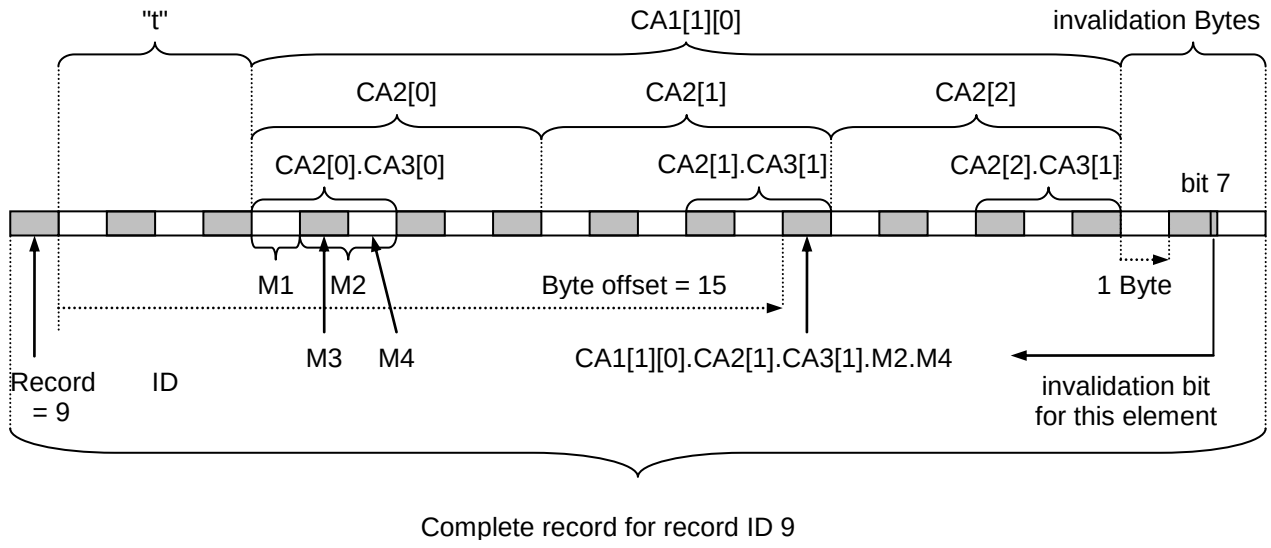


Figure 17 Record layout for composed signal of example

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

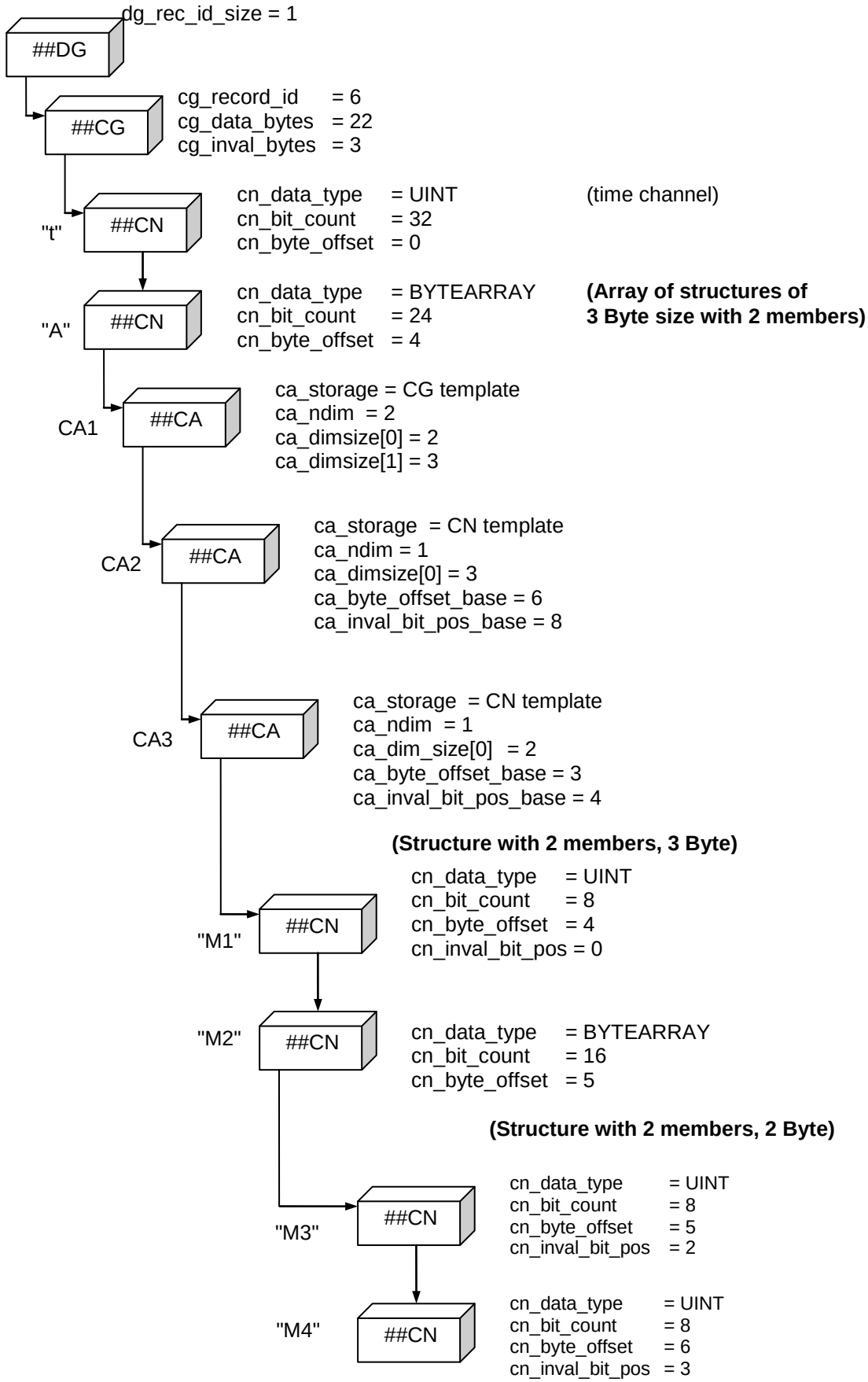


Figure 18 Block structure for composed signal of example

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.21 The Data Block DTBLOCK

The data section of the DTBLOCK contains a sequence of records. It contains records of all channel groups assigned to its parent DGBLOCK.

4.21.1 Block structure of DTBLOCK

Table 64 Structure of DTBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##DT”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			
dt_data	…	variable	….

4.21.2 Data Block Format

The recording of a signal results in a series of signal values. The signal values, often also denoted as "samples", are stored in the data block. A data block always is related to a data group and can only contain signal values for channels respectively channel groups out of this data group.

Signal values which are always acquired together (i.e. at the same instant of time) are collected and encoded in so-called data records. A data block thus consists of an arbitrary number of data records. The data records are stored Byte aligned in the data block without any gaps.

Each data record normally also contains the time stamp when it was recorded and/or some other synchronization value (e.g. an angle for angle-based measurements). The synchronization values are described by so-called "master" channels. Master channel values are always relative to their respective start value specified globally for the measurement file in HDBLOCK (usually used to denote the start of the measurement).

In addition to the signal data values, a record optionally may contain so-called "invalidation bits". Each invalidation bit is associated to one or more channels and can be used to indicate that the corresponding signal value in the current record is invalid.

All channels whose signal values are stored in the same data record are collected in a channel group. Vice versa, the signal values of all channels in a channel group are acquired simultaneously. The size and layout of a record for a channel group generally is fixed (except for a variable length signal data (VLSD) channel group, see section 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK).

The layout of a data record is defined by the channels of the respective channel group. Each channel specifies how its corresponding value is encoded in the data Bytes of the record, i.e. the data type, the number of bits and the position (Byte and bit offset) of the value.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

The value in the record is either the actual signal value or a reference to the signal value stored in a different location (special cases for signal values with variable length and synchronized streams):

- For channel type "variable length data channel" (cn_type = 1), the value in the record is only an unsigned integer offset value used for indirect addressing of the actual signal value in the SDBLOCK referenced by cn_data.
- For the channel type "synchronization channel" (cn_type = 4), the value in the record is only a synchronization value used for look-up of the actual signal value in a stream specified by the ATBLOCK referenced by cn_data.

4.21.3 Sorted and Unsorted Data

In general, a data group can either be "sorted" or "unsorted". A sorted data group allows index based and thus generally faster read access for individual signal values, whereas for recording measurement data it usually is easier and faster to write an unsorted data group, because this allows "streaming" of the records, i.e. just appending them to the end of the file.

A sorted data group requires that its data block may only contain fixed-length data records of the same type, i.e. of the same channel group. As a consequence, the sorted data group cannot contain more than one channel group (hence, it may not contain a VLSD CGBLOCK). Also, a sorted data group cannot contain a channel with a CABLOCK using the "CG template" storage type (see 4.19 The Channel Array Block CABLOCK).

In case of an unsorted data group, it may contain several channel groups and its data block may contain records of the different channel groups. To be able to distinguish the different record types (of the different channel groups), each record must be preceded by a record ID. The record ID is an unsigned Integer value using 1, 2, 4 or 8 Bytes (see dg_rec_id_size). Each record thus is clearly related to a channel group.

Since a sorted data group only contains records of the same type the record ID is usually omitted by setting dg_rec_id_size to zero. Apart from the record ID, the record structure itself is identical for sorted and unsorted data groups.

Reading a data block of an unsorted data group, the records have to be read sequentially from the start of the data block. For each record, the record ID must be checked and the size of the record for the respective channel group must be considered. For a sorted data group, each record in the data block has the same length, thus index-based access is possible because the start address of a record can be calculated for a given record index by multiplication with the record size.

Please note that the terms "sorted" and "unsorted" do not apply to the sorting of the records within a data block. The distribution of records of different types in a data block of an (unsorted) data group is arbitrary. However, when considering the sequence of records of the same record type within a data block (regardless if the data group is sorted or unsorted), the values of each master channel in the record (e.g. time stamps or angle values) must be monotonously increasing[5]. That means that when advancing from one record to the next (of the same type), the master channel value in the record must be larger than or equal to in the previous one. If this is not the case, this should be considered as an error.

[5] A non-strictly monotonously increasing master channel is an indicator for an error in the measurement setup in most cases though.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

If all data groups in an MDF file are sorted, the MDF file will be called "sorted". If at least one data group is unsorted, the MDF file is called "unsorted". Normally a recording tool will write unsorted MDF files. The sorting of the unsorted file may be done by the recording tool after recording has finished or by an analysis tool when opening the file for the first time.

4.21.4 DLBLOCK vs. LDBLOCK vs. single Data Block

There are multiple different ways to store data inside a MDF file. The appropriate way depends on the circumstances while recording and the usage scenarios.

Using a single data block (DT- or DVBLOCK) is available for all configurations – unsorted, sorted and sorted with remote master. It ensures that the data is not spread over the whole file. Reading all the data from the data block does not require seeks. However, there are also drawbacks for this method. If multiple data groups are written, the amount of data for each group needs to be known up front to reserve the appropriate space. The written block will also usually exceed the recommended size for compression. Using a single DTBLOCK is the usual way for writing unsorted files.

The DLBLOCK removes the need of knowing the space up front. Instead, another DTBLOCK can be appended to the DLBLOCK when the current DTBLOCK is full. This method allows writing a sorted file directly without special post-processing at the expense of additional buffers (usually one for each group). It also gives the tool control over the DTBLOCK size to enable efficient compression. A general recommendation is to choose the DTBLOCK size in a way that records are not split across multiple DTBLOCKs.

LDBLOCKs provide similar functionality as the DLBOCKs. The difference between those configurations is that LDBLOCKs store the invalidation bits in a separate location – or as a NIL-link if not required. This optimization compared to the DLBLOCK allows for faster reading & faster detection of invalid samples. It is also the only allowed configuration for CGBLOCKs using the remote master feature.

4.21.5 Reading Signal Values from a Data Record

The structure of the data record is defined by the following elements of CNBLOCK:

- channel data type (cn_data_type)
- number of bits (cn_bit_count)
- Byte offset (cn_byte_offset)
- bit offset (cn_bit_offset)
- position of invalidation bit (cn_inval_bit_pos) (counted from start of range for invalidation Bytes)

4.21.5.1 Reading the Invalidation Bit

If the "all values invalid" flag (bit 0) in cn_flags is set (1), all values of this channel are considered to be invalid. If both the "all values invalid" flag (bit 0) and the "invalidation bit valid" flag (bit 1) in cn_flags are not set (0), then all values of this channel are considered to be valid. For all other cases we have to examine for each value the specified invalidation bit, as explained in this section.

The invalidation Bytes are a range of Bytes in the record (length is specified by cg_inval_bytes) directly after the data Bytes (which contain the signal values). The invalidation bit is a single bit within the invalidation Bytes. It could be considered as a 1-bit signal and is read similar to the bits of a signal value (see below), just that its bit and Byte

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

offset must be calculated from cn_inval_bit_pos, and that the bit and Byte offset of the invalidation bit must be applied to the invalidation Bytes instead to the data Bytes. Since we only have a single bit, no Byte order has to be considered.

Steps to read the invalidation bit value within a record:

1) Skip record ID and data Bytes (dg_rec_id_size + cg_data_bytes)
2) Read Byte specified by (cn_inval_bit_pos >> 3)
3) Within this Byte read the bit specified by (cn_inval_bit_pos & 0x07)
4) If the bit is set (1), then the signal value is invalid, otherwise it is valid.

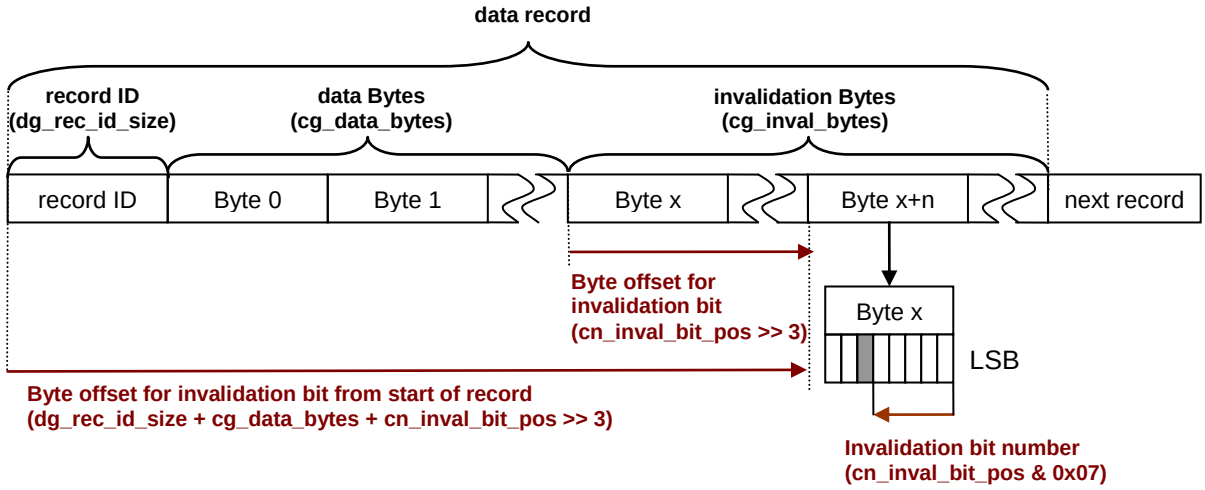


Figure 19 Invalidation Bit

4.21.5.2 Reading the Signal Value

The signal value is encoded in the data Bytes of the record, i.e. in the range of Bytes directly after the record ID with a length given by cg_data_bytes. The start of the signal value is always counted from the start of the data Bytes. So, if a record ID is present, we first have to skip the specified number of Bytes used for it. Then the data Byte offset determines the first Byte of the "signal Bytes", i.e. the first Byte of the range of Bytes that contains all bits of the signal value.

Next, we copy the signal Bytes to a memory buffer. Here we have to consider the Byte order specified by the channel data type. If the specified Byte order is different to the memory system our evaluation program runs on, we have to adapt the Byte order. This means that we have to swap the required Bytes when copying them to a memory buffer (see example below).

Now we apply the start bit offset to the memory buffer, i.e. we right-shift the bits by cn_bit_offset. As a result, the LSB of the signal becomes the LSB of the first Byte of the buffer. We now can interpret the number of bits for the signal as value, which possibly requires masking unused bits.

Recapitulating, the steps to de-code the record value are

1) Skip Bytes used for record ID (dg_rec_id_size)
2) Apply start Byte offset (cn_byte_offset)
3) Copy signal Bytes to memory buffer

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4) Possibly swap signal Bytes (depending on Byte order)
5) Apply bit offset = right shift memory buffer by cn_bit_offset bits
6) Mask unused bits
7) Read and interpret the value in memory buffer

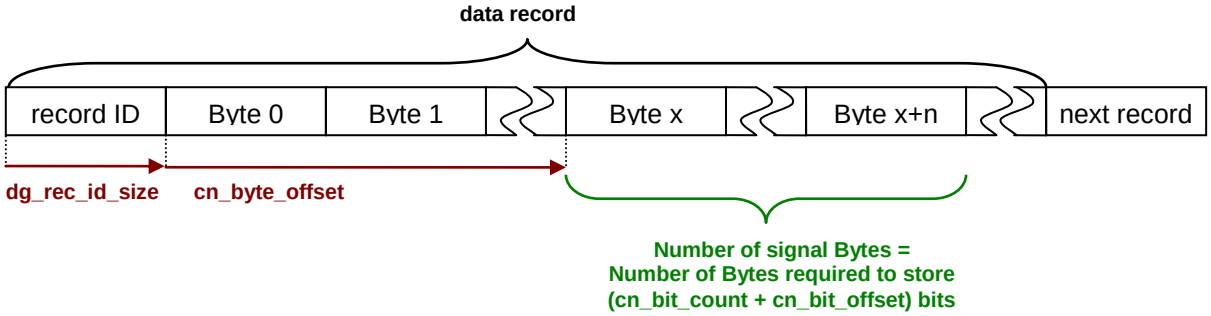


Figure 20 Plain record data

4.21.6 Example 1: Little Endian (Intel) Byte Order

We assume that we evaluate the MDF file on an Intel system. The CNBLOCK of the signal we want to de-code has the following values:

- channel type cn_type = 0 (fixed-length data channel)
- channel data type cn_data_type = 0 (unsigned integer with LE (Intel) Byte order)
- number of bits cn_bit_count = 14
- Byte offset cn_byte_offset = 2
- bit offset cn_bit_offset = 6
- signal value is valid, i.e. "all values invalid" flag (bit 0) in cn_flags not set and invalidation bit is not used or invalidation bit in record is not set (see previous section).

The number of Bytes used for the record ID is dg_rec_id_size =1, so the first Byte of a record is the record ID.

As can be seen in Figure 21, after extracting the relevant bits for the signal, we get a value of 16182 in decimal notation.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

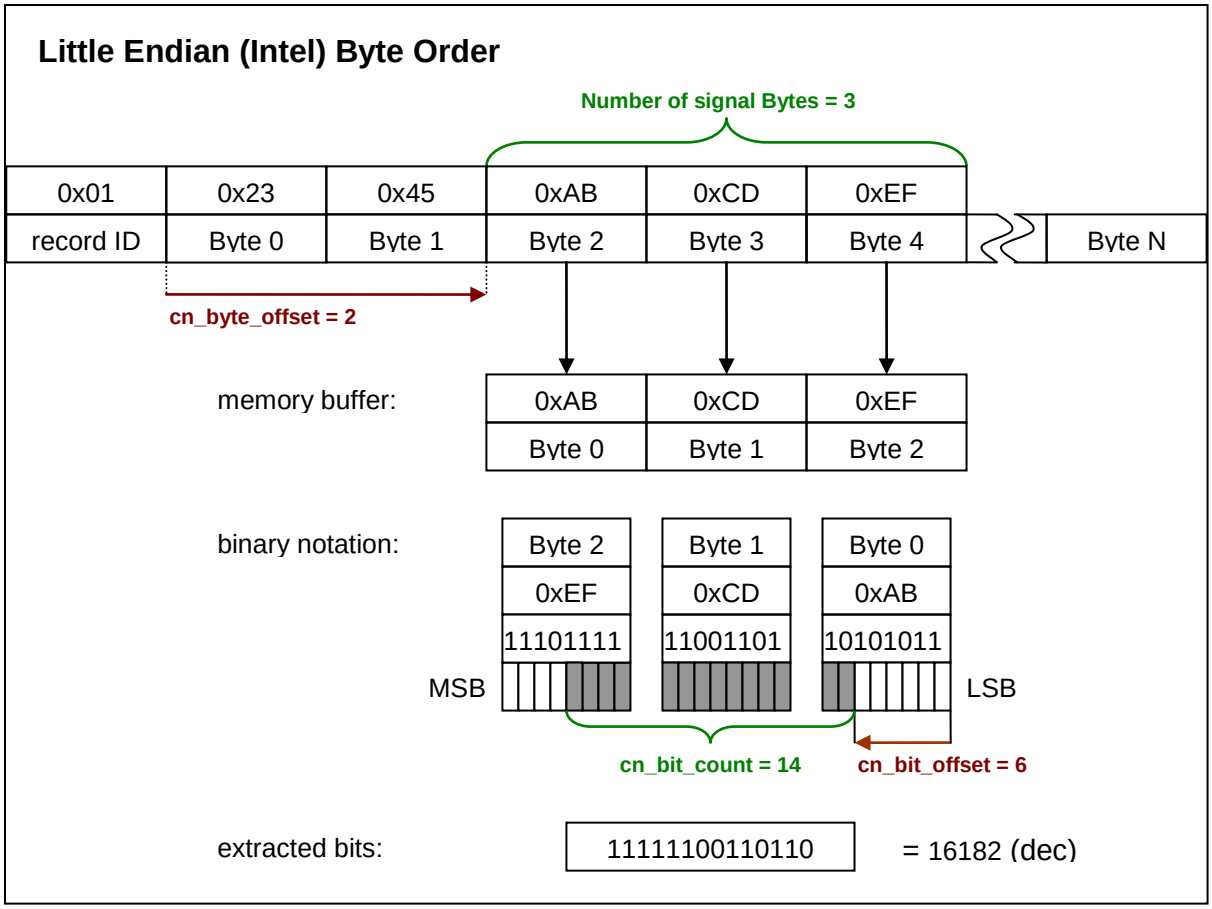


Figure 21 Example for Little Endian Byte Order

4.21.7 Example 2: Big Endian (Motorola) Byte Order

For this example we consider exactly the same settings as for Example 1, except that the channel data type now is unsigned integer with BE (Motorola) Byte order (cn_data_type = 1), i.e. the signal values probably have been acquired from an ECU with a Motorola processor.
The process to de-code the signal value is exactly the same, with the only difference that when copying the signal Bytes to the memory buffer, we have to swap the Bytes (Byte 0 with Byte N-1, Byte 1 with Byte (N-2), etc.). The application of the bit offset to the memory buffer then is identical (usually implemented by a left shift operation).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

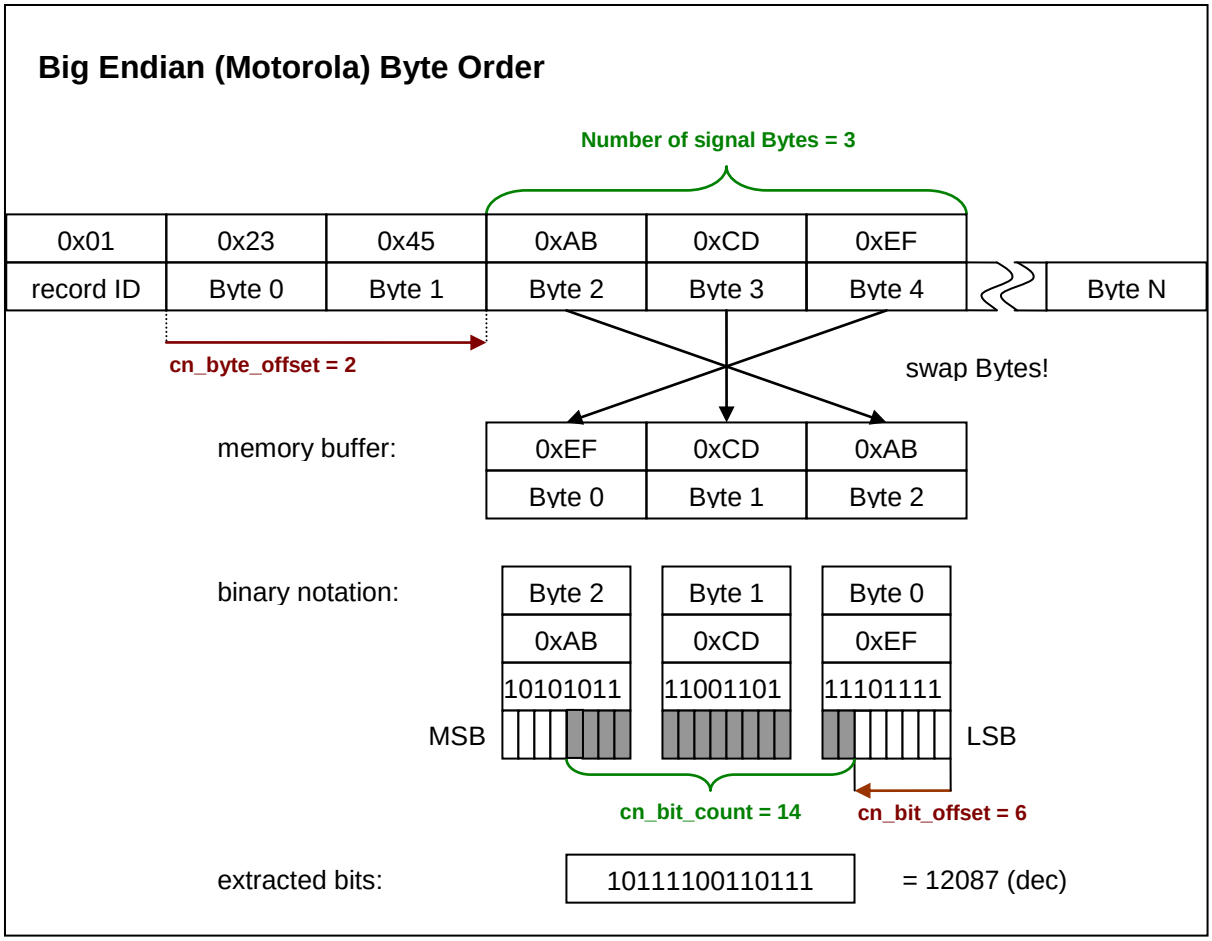


Figure 22 Example for Big Endian Byte Order

4.22 The Data Values Block DVBLOCK

The data section of the DVBLOCK contains a sequence of records without their invalidation information. The size of these records is described by cg_data_bytes. It contains records of the (single) channel group assigned to its parent DGBLOCK. Unsorted writing is not allowed with this block. It is also not allowed to include a record id.

4.22.1 Block structure of DVBLOCK

Table 65 Structure of DVBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##DV”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

dv_data	…	variable	Length of the data section must be a multiple of cg_data_bytes.

4.23 The Invalidation Data Block DIBLOCK

The data section of the DIBLOCK contains a sequence of record invalidation information without the signal data. The size of these records is described by cg_inval_bytes. It contains records of the (single) channel group assigned to its parent DGBLOCK. Unsorted writing is not allowed with this block.

4.23.1 Block structure of DIBLOCK

Table 66 Structure of DIBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##DI”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			
di_data	…	variable	Length of the data section must be a multiple of cg_inval_bytes.

4.24 The Sample Reduction Block SRBLOCK

The SRBLOCK serves to describe a sample reduction for a channel group, i.e. an alternative sampling of the signals in the channel group with a usually lower number of sampling points. There can be several sample reductions for the same channel group which are stored in a forward linked list of SRBLOCKs starting at the CGBLOCK.

Each SRBLOCK describes a sample reduction calculated with a specific interval length in one of the available synchronization domains, i.e. for one of the master channels in the channel group, or for the record index (see 4.4.6 Synchronization Domains). Roughly explained, the reduction divides the axis of the synchronization domain into intervals of the given fixed length and compresses all samples within one interval to three values: mean, minimum and maximum value. Empty intervals will be omitted.

Each sample reduction applies to all channels of its parent channel group. For details of calculating and storing a sample reduction please refer to 4.24.3 Layout of Sample.

When using sample reduction for logical groups (groups consisting of multiple CGBLOCKs and thus multiple SRBLOCKs), the configuration of the SRBLOCKs within the logical group must be the same. This means that the n-th SRBLOCK in the linked list of SRBLOCKs attached to each CGBLOCK of the logical group have identical values for sr_cycle_count, sr_interval and sr_sync_type. Single CGBLOCKs within a logical group may decide to not have any linked SRBLOCKs if no signals in the CGBLOCK can be reduced.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Note: Sample reductions are only "convenience" information to accelerate graphical display or offline analysis of the signal data. It may be omitted without any loss of information. If not present, a sample reduction can be added later by offline processing of the file.

4.24.1 Block Structure of SRBLOCK

Table 67 Structure of SRBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##SR”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
sr_sr_next	LINK	1	Pointer to next sample reduction block (SRBLOCK) (can be NIL)
sr_data	LINK	1	Pointer to reduction data block with sample reduction records (RD-/RVBLOCK or DZBLOCK of this block type) or data list block for reduction data blocks (DL-/LDBLOCK or HLBLOCK if required). RV- and LDBLOCK are optional for sorted groups, but required for column-oriented storage.
Data section			
sr_cycle_ count	UINT64	1	Number of cycles, i.e. number of sample reduction records in the reduction data block.
sr_interval	REAL	1	Length of sample interval > 0 used to calculate the sample reduction records (see explanation below). Unit depends on sr_sync_type.
sr_sync_type	UINT8	1	Sync type 1 = sr_interval contains time interval in seconds 2 = sr_interval contains angle interval in radians 3 = sr_interval contains distance interval in meter 4 = sr_interval contains index interval for record index See also section 4.4.6 Synchronization Domains.
sr_flags	UINT8	1	Flags The value contains the following bit flags (Bit 0 = LSB):

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Bit 0: invalidation Bytes flag If set, the sample reduction record contains invalidation Bytes, i.e. after the three data Byte sections for mean, minimum and maximum values, there is one invalidation Byte section. If not set, the invalidation Bytes are omitted. Must only be set if cg_inval_bytes > 0. **Bit 1:** dominant invalidation bit If set, the invalidation bit for the sample reduction record must be set when any of the underlying raw records is invalid (4.24.3 Variant 1). If not set, the invalidation bit for the sample reduction record must only be set when all of the underlying raw records is invalid (4.24.3 Variant 2). Only valid if “invalidation Bytes flag” is set *valid since MDF 4.2.0, should not be set for earlier versions*
sr_reserved	BYTE	6	Reserved

4.24.2 Motivation for Sample Reduction

When drawing a graphical diagram of the signal values for a channel which has a large number of sample points, the application usually has two options:

- Print each single sample point into the graph and connect it with the previous one. Since there are more sample points than pixels for the main axis (i.e. axis for values of the synchronization domain), several sample points will be drawn onto the same pixel position. Assuming that the main axis is horizontal, we thus will not get a point but a vertical line. The advantage here is that this line shows the minimum and maximum value that occurred for the interval represented by the pixel. However, drawing each single sample point may be quite slow, especially for a huge number of samples.
- Print only one sample point per pixel of the main axis. Thus, we really get a point for each pixel and accelerate the drawing. On the other hand, the local minimum and maximum values that occurred for all values represented by the single pixel will not be visible any longer.

The idea of the sample reduction is to provide a trade-off between the two methods. For each pixel of the main axis three values are required: the minimum, maximum and mean value for the interval that will be reduced to a pixel position on screen. Obviously, the drawing application could calculate these values each time again from the row of sample points. With the SRBLOCK, these values can be calculated only once and stored in the MDF file.

The sample reduction applies to all channels of a channel group because the sample points of the channels in a record have the same synchronization values (time stamp, index, angle or distance value). The interval represented by a pixel can vary due to different screen resolutions, or when the graph is zoomed. Thus, a number of sample reductions can be stored for different interval lengths. The drawing application then can determine the best fitting interval length for drawing the graph.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.24.3 Layout of Sample Reduction Records

For a given channel group, a linked list of sample reduction blocks (SRBLOCKs) can be added. Each SRBLOCK applies to exactly one synchronization domain and uses a different interval length for the sample reduction. In the simplest case it points to an own reduction data block (RDBLOCK) which contains the "sample reduction records" (see 4.4.5 Data storage).

Executing the sample reduction requires to divide the axis of the synchronization domain into intervals of equal length. Each interval that contains at least one sample point will be stored as sample reduction record. For each channel, the sample reduction record contains three signal values. For normal channels, these are the minimum, maximum and mean value for all samples of the channel within the given interval. For the master channel of the synchronization domain used for the sample reduction, the three values have a different meaning, see Table 68.

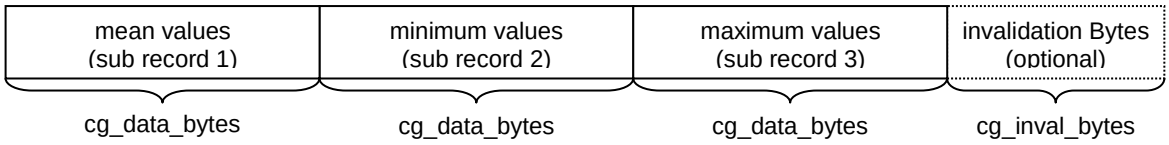


Figure 23 Layout of a sample reduction record

Each sample reduction record will be stored in the RDBLOCK referenced by the SRBLOCK. A sample reduction record contains three "sub records" and optionally an additional section with invalidation Bytes (see Figure 23). Since the RDBLOCK referenced by an SRBLOCK must not contain other sample reduction records than for this SRBLOCK, a record ID is not necessary. The length of the data part of the sample reduction record is 3 x cg_data_bytes. If "invalidation Bytes" flag (bit 0) is set, cg_inval_bytes Bytes for the invalidation Bytes follows. These contain the invalidation bits for the sample reduction record.

The layout for each of the three sub records is identical to the data Bytes in a normal data records for this channel group (in DTBLOCK). This means that a signal value is stored in a sub record for each channel using the same rules as for a normal data record. The meaning of the signal value depends on the sub record, see Table 68.

Table 68 Meaning of signal values in sub records of a sample reduction record

	Normal numeric channels (cn_data_type ≤ 5)	**Master channel matching sr_sync_type**
sub record 1	Mean value of all samples for this channel within the given interval	Start value v of interval. The interval is defined as [v, v+L[, where L is the fixed length of the intervals for this SRBLOCK. This value must be strictly monotonously increasing for the sequence of sample reduction records stored in the RDBLOCK.
sub record 2	Minimum value of all samples for this channel within the given interval	Minimum raster value Δ > 0 for the given interval, or 0 if no valid raster value available for the interval.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

		The raster value of a sample is the difference between the value of this master channel in the current sample and the value in the previous one. The raster value is undefined if this is the very first sample (record index i = 0), or the first sample after some recording or recording/acquisition interrupt event for this channel group (see 4.12 The Event Block EVBLOCK). Mathematically the minimum raster value of the interval [v, v+L[is defined as minimum of all $\Delta_i = v_i - v_{i-1}$ for all record indices i > 0 such that v_i is in [v, v+L[and that there is not any event that applies to this channel group with ev_type ≤ 2 and with synchronization value v_e for which $v_{i-1} < v_e \leq v_i$. If there is no single valid Δ_i for this interval exists, the value must be set to 0.
sub record 3	Maximum value of all samples for this channel within the given interval	Maximum raster value Δ > 0 for the given interval, or 0 if no valid raster value available for the interval (see explanation for sub record 2).

Please note that the determination of minimum, maximum and mean value considers the raw values and not the physical signal values. Invalid signal values (invalidation bit set) must not be considered for the statistics. If the signal value has been invalid for all samples within the interval, and if invalidation Bytes are used, i.e. "invalidation Bytes" flag (bit 0) is set, then the respective invalidation bit of the channel in the invalidation Bytes of the sample reduction record must be set. In this case, as best practice, either the last valid signal value should be used for the mean value, or zero if none of the previous values has been valid yet.

There are two variants of dealing with invalidation bits in sample reduction:

- Variant 1 (sr_flags: Bit 0 is set and Bit 1 is set): the invalidation bit is set when any raw sample used to calculate the data of the reduced sample is invalid.
- Variant 2 (sr_flags: Bit 0 is set and Bit 1 is not set): the invalidation bit is set only when all raw samples used to calculate the data of the reduced sample are invalid.

Variant 1 emphasizes the interpretation of the invalidation bit being an error indicator. It does not hide invalid samples due to reduction at the cost of making the time range overly large. Variant 2 does the opposite. Invalid samples are worst case only visible at raw data. Please note that Variant 2 is the required version for MDF 4.1.1 and lower.

For non-numerical channel data types (cn_data_type > 5) a sample reduction does not make sense because statistics are not defined for these values. In case both numerical and non-numerical channel data types occur within the same channel group, the minimum, maximum and mean value for the channel with the non-numerical channel data type is not defined. As good practice, simply store the non-numerical values of the first sample point within the interval for all three sub records of the sample reduction record.

When storing the data in RVBLOCKs / RIBLOCKs (e.g. with column-oriented storage), the layout for the reduced data stays the same, except that it is distributed across 2 blocks.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

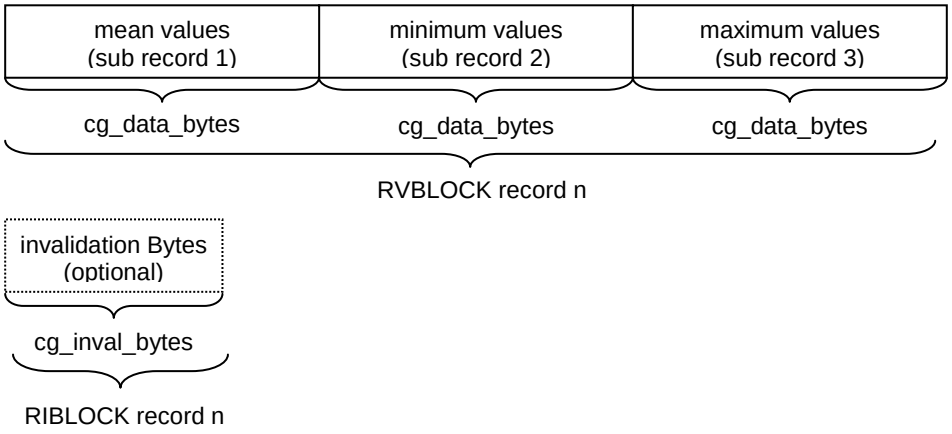


Figure 24 Layout of a sample reduction record in RV/RIBLOCKs

4.25 The Reduction Data Block RDBLOCK

The data section of the RDBLOCK contains a sequence of sample reduction records. It can only contain sample reduction records of its parent SRBLOCK. For details please refer to 4.24 The Sample Reduction Block SRBLOCK.

4.25.1 Block structure of RDBLOCK

Table 69 Structure of RDBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##RD”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			
rd_data	…	variable	….

4.26 The Reduction Values Block RVBLOCK

The data section of the RVBLOCK contains a sequence of reduction records without their invalidation information. It can only contain sample reduction records of its parent SRBLOCK. For details please refer to 4.24 The Sample Reduction Block SRBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.26.1 Block structure of RVBLOCK

Table 70 Structure of RDBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##RV”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			
rv_data	…	variable	Length of the data section must be a multiple of (3 x cg_data_bytes) of the (grand-)parent CGBLOCK.

4.27 The Reduction Data Invalidation Block RIBLOCK

The data section of the RIBLOCK contains a sequence of sample reduction invalidation byte areas. It can only contain records of its parent SRBLOCK. For details please refer to 4.24 The Sample Reduction Block SRBLOCK.

4.27.1 Block structure of RIBLOCK

Table 71 Structure of RIBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##RI”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			
ri_data	…	variable	Length of the data section must be a multiple of cg_inval_bytes of the (grand-)parent CGBLOCK.

4.28 The Signal Data Block SDBLOCK

The data section of the SDBLOCK contains a sequence of signal values of variable length.

A variable length signal data (VLSD) value starts with a 32-bit unsigned integer value (LE/Intel Byte order) which specifies the length of the value data, i.e. the number of contiguous Bytes following the 32-bit integer value. These Bytes represent the actual data of the signal value. The next VLSD value must start directly after the previous one, i.e. there is no gap.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

An SDBLOCK can only contain signal values of its parent CNBLOCK. The parent CNBLOCK must be a "variable length data channel" (cn_type = 1). The channel data type and the conversion rule (or MIME content-type string) of the parent CNBLOCK apply to the VLSD value in the SDBLOCK.

The parent CNBLOCK also specifies a value in the (fixed length) records of its DTBLOCK. Independent of the channel data type and the conversion rule, the record value always must be interpreted as an unsigned integer value (LE/Intel Byte order) with the number of bits specified in the CNBLOCK (usually 32 or 64 bits). This value specifies the position of the VLSD value, i.e. the start offset of the 32-bit length value within the data section of the SDBLOCK (counted from the beginning of the data section of the SDBLOCK).

In other words, the fixed length record for the channel group of the "variable length data channel" simply contains a reference to the VLSD signal value in the SDBLOCK of this channel. Thus, each VLSD value implicitly is synchronized with the other values in the fixed length record.

Note: A single (huge) SDBLOCK can be replaced by a DLBLOCK that references a number of (smaller) SDBLOCKs. The offset value, however, always assumes that there is one single SDBLOCK (created by concatenating the data sections of the SDBLOCKs referred to in the DLBLOCK).

Writing unsorted data groups (i.e. an unsorted MDF file), it is possible to replace the SDBLOCK by a so-called VLSD CGBLOCK in order to stream the VLSD values in records. See 4.14.4 Variable Length Signal Data (VLSD) CGBLOCK.

Note that a channel group may contain more than one "variable length data channels".

4.28.1 Block structure of SDBLOCK

Table 72 Structure of SDBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##SD”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
Data section			
sd_data	…	variable	….

4.28.2 Example for Usage of SDBLOCK

Figure 25 shows an example. There is a (sorted) data group which contains a channel group with two channels. The first channel is the time master channel and the second channel is a "variable length data channel" (cn_type = 1). Note that no record ID is used.

The second channel references an SDBLOCK with the VLSD values. The fixed length data record stored in the DTBLOCK for this channel group contains the offset value which refers to the start of the VLSD value in the data section of the SDBLOCK. At this position, the

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

length of the plain data is given as 32-bit unsigned Integer. The following range of Bytes (of the specified length) then is the actual signal value. How to interpret this range of Bytes depends on the data type and the conversion rule or MIME type of the CNBLOCK. A common use case would be to store strings of variable length in this way, e.g. if the maximum length of the string is not known beforehand or in order to avoid a waste of memory space by reserving the maximum length for a (fixed length) record value.

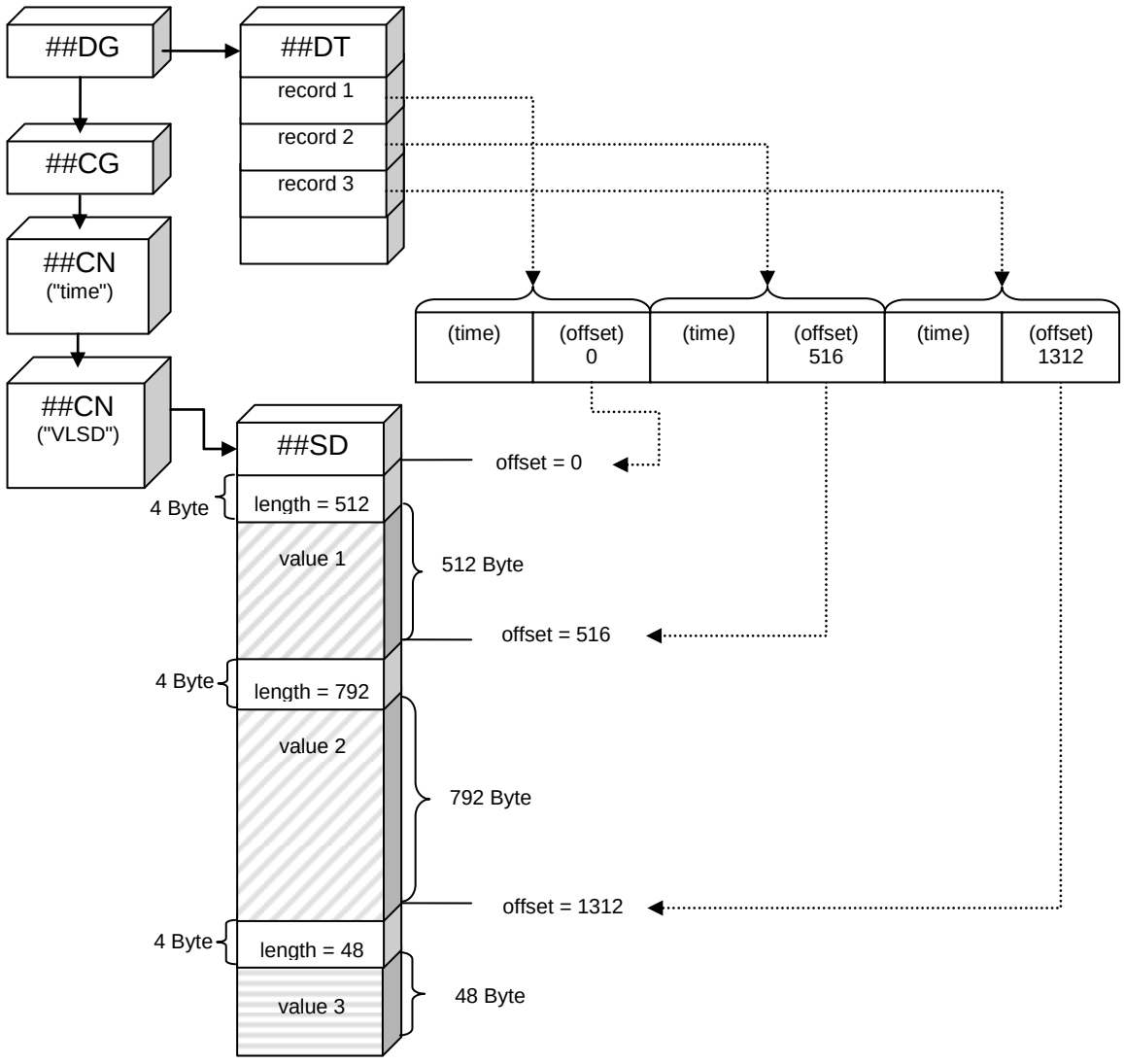


Figure 25 Example for usage of SDBLOCK

4.29 The Data List Block DLBLOCK

The DLBLOCK references a list of data blocks (DTBLOCK) or a list of signal data blocks (SDBLOCK) or a list of reduction data blocks (RDBLOCK). This list of blocks is equivalent to using a single (signal/reduction) data block and can be used to avoid a huge data block by splitting it into smaller parts.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Each single block referenced in this list also can be replaced by a zipped data block (DZBLOCK) for the respective block type. There is no change to the DLBLOCK when using a DZBLOCK instead of the original block type with uncompressed data section: A DZBLOCK always must be treated as if there was its uncompressed equivalent (see section 4.31 The Data Zipped Block DZBLOCK). Hence, whenever we refer to the data section of a referenced block, in case of a DZBLOCK this is the uncompressed data section of the original block type.

In order to optimize the read access of a specific record / signal value in the list of DTBLOCKs/SDBLOCKs/RDBLOCKs, a list of "start offsets" for the data sections of the referenced blocks is maintained. If the (uncompressed) data sections of all referenced blocks would be concatenated, the start offset of a block defines the position of its data section in this concatenation.

For the special case that all (uncompressed) data sections of the referenced blocks have the same length, a calculation can be used instead of a search in the list of start offsets. To indicate this possibility, an extra element specifies the equal data section length.

Starting with MDF 4.2.0, the DLBLOCK optionally can store the master channel values of the first record in each referenced data block. This can be used to improve performance when seeking for a certain master channel value, i.e. to immediately find the relevant data block to narrow down a binary search to records in this block. Thus, unnecessary reading and processing time can be avoided, especially in case of compressed data blocks.

4.29.1 Block structure of DLBLOCK

Table 73 Structure of DLBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##DL”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
dl_dl_next	LINK	1	Pointer to next data list block (DLBLOCK) (can be NIL).
dl_data	LINK	N	Pointers to the data blocks (DTBLOCK, SDBLOCK or RDBLOCK or a DZBLOCK of the respective block type). None of the links in the list can be NIL. It is not allowed to mix the data block types: all links must uniformly reference the same data block type (DTBLOCK/SDBLOCK/RDBLOCK or an equivalent DZBLOCK). Also all DLBLOCKs in the linked list of DLBLOCKs must reference the same data block type Note: a mixture between DZBLOCKs and uncompressed data blocks is allowed. However, if a DZBLOCK is in the list, the complete linked list of DLBLOCKs must be preceded by a HLBLOCK. See also explanation of HLBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
Data section			
dl_flags	UINT8	1	Flags Bit combination of flags as defined below in Table 74. Note: The value of dl_flags must be equal for all DLBLOCKs in the linked list.
dl_reserved	BYTE	3	Reserved
dl_count	UINT32	1	Number of referenced blocks N. If the "equal length" flag (bit 0 in dl_flags) is set, then dl_count must be equal for each DLBLOCK in the linked list except for the last one. For the last DLBLOCK (i.e. dl_dl_next = NIL) in this case the value of dl_count can be less than or equal to dl_count of the previous DLBLOCK.
dl_equal_ length	UINT64	1	**Only present if "equal length" flag (bit 0 in dl_flags) is set.** Equal data section length. Every block in dl_data list has a data section with a length equal to dl_equal_length. This must be true for each DLBLOCK within the linked list, and has only one exception: the very last block (dl_data[dl_count-1] of last DLBLOCK in linked list) may have a data section with a different length which must be less than or equal to dl_equal_length.
dl_offset	UINT64	N	**Only present if "equal length" flag (bit 0 in dl_flags) is not set.** Start offset (in Bytes) for the data section of each referenced block. If the (uncompressed) data sections of all blocks referenced by dl_data list (for all DLBLOCKs in the linked list) are concatenated, the start offset for a referenced block gives the position of the data section of this block within the concatenated section. The start offset dl_offset[i] thus is equal to the sum of the data section lengths of all referenced blocks in dl_data list for every previous DLBLOCK in the linked list, and of all referenced blocks in dl_data list up to (i-1) for the current DLBLOCK. As a consequence, the start offset dl_offset[0] for the very first DLBLOCK must always be zero.
dl_time_ values	INT64 or REAL	N	**Only present if "time values" flag (bit 1 in dl_flags) is set.** The list stores the (raw) time values of the first record in each referenced data block. In case of RDBLOCKs, the interval start time is used, i.e. the (raw) time value in sub record 1 (see Table 68). Note that there is no use case for dl_time_values in case the DLBLOCK references signal data blocks (SDBLOCK or respective DZBLOCK).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			"First" record means the first record which starts in the data block, i.e. whose first byte is contained. Although not recommended, a record may be distributed to two or more data blocks. In such a case, if no record start byte is contained in the data block, the time value of the current (single, partly contained) record in this data block is used. dl_time_values[i] maps to the data block referenced by dl_data[i]. It is the raw value of the (virtual or non-virtual) time master channel (channel with cn_type = 2 or 3 and cn_sync_type = 1) in the channel group that defines the layout of the records stored in the data blocks (sorted data group required). For a virtual time master channel, the raw value is equivalent to the zero-based record index. The raw values are stored either as INT64 or REAL, depending if the data type of the time channel is an Integer or a Floating-point type (see cn_data_type). This is independent of the actually used number of bits (cn_bit_count) in the time master channel CNBLOCK. In order to get the physical time value, the conversion rule (if present) of the time master channel must be applied.
dl_angle_ values	INT64 or REAL	N	**Only present if "angle values" flag (bit 2 in dl_flags) is set.** Like dl_time_values, but here the raw values of the (virtual or non-virtual) angle master channel (channel with cn_type = 2 or 3 and cn_sync_type = 2) are stored. For details please refer to dl_time_values.
dl_distance_ values	INT64 or REAL	N	**Only present if "distance values" flag (bit 3 in dl_flags) is set.** Like dl_time_values, but here the raw values of the (virtual or non-virtual) distance master channel (channel with cn_type = 2 or 3 and cn_sync_type = 3) are stored. For details please refer to dl_time_values.

A linked list of DLBLOCKs can always be replaced by a single DLBLOCK. Furthermore, a single DLBLOCK can always be replaced by a single block of the referenced block type, i.e. a single DTBLOCK, SDBLOCK or RDBLOCK. The data section of this single block can be constructed by simply concatenating the data sections of all blocks referenced by the DLBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 74 Bit flags for dl_flags (Bit 0 = LSB)

Bit	Description
Bit 0	Equal length flag. If set, each DLBLOCK in the linked list has the same number of referenced blocks (dl_count) and the (uncompressed) data sections of the blocks referenced by dl_data have a common length given by dl_equal_length. The only exception is that for the last DLBLOCK in the list (dl_dl_next = NIL), its number of referenced blocks dl_count can be less than or equal to dl_count of the previous DLBLOCK, and the data section length of its last referenced block (dl_data[dl_count-1]) can be less than or equal to dl_equal_length. If not set, the number of referenced blocks dl_count may be different for each DLBLOCK in the linked list, and the data section lengths of the referenced blocks may be different and a table of offsets is given in dl_offset. Note: The value of the "equal length" flag must be equal for all DLBLOCKs in the linked list.
Bit 1	Time values flag. If set, the DLBLOCK contains a list of (raw) time values in dl_time_values which can be used to improve performance for binary search of time values in the list of referenced data blocks. The bit must not be set if the DLBLOCK references signal data blocks or if the DLBLOCK contains records from multiple channel groups (i.e. if parent is an unsorted data group). The bit can only be set if the channel group which defines the record layout contains either a virtual or a non-virtual time master channel (cn_type = 2 or 3 and cn_sync_type = 1). Note that for RDBLOCKs, it only makes sense to store the time values if the parent SRBLOCK has sr_sync_type = 1 (time). Note: The value of the "time values" flag must be equal for all DLBLOCKs in the linked list. *Valid since MDF 4.2.0, should not be set for earlier versions*
Bit 2	Angle values flag. If set, the DLBLOCK contains a list of (raw) angle values in dl_angle_values which can be used to improve performance for binary search of angle values in the list of referenced data blocks. The bit must not be set if the DLBLOCK references signal data blocks or if the DLBLOCK contains records from multiple channel groups (i.e. if parent is an unsorted data group). The bit can only be set if the channel group which defines the record layout contains either a virtual or a non-virtual angle master channel (cn_type = 2 or 3 and cn_sync_type = 2). Note that for RDBLOCKs, it only makes sense to store the angle values if the parent SRBLOCK has sr_sync_type = 2 (angle).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Bit	Description
	Note: The value of the "angle values" flag must be equal for all DLBLOCKs in the linked list. *Valid since MDF 4.2.0, should not be set for earlier versions*
Bit 3	Distance values flag. If set, the DLBLOCK contains a list of (raw) distance values in dl_distance_values which can be used to improve performance for binary search of distance values in the list of referenced data blocks. The bit must not be set if the DLBLOCK references signal data blocks or if the DLBLOCK contains records from multiple channel groups (i.e. if parent is an unsorted data group). The bit can only be set if the channel group which defines the record layout contains either a virtual or a non-virtual distance master channel (cn_type = 2 or 3 and cn_sync_type = 3). Note that for RDBLOCKs, it only makes sense to store the distance values if the parent SRBLOCK has sr_sync_type = 3 (distance). Note: The value of the "distance values" flag must be equal for all DLBLOCKs in the linked list. Valid since MDF 4.2.0, should not be set for earlier versions

Example

For the following example there would be no difference when using SDBLOCKs or RDBLOCKs instead of DTBLOCKs. It shows how to replace a linked list of DLBLOCK by a single DTBLOCK.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

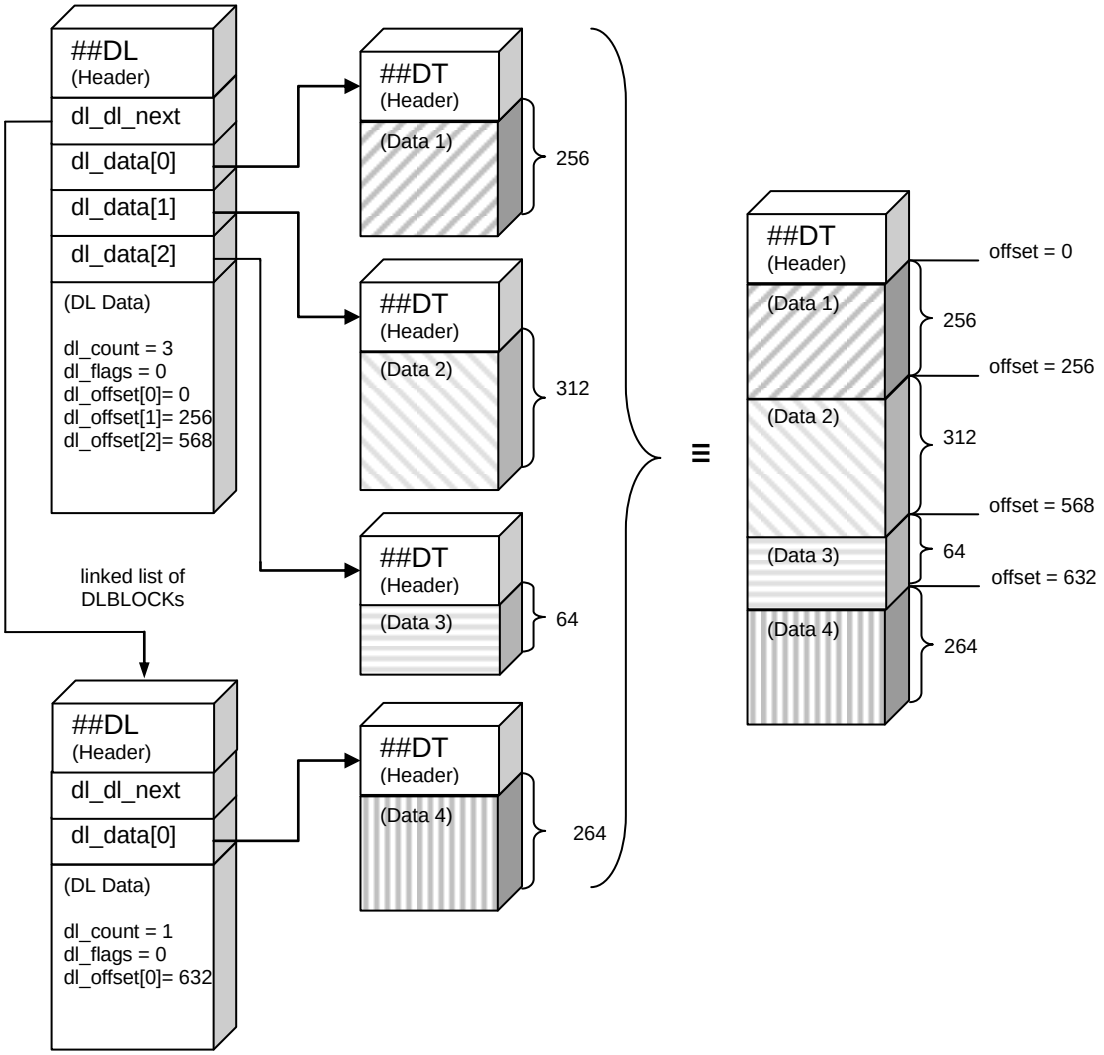


Figure 26 Example for usage of DLBLOCK

Note: When splitting a data block (DT-/SD-/RDBLOCK) by using a data list (DLBLOCK), its data section is fragmented and distributed over several smaller data blocks. For reading a record, please consider that it may not be contained completely in the current fragment of the data section. Instead, the record also may be fragmented and distributed over subsequent data sections. For instance, in the above example consider a record with a length of 120 Bytes and which starts at offset position 560. In this case, the first 8 Bytes are contained in the second part (Data 2), the next 64 Bytes in the third part (Data 3) and the rest at the beginning of the forth part (Data 4) of the fragmented data section.

4.30 The List Data Block LDBLOCK

The LDBLOCK is an alternative to the DLBLOCK that allows the separation of data and invalid bit information. It cannot be used for unsorted storage. The block itself stores a list of links to blocks containing the signal data (DV-/RVBLOCK) followed by an optional list of links to blocks containing invalidation data (DI-/RIBLOCK). The value data block (DV- or RVBLOCK) and the invalid data block (DI- or RIBLOCK) with the same index form a pair. Each pair must contain the same number of samples.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Single DI-/RIBLOCKs should be omitted (i.e. respective link is set to NIL) in case no invalid bit is set in that block. The omitted block is treated as being there with all records being valid. The complete list of invalidation data blocks must be omitted in case of no invalidation data being used (cg_inval_bytes = 0). These optimizations allow the reading client to efficiently scan signal data for invalid samples without the need for reading the complete invalidation bits of the signals.

There are two benefits for using the LDBLOCK. The separation of the invalid data from the signal values allows

1. faster checking whether a signal contains values where the invalid bit is set (please note that this requires that the writer sets NIL links in case no invalid bit was set in the respective invalidation data block)
2. faster reading of data in some programming languages due to its simpler layout.

The LDBLOCK is the required approach for column-oriented storage.

4.30.1 Block structure of LDBLOCK

Table 75 Structure of LDBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##LD”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links.
Link section			
ld_ld_next	LINK	1	Pointer to next list data block (LDBLOCK) (can be NIL).
ld_data	LINK	N	Links to value blocks (DVBLOCK / RVBLOCK) that contain the actual data of the group. None of the links in the list can be NIL. It is not allowed to mix the data block types: all links must uniformly reference the same data block type (DV- or RVBLOCK or an equivalent DZBLOCK). Also, all LDBLOCKs in the linked list of LDBLOCKs must reference the same data block type Note: a mixture between DZBLOCKs and uncompressed data blocks is allowed. Please note that in contrast to the DLBLOCK, no HLBLOCK is required in this case.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
ld_inval_data	LINK	N	**Only present if "invalid data present" flag (bit 31 in ld_flags) is set.** Links to invalidation data blocks (DIBLOCK / RIBLOCK) that contain the actual invalidation data of the group. These links can be NIL if the respective DI- or RIBLOCK does not contain any invalidation bit set for the channels contained in the parent channel group. A flag indicates the presence or the absence of this whole section of links (e.g. if no invalidation bytes are used, cg_inval_bytes = 0). It is not allowed to mix the data block types: all links must uniformly reference the same data block type (DI- or RIBLOCK or an equivalent DZBLOCK). Also, all LDBLOCKs in the linked list of LDBLOCKs must reference the same data block type. Note: a mixture between DZBLOCKs and uncompressed data blocks is allowed. Please note that in contrast to the DLBLOCK, no HLBLOCK is required in this case.
Data section			
ld_flags	UINT32	1	Flags Bit combination of flags as defined below in Table 76. Note: The value of ld_flags must be equal for all LDBLOCKs in the linked list except for the "invalid data present" flag (bit 31).
ld_count	UINT32	1	Number of referenced blocks N. If the "equal sample count" flag (bit 0 in ld_flags) is set, then ld_count must be equal for each LDBLOCK in the linked list except for the last one. For the last LDBLOCK (i.e. ld_ld_next = NIL) in this case the value of ld_count can be less than or equal to ld_count of the previous LDBLOCK.
ld_equal_ sample_count	UINT64	1	**Only present if "equal sample count" flag (bit 0 in ld_flags) is set.** Equal data section length. Every block in ld_data list and, if present, in ld_inval_data list contains the same number of samples given by ld_equal_sample_count. This must be true for each LDBLOCK within the linked list, and has only one exception: the very last block (ld_data[ld_count-1] / ld_inval_data[ld_count-1] of last LDBLOCK in linked list) may contain a different number of samples which must be less than or equal to ld_equal_sample_count.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
ld_sample_ offset	UINT64	N	**Only present if "equal sample count" flag (bit 0 in ld_flags) is not set.** Start offset (in samples) for the data section of each referenced block. This means that the first record in the data section of the referenced block is the record with the index equal to ld_sample_offset.
ld_time_ values	INT64 or REAL	N	**Only present if "time values" flag (bit 1 in ld_flags) is set.** The list stores the (raw) time values of the first record in each referenced data block. In case of RVBLOCKs, the interval start time is used, i.e. the (raw) time value in sub record 1 (see Table 68). "First" record means the first record which starts in the data block. ld_time_values[i] maps to the data block referenced by ld_data[i]. It is the raw value of the (virtual or non-virtual) time master channel (channel with cn_type = 2 or 3 and cn_sync_type = 1) in the channel group that defines the layout of the records stored in the data blocks (sorted data group required). For a virtual time master channel, the raw value is equivalent to the zero-based record index. The raw values are stored either as INT64 or REAL, depending if the data type of the time channel is an Integer or a Floating-point type (see cn_data_type). This is independent of the actually used number of bits (cn_bit_count) in the time master channel CNBLOCK. In order to get the physical time value, the conversion rule (if present) of the time master channel must be applied.
ld_angle_ values	INT64 or REAL	N	**Only present if "angle values" flag (bit 2 in ld_flags) is set.** Like ld_time_values, but here the raw values of the (virtual or non-virtual) angle master channel (channel with cn_type = 2 or 3 and cn_sync_type = 2) are stored. For details please refer to ld_time_values.
ld_distance_ values	INT64 or REAL	N	**Only present if "distance values" flag (bit 3 in ld_flags) is set.** Like ld_time_values, but here the raw values of the (virtual or non-virtual) distance master channel (channel with cn_type = 2 or 3 and cn_sync_type = 3) are stored. For details please refer to ld_time_values.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

A linked list of LDBLOCKs can always be replaced by a single LDBLOCK. In contrast to the DLBLOCK, a single LDBLOCK can only be replaced by a single block of the referenced data block (DV-/RVBLOCK) in case no invalidation data is used, i.e. if "invalid data present" flag (bit 31 in ld_flags) is not set.

Table 76 Bit flags for ld_flags (Bit 0 = LSB)

Bit	Description
Bit 0	Equal sample count flag. If set, each LDBLOCK in the linked list has the same number of referenced blocks (ld_count) and the (uncompressed) data sections of the blocks referenced by ld_data / ld_inval_data contain the same number of samples given by ld_equal_sample_count. The only exception is that for the last LDBLOCK in the list (ld_ld_next = NIL), its number of referenced blocks ld_count can be less than or equal to ld_count of the previous LDBLOCK, and the number of samples in its last referenced block (ld_data[ld_count-1] / ld_inval_data[ld_count-1]) can be less than or equal to ld_equal_sample_count. If not set, the number of referenced blocks ld_count may be different for each LDBLOCK in the linked list, and the number of samples in the referenced blocks may be different and a table of start samples indices is given in ld_sample_offset. Note: The value of the "equal sample count" flag must be equal for all LDBLOCKs in the linked list.
Bit 1	Time values flag. If set, the LDBLOCK contains a list of (raw) time values in ld_time_values which can be used to improve performance for binary search of time values in the list of referenced data blocks. The bit can only be set if the channel group which defines the record layout contains either a virtual or a non-virtual time master channel (cn_type = 2 or 3 and cn_sync_type = 1). Note that for RVBLOCKs, it only makes sense to store the time values if the parent SRBLOCK has sr_sync_type = 1 (time). Note: The value of the "time values" flag must be equal for all LDBLOCKs in the linked list.
Bit 2	Angle values flag. If set, the LDBLOCK contains a list of (raw) angle values in ld_angle_values which can be used to improve performance for binary search of angle values in the list of referenced data blocks. The bit can only be set if the channel group which defines the record layout contains either a virtual or a non-virtual angle master channel (cn_type = 2 or 3 and cn_sync_type = 2). Note that for RVBLOCKs, it only makes sense to store the angle values if the parent SRBLOCK has sr_sync_type = 2 (angle).

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Bit	Description
	Note: The value of the "angle values" flag must be equal for all LDBLOCKs in the linked list.
Bit 3	Distance values flag. If set, the LDBLOCK contains a list of (raw) distance values in ld_distance_values which can be used to improve performance for binary search of distance values in the list of referenced data blocks. The bit can only be set if the channel group which defines the record layout contains either a virtual or a non-virtual distance master channel (cn_type = 2 or 3 and cn_sync_type = 3). Note that for RVBLOCKs, it only makes sense to store the distance values if the parent SRBLOCK has sr_sync_type = 3 (distance). Note: The value of the "distance values" flag must be equal for all LDBLOCKs in the linked list.
Bit 31	Invalid data present flag. This flag indicates that the block links invalidation data blocks (DIBLOCK / RIBLOCK).

4.31 The Data Zipped Block DZBLOCK

The DZBLOCK represents a zipped (compressed) replacement for a (signal/reduction) data block (DTBLOCK, SDBLOCK or RDBLOCK or their alternate representations when using LDBLOCKs – DVBLOCK/DIBLOCK or RVBLOCK/RIBLOCK). It can be used instead of one of these block types: whenever there is a DZBLOCK, a reading application internally must replace it with the original block type containing the unzipped (uncompressed) data.

The DZBLOCK stores the block type of the replaced data block and contains its data section compressed using the denoted zip algorithm[6]. It can be used to reduce the size of the MDF file during writing by compressing the signal data "on the fly". This is of advantage for logger applications with reduced disk space. It also avoids compressing the complete file which would make it impossible to read the meta information of the signals without unpacking the complete file again.

Replacing a data block with a DZBLOCK, the logic of the MDF block structure stays unchanged with one single exception: if the DZBLOCK is part of a data list (DLBLOCK or linked list of DLBLOCKs), a HLBLOCK must precede the first DLBLOCK for this data list (see 4.32 The Header List Block HLBLOCK). This is not required if you are using the LDBLOCK for storing the data. Also, all DZBLOCKs within a data list must use the same zip algorithm.

For a sorted data group, the uncompressed data of a DZBLOCK should not exceed a certain size in order to allow unpacking of the data in virtual memory for a fast index-based read access. We suggest that the uncompressed length should not be larger than four MByte. As a result, huge data blocks should be split into smaller ones using a data list (DLBLOCKs preceded by HLBLOCK). For an unsorted data group, one single DZBLOCK is acceptable

[6] Currently only the Deflate zip algorithm (see [12] and [24]) is supported.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

because the records only can be accessed sequentially which usually also is possible quite efficiently for compressed data using a read stream.

Please note that in case of a very small data section the overhead for the compression and the DZBLOCK may result in consuming more space than without compression. A tool may determine whether the DZBLOCK with the compressed data is smaller or larger than the original DTBLOCK, and simply write the smaller variant.

The DZBLOCK type has been introduced in MDF version 4.1.0 and must not be used in previous MDF versions. Applications implemented for an earlier MDF 4.0.x version will not be able to recognize this block type. Typically, they should ignore this block and refuse reading the signal data for this channel (group). However, they still can access all meta information of the signals as for MDF 4.0.0 (forward compatibility).

4.31.1 Block structure of DZBLOCK

Table 77 Structure of DZBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##DZ”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
Data section			
dz_org_block_type	CHAR	2	Block type identifier of the original (replaced) data block without the "##" prefix, i.e. either "DT", "SD", "RD" or "DV", "DI", "RV", "RI".
dz_zip_type	UINT8	1	Zip algorithm used to compress the data stored in dz_data
			0 = Deflate The Deflate zip algorithm as used in various zip implementations (see [12] and [24])
			1 = Transposition + Deflate Before compression, the data block is transposed as explained in 4.31.2 Transposition of Data. Typically, only used for sorted data groups and DT-/DV-/DIBLOCK or RD-/RV-/RIBLOCK types.
dz_reserved	BYTE	1	Reserved
dz_zip_parameter	UINT32	1	Parameter for zip algorithm. Content and meaning depends on dz_zip_type: For dz_zip_type = 1, the value must be > 1 and specifies the number of Bytes used as columns, i.e. usually the length of the record for a sorted data group.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

			Otherwise the value must be 0.
dz_org_data_ length	UINT64	1	Length of uncompressed data in Bytes, i.e. length of data section for original data block. For a sorted data group, this should not exceed 2^{22} Byte (4 MByte).
dz_data_length	UINT64	1	Length N of compressed data in Bytes, i.e. the number of Bytes stored in dz_data. If dz_org_data_length < dz_data_length + 24 the overhead of the compression is higher than the memory saved. In this case, we recommend writing the original data block instead.
dz_data	BYTE	N	Contains compressed binary data for data section of original data block.

4.31.2 Transposition of Data

The transposition of a 2-dimensional matrix A is an operation that calculates the "transpose" of the matrix, denoted as A^T. If A is a M x N matrix, its transpose A^T is a N x M matrix where $A^T(n,m) = A(m,n)$ for each element $1 \leq m \leq M$ and $1 \leq n \leq N$. Technically this means that the rows of matrix A are written as columns of matrix A^T. The transposition easily can be reverted simply by applying the same operation again, i.e. $A = (A^T)^T$.

In case of dz_zip_type = 1, the data to be compressed, i.e. the data section of the original data block, is transposed before performing the compression. In order to restore the original data when reading, a transposition must be applied again after de-compression.

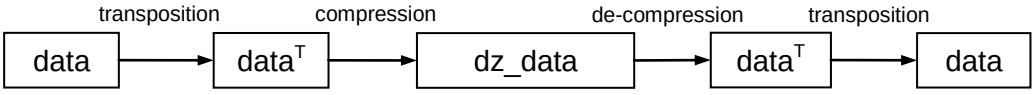


Figure 27 Operations for compression and de-compression for dz_zip_type = 1

The value of dz_zip_parameter gives the number of Bytes to use as columns N when viewing the original data Bytes as 2-dimensional matrix. The number of rows M can be calculated by dividing the total number of Bytes given in dz_org_data_length by N. In case there is a remainder for the division, the incomplete row simply is ignored for the transposition and left unchanged at the end of the transposed data (see example).

Typically, the transposition is only used for sorted data groups where the data to be compressed contains records of a fixed length, and the value of dz_zip_parameter is set to the record length. In case certain Bytes only rarely change between records, e.g. the more significant Bytes of the time stamp, a transposition results in long rows of Bytes with equal value which allows a high compression rate for the Deflate zip algorithm.

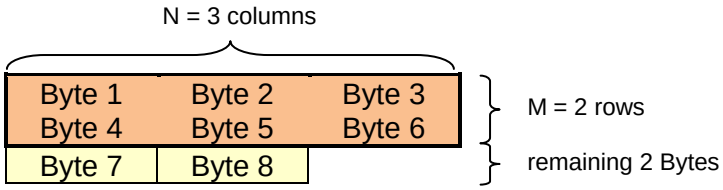
Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Example

Assume a data section with dz_org_data_length = 8 Bytes:

Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8

With dz_zip_parameter = 3, this is seen as matrix with N = 3 columns and M = 8 div 3 = 2 rows. There is an incomplete row with 8 mod 3 = 2 Bytes.



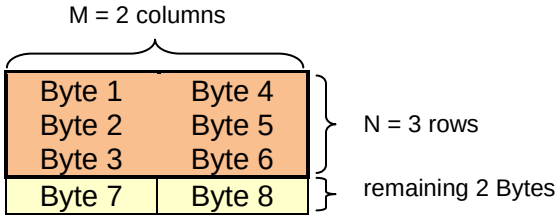
After the transposition, the data looks as follows

Byte 1	Byte 4
Byte 2	Byte 5
Byte 3	Byte 6
Byte 7	Byte 8

The stream of Bytes to be compressed then looks like this:

Byte 1	Byte 4	Byte 2	Byte 5	Byte 3	Byte 6	Byte 7	Byte 8

For reading, the compressed data first must be de-compressed. The resulting stream of Bytes should be equal to the one above. To restore the original data, the stream now must be seen as the transpose matrix with N x M = 3 x 2 with the remaining 2 Bytes attached.



The transposition of the 3 x 2 matrix results in the original 2 x 3 matrix, again with the remaining Bytes attached.

Byte 1	Byte 2	Byte 3
Byte 4	Byte 5	Byte 6
Byte 7	Byte 8	

Flattening out the table, we obtain the original stream of Bytes.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

4.32 The Header List Block HLBLOCK

The HLBLOCK represents the "header" of a list of data blocks, i.e. the start of a linked list of DLBLOCKs. It contains information about the DLBLOCKs and the contained data blocks.

The HLBLOCK indicates that the list of data blocks referenced by its descendant DLBLOCKs contains zipped data blocks (DZBLOCKs). It also states the zip algorithm used by the DZBLOCKs in the list. Thus, applications not supporting compressed data blocks or the specified zip algorithm can immediately reject reading the signal data without further investigation of the referenced blocks.

The HLBLOCK type has been introduced in MDF version 4.1.0 and must not be used in previous MDF versions. Applications implemented for an earlier MDF 4.0.x version will not be able to recognize this block type. Typically, they should ignore this block and refuse reading the signal data for this channel (group). However, they still can access all meta information of the signals as for MDF 4.0.0 (forward compatibility).

In order to avoid that older applications unnecessarily reject reading the data block list, a HLBLOCK must only be used if there really is at least one DZBLOCK in the list of data blocks. If there is none at all, the HLBLOCK should be omitted and the parent block can immediately reference the DLBLOCK. Note that it is quite easy to insert (or remove) a HLBLOCK, e.g. when writing the first DZBLOCK.

For future MDF versions, the HLBLOCK also may be used to indicate new data encoding types or changes to the data (list) blocks. Therefore, as an exception to Rules to Ensure MDF Compatibility between Versions, an application reading a future MDF4.x version should not process the HLBLOCK and its descendants in the following cases:

- unknown flags are set in hl_flags
- hl_zip_type has some unknown value (unknown/unsupported zip algorithm)

4.32.1 Block structure of HLBLOCK

Table 78 Structure of HLBLOCK

Name	Type	Count	Description
Header section			
id	CHAR	4	Block type identifier, always “##HL”
reserved	BYTE	4	Reserved used for 8-Byte alignment
length	UINT64	1	Length of block
link_count	UINT64	1	Number of links
Link section			
hl_dl_first	LINK	1	Pointer to the first data list block (DLBLOCK)
Data section			
hl_flags	UINT16	1	Flags The value contains the following bit flags (Bit 0 = LSB):

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Name	Type	Count	Description
			Bit 0: Equal length flag For the referenced DLBLOCK (and thus for each DLBLOCK in the linked list), the value of the "equal length" flag (bit 0 in dl_flags) must be equal to this flag.
			Bit 1: Time values flag For the referenced DLBLOCK (and thus for each DLBLOCK in the linked list), the value of the "time values" flag (bit 1 in dl_flags) must be equal to this flag.
			Bit 2: Angle values flag For the referenced DLBLOCK (and thus for each DLBLOCK in the linked list), the value of the "angle values" flag (bit 2 in dl_flags) must be equal to this flag.
			Bit 3: Distance values flag For the referenced DLBLOCK (and thus for each DLBLOCK in the linked list), the value of the "distance values" flag (bit 3 in dl_flags) must be equal to this flag.
hl_zip_type	UINT8	1	Zip algorithm used by DZBLOCKs referenced in the list, i.e. in an DLBLOCK of the link list starting at hl_dl_first. Note: all DZBLOCKs in the list must use the same zip algorithm. For possible values, please refer to dz_zip_type member of DZBLOCK.
hl_reserved	BYTE	5	Reserved

The next figure demonstrates the usage of a HLBLOCK by showing an example for a list of data blocks with and without compression, here for DTBLOCKs referenced by dg_data.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

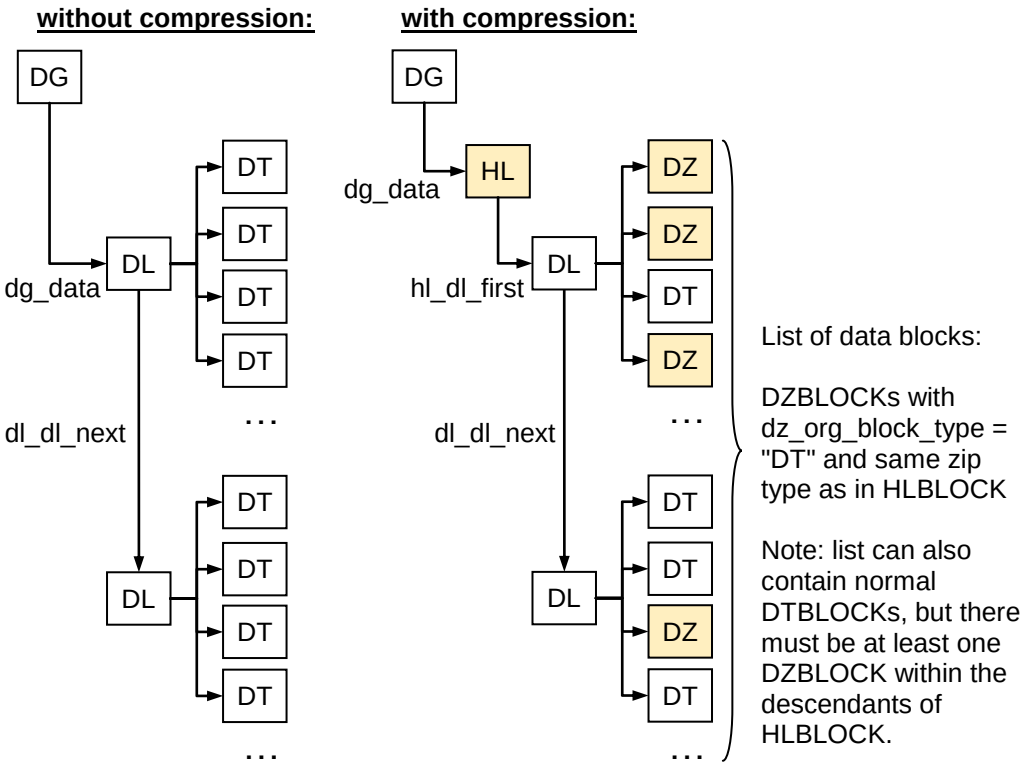


Figure 28 List of data blocks with and without compression

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

5 MDF Meta Data Format

5.1 Content of the MDBLOCK

The content of an MDBLOCK is a string, but unlike a TXBLOCK the string is interpreted as XML. The string content must conform to the following rules:

- The string must be well-formed XML according to version 1.0 (see [22]). In short this means:
  - each tag has a matching closing tag or is an empty tag
  - all tags are nested correctly
  - there is exactly one root tag
  - the XML prologue (e.g. "<?xml version="1.0" encoding="UTF-8" ?>") can be omitted.
- The string must validate against the schema determined by the referencing link field.
  - E.g. an MDBLOCK referenced by the cn_md_comment field in the CNBLOCK must validate against cn_comment.xsd.
- Additionally, the following rules apply
  - The string must always be UTF-8 encoded (see [21]).
  - The default namespace for the root tag is assumed to be http://www.asam.net/mdf/v4. Thus usually no namespace needs to be specified.

A minimal MDBLOCK referenced from the CNBLOCK field cn_md_comment would look like this:

```
<CNcomment>
  <TX/>
</CNcomment>
```

5.1.1 XML Versioning

For each version of MDF there will be a set of schemas that define the allowed XML content (see [22] and [23]).

All schemas released within the same MDF major version will use the same namespace (i.e. http://www.asam.net/mdf/v4 for the current version). To find the correct set of schemas, the MDF version in the HDBLOCK shall be used.

Changes to the schemas within the same major version of MDF will be done in a backwards compatible way by never removing tags and adding new tags as optional.

An older tool can read the XML of a newer version by ignoring the tags that it does not know about.

Note: However, it is not possible to use an older schema to validate a newer version XML directly. This is an inherent problem of the version 1.0 of the XML Schema standard and will be resolved by version 1.1 of that standard.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

5.1.2 Standard Tags

All schemas for the MDBLOCKs adhere to the following standard structure:

<TX>	Usually a TXBLOCK can be used instead of the MDBLOCK. The value of this tag has the same meaning as the string in the TXBLOCK would have. The meaning is thus dependent on the field which references the MDBLOCK. Other than the TXBLOCK the TX tag may also contain text with markup (see 5.1.4 Embedding of Markup).
… Schema specific …	Depending on the schema there may be 0 or more specific tags.
<common_properties>	The common properties are used to store user defined information which is tool independent. The properties are stored as key - value pairs which are organized in a hierarchy.
<extensions>	Tool specific data can be written into the extensions section using a tool specific XML schema and namespace.

An MDBLOCK referenced from the CNBLOCK field cn_md_comment may look like in the following example.

Example

```
<CNcomment>
  <!-- required substitute for TXBLOCK -->
  <TX>Channel of great importance.</TX>

  <!-- channel comment specific tags -->
  <names>
    <display>RPM</display>
  </names>
  <raster min="1" max="2">1.5</raster>

  <!-- optional general tags -->
  <common_properties>
    <e name="blub" ci="1" desc="Blubber factor">17</e>
    <tree name="user" desc="user doing the recording">
      <e name="first name">John</e>
      <e name="last name">Smith</e>
    </tree>
  </common_properties>

  <extensions>
    <extension ci="2">
      <vendor_tag xmlns="http://vendor2/my-namespace">
        <something/>
      </vendor_tag>
    </extension>
  </extensions>
</CNcomment>
```

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

5.1.3 Standard Attributes

Some attributes are used universally throughout the schemas:

Table 79 Attributes for tags in MDBLOCK

Attribute name	Description
ci	Identification of the program that has written this element. The value is an index into the list of FHBLOCKs. The first FHBLOCK is index 0. Default value: 0.
xml:lang	Language of element content as defined in XML standard. Examples: en-US = English as used in the united states en = Generic English Default value: "" (unknown)

See the schema for which tag supports which attribute.

Example

```
<CNcomment>
  <TX ci="1"/>
  <common_properties>
    <e name="reason" ci="1" xml:lang="en">Therefore!</e>
  </common_properties>
</CNcomment>
```

In addition to the attributes mentioned above, each tag supports any number of custom attributes. Custom attributes must be in a different namespace to avoid name clashes with further versions of MDF.

Note: This mechanism is only recommended if the <extension> tag can not be used (see 5.3 Vendor Extensions).

Example

```
<CNcomment xmlns:v="http://vendor1/my-namespace">
  <TX v:vendor_attr="special">Comment.</TX>
</CNcomment>
```

5.1.4 Embedding of Markup

Some standards allow markup to be used in comment and description fields. Texts with markup can be embedded as long as the markup is valid XML and has its own namespace.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Example

```
<CNcomment xmlns:xh="http://www.w3.org/1999/xhtml">
  <TX>
    <xh:p>
      My <xh:em>italic</xh:em> paragraph.
    </xh:p>
  </TX>
</CNcomment>
```

Notes:

- Embedding of markup is only recommended if the source of the comment or description contains markup.
- MDF tools that do not understand the markup language may ignore the markup tags and display the remaining text.
- Even though the schema allows any namespace only HTML like markup schemes are allowed. For a general extension mechanism see 5.3 Vendor Extensions.

5.2 Common Properties

5.2.1 <e> Tag

The <e> tag is a tag that can be used to add custom information fields to the XML string without need to specify a separate schema. It is recommend to use the <e> tag if the added information should be readable (and possibly be edited) by other applications, too.

Each <e> tag can contain plain text or embedded markup (see 5.1.4 Embedding of Markup).

If <e> tag stores a numerical value (type "decimal", "integer" or "float"), optionally a unit can be specified by using one of the attributes "unit" or "unit_ref".

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table 80 Additional attributes for <e> tag

Attribute name	Description
name	Name of the property. Used to identify the property within the containing hierarchy (<common_properties> or <tree>). Mandatory attribute.
desc	Description text that can be used to explain the meaning of element content. Optional attribute, no default.
unit	string to be displayed as unit Optional attribute, no default.
unit_ref	Reference to an HDO unit. String represents the ID of an HDO <UNIT> tag globally defined in <ho:UNIT-SPEC> in MDBLOCK referenced by hd_md_comment. Optional attribute, no default.
type	Data type of element content. This can have one of the following pre-defined values: "string", "decimal", "integer", "float", "boolean", "date", "time", "dateTime". The types are a subset of the primitive data types available in XML, see [22] for the possible content for each type. They can be used to limit the values that can be entered by the user or to choose an appropriate way of displaying the information (e.g. calendar for a date). Optional attribute, default value: "string"
ro	Read only flag. If flag is "true" the value may only be displayed to the user but may not be edited. Optional attribute, default value: "false"

Example

```
...
<common_properties>
  <e name="firstName">John</e>
  <e name="lastName">Smith</e>
  <e name="creation date" type="date" ro="true">2000-04-01</e>
  <e name="tel" ci="1" desc="phone number">01234</e>
  <e name="weight" unit="Stone" type="float">14.3</e>
  <e name="length" unit_ref="unitSiBase_meter" type="float">1.93</e>
</common_properties>
...
```

5.2.2 <tree> Tag

The properties specified with the <e> tag can be arranged in a hierarchical way by using the <tree> tag. The <tree> tag is identified by its name attribute and optionally a descr attribute (same as the <e> tag). The <tree> tag can contain further child <e>, <tree>, <elist> and <list> tags. The names of the child tags each must be unique within the immediately containing tag but can be repeated in other parts of the hierarchy.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Example

```
...
<common_properties>
  <tree name="RecordingUser">
    <e name="first name">John</e>
    <e name="last name">Little</e>
  </tree>
  <tree ci="1" name="PostprocessingUser">
    <e ci="1" name="first name">Robin</e>
    <e ci="1" name="last name">Hood</e>
    <tree ci="2" name="address">
      <e ci="2" name="place">Sherwood Forrest</e>
      <e ci="2" name="county">Nottinghamshire</e>
    </tree>
  </tree>
</common_properties>
...
```

5.2.3 <list> Tag

Instead of a hierarchical way, the information may also be arranged as sorted list using a <list> and contained <li> tags. The <list> tag is identified by its name attribute and optionally a descr attribute (same as the <tree> or <e> tag). It only contains <li> tags for the list items. The sort order of the <li> tags is given by the ordering in XML.

Each <li> tag can contain child <e>, <tree>, <elist> and <list> tags. The names of the child tags each must be unique within the immediately containing tag but can be repeated in other parts of the hierarchy.

Note that the list items in a list not necessarily contain the same properties. For a list with simple list items of the same type, consider using an <elist> tag.

Example

```
...
<common_properties>
  <list name="Merry men">
    <li>
      <e name="first name">John</e>
      <e name="last name">Little</e>
      <e ci="1" name="height" unit="feet" type="float">6.9</e>
    </li>
    <li ci="1">
      <e ci="1" name="last name">Tuck</e>
      <e ci="1" name="title">Friar</e>
      <e ci="2" name="weight" unit="stone" type="float">18.9</e>
    </li>
  </list>
</common_properties>
```

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

5.2.4 <elist> Tag

In contrast to the <list> tag, the <elist> tag only can contain simple elements of the same type. An <elist> tag is identified by its name attribute and can have the same attributes as an <e> tag. It only contains <eli> tags for the list items. The sort order of the <eli> tags is given by the ordering in XML.

Each <eli> tag can have the same content as an <e> tag.

Example

```
...
<common_properties>
  <elist ci="1" name="Outlaws">
    <eli ci="1">Robin Hood</eli>
    <eli ci="1">John Little</eli>
    <eli ci="2">Friar Tuck</eli>
  </elist>
  <elist name="score" unit="points" type="integer">
    <eli>1</eli>
    <eli ci="1">1</eli>
    <eli>2</eli>
    <eli>4</eli>
  </elist>
</common_properties>
```

5.3 Vendor Extensions

The recommended way of embedding vendor specific information is using the <extensions> tag. The <extensions> tag contains a list of <extension> tags. Each tool will write its own <extension> tag. The writing tool and vendor is identified by the ci attribute.
The information within the <extension> tag is tool specific. It may be any well-formed XML with namespaces different from the MDF XML namespace. A reading tool should check if it knows the namespace to determine if it can process the information.

Example

```
...
<extensions>
  <extension xmlns:v="http://vendor3/my-namespace" ci="3">
    <v:vendor_tag>
      <v:something/>
    </v:vendor_tag>
    <v:vendor_tag2/>
  </extension>
  <extension xmlns:v="http://vendor2/my-namespace" ci="2">
    <v:vendor2_tag>
      <v:something/>
    </v:vendor2_tag>
  </extension>
  <extension ci="2">
    <vendor_tag xmlns="http://vendor2/my-namespace">
      <something/>
    </vendor_tag>
  </extension>
  <extension ci="1" xmlns:v="http://vendor1/my-namespace" v:vendor_attr="test"/>
</extensions>
...
```

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

5.4 Alternative Names

Names are used throughout the MDF standard to identify items to the user. Most of the names come from sources that are covered by other standards like MCD-2 MC or FIBEX. These standards provide several names for the same object. All named objects can usually also be described. MDF uses the following pattern to store multiple names:

- The main name is stored as a TXBLOCK and can be used to identify objects. This name is mandatory.
- Alternative names are stored as part of the comment MDBLOCK and can be used for display purposes. These names are optional.
- Descriptions are also stored as part of the comment MDBLOCK. They are optional.

Usually the alternative names are stored in the <names> tag (for exceptions see the 4.15 SIBLOCK). Within the <names> tag any number of names and descriptions can be specified using the following tags:

<name>	Translated alternative name, intended to be unique. See Naming Rules for allowed characters.
<display>	Display name, not required to be unique. See Naming Rules for allowed characters.
<vendor>	Tool specific internal names (tool identified by standard ci attribute). No limits on contents.
<description>	Description of the named object. May contain embedded markup, see 5.1.4 Embedding of Markup.

The xml:lang can be used to specify the language. Each tag may appear either once without a language or several times with unique languages specified.

5.5 Formula Specifications

A formula can be specified in the following cases

- algebraic conversion (CCBLOCK)
- dependent (i.e. calculated) channel (CNBLOCK)
- event condition (EVBLOCK)

There are many different ways to specify a formula depending on the case and the tool. To allow free exchange between tools the formula is always specified in a standardized syntax. For documentation purposes any number of alternate formulas may be specified in custom syntax.

As the standardized syntax MDF uses GES in version 1.0. Future versions of GES may be used by specifying the version, but only if the formula cannot be expressed in version 1.0.

<formula>	Calculation of a dependent channel, an algebraic conversion or a trigger condition in formalized syntax.
<syntax>	Child tag of <formula> to define the calculation rule in GES (use optional attribute "version" to specify GES version; default value: "1.0")

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

<custom_syntax>	Child tag of <formula> to define a custom (e.g. tool specific) syntax (use required attributes "source" and "version" to describe which syntax has been used) Several tags <custom_syntax> with unique syntax (combination of source/version) are allowed!
<variables>	Child tag of <formula> to define the input variables of the condition (optional, not allowed when used for cc_md_comment).
<var>	Child tag of <variables> to define mapping of variable name used in syntax to an MDF channel Attribute "name" contains the name of the variable used in the formula string. Attributes "cn", "cs", "cp", "gn", "gs", "gp" and "idx" are used to identify the channel corresponding to the variable in the formula (see 4.4.3 Identification of Channels). idx contains a white-space separated list of the indices specifying the channel if it is a component of a (possibly nested) array. The ordering of the indices starts with the first dimension of the outer most array and advances to the last dimension of the inner most array. Note: The channel does not actually need to be stored in the MDF file. Attribute "raw" contains true if the raw value of the channel must be used, otherwise the physical value must be used.

Example

This example shows how to store a formula in the <syntax> tag using an MCD-2 MC V1.5.1 compliant syntax. In addition, the formula here is stored in a <custom_syntax> tag preserving the old MCD-2 MC ("ASAP2") V1.5.1 syntax. The two variables X1 and X2 correspond to channels where X1 is the component of an array. X1 uses the physical values and X2 the raw values of the respective channel.

```
...
<formula>
  <syntax version="1.0">asin(pow(X1,2))*log(X2)</syntax>
  <custom_syntax source="ASAP2" version="1.5">
    arcsin(X1^2)*ln(X2)
  </custom_syntax>
  <variables>
    <var cn="rpm" cs="src1" cp="plupla" gn="10ms" gs="src1" gp="plupla"
         idx="2 3">X1</var>
    <var cn="rpm2" raw="true">X2</var>
  </variables>
</formula>
...
```

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

6 Terms and Definitions

For the purposes of this ASAM standard the following terms and definitions apply (in alphabetical order).

Block Basic structure principle of MF4 files. A block consists of a number of contiguous bytes.

MF4 File Binary file that contains the storage measurement data and raw messages of bus traffic. Metadata, which are necessary for interpreting the raw data are also contained.

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

7 Symbols and Abbreviated Terms

ATBLOCK	Attachment block
ASAM	Association for Standardisation of Automation and Measuring Systems
BE	Big Endian (Motorola) Byte order (high Byte first)
CABLOCK	Channel Array block
CAN	Controller Area Network
CCBLOCK	Channel Conversion block
CGBLOCK	Channel Group block
CHBLOCK	Channel Hierarchy block
CNBLOCK	Channel block
DGBLOCK	Data Group block
DIBLOCK	Data Invalidation block
DLBLOCK	Data List block
DTBLOCK	Data block
DVBLOCK	Data Values block
DZBLOCK	Data Zipped block
ECU	Electronic Control Unit
EVBLOCK	Event block
FHBLOCK	File History block
FIBEX	Field Bus Exchange Format [5]
GES	General Expression Syntax [2]
HDBLOCK	Header block
HDO	ASAM Harmonized Data Objects [4]
HLBLOCK	Header of List block
IDBLOCK	Identification block
LDBLOCK	List Data block
LE	Little Endian (Intel) Byte order (low Byte first)
LIN	Local Interconnect Network
LSB	Least Significant Bit
MD5	Message-Digest Algorithm 5
MDBLOCK	Meta Data block

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

MDF	Measurement Data Format
MIME	Multipurpose Internet Mail Extensions
MSB	Most Significant Bit
NIL	NIL pointer (0x0000000000000000)
OEM	Original Equipment Manufacturer
RDBLOCK	Reduction Data block
RIBLOCK	Reduction Invalidation data block
RVBLOCK	Reduction Values block
SBC	Single Byte Character string
SDBLOCK	Signal Data block
SIBLOCK	International System of Units
SRBLOCK	Sample Reduction block
TXBLOCK	Text block
UTC	Universal Time Coordinated
UTF	Unicode Transformation Format
VLSD	Variable length signal data

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

8 Bibliography

[1] ASAM e.V.: ASAM MCD-2 MC (ASAP2/ A2L); 2018

[2] ASAM e.V.: ASAM General Expression; 2017

[3] ASAM e.V.: ASAM Data Types; 2005

[4] ASAM e.V.: ASAM Harmonized Data Objects; 2006

[5] ASAM e.V.: ASAM MCD-2 NET (FIBEX); 2017

[6] ASAM e.V.: ASAM ODS; 2018

[7] ASAM e.V.: ASAM MDF Bus Logging; 2014

[8] ASAM e.V.:ASAM MDF Classification Results; 2014

[9] ASAM e.V. ASAM MDF Measurement Environment; 2012

[10] ASAM e.V.: ASAM MDF Naming of Channels and Channel Groups, 2012

[11] CANopen: Application Layer and Communication Profile; www.can-cia.org

[12] DEFLATE compressed data format specification; http://tools.ietf.org/html/rfc1951

[13] IEEE 754-2008: IEEE Standard for Binary Floating-Point Arithmetic; 2008

[14] ISO/IEC 9834-8:2014: ISO standard for Information technology -- Procedures for the operation of object identifier registration authorities -- Part 8: Generation of universally unique identifiers (UUIDs) and their use in object identifiers

[15] The MD5 Message-Digest Algorithm: http://tools.ietf.org/html/rfc1321; 1992

[16] MD5 Hash Algorithm: www.w3.org/TR/1998/REC-DSig-label/MD5-1_0; 1998

[17] Internet Media Type registration, consistency of use: www.w3.org/2001/tag/2002/0129-mime; 2002

[18] Internet Assigned Numbers Authority (IANA) registered Mime Types: www.iana.org/assignments/media-types/; 2019

[19] The International System of Units (SI): www.bipm.org/en/si/

[20] Universal Time Coordinated (UTC): http://www.bipm.org/en/scientific/tai/time_server.html

[21] Unicode (UTF-8, UTF-16): www.unicode.org

[22] Extensible Markup Language (XML): www.w3.org/xml

[23] XML Schema Part 0: Primer Second Edition: www.w3.org/TR/xmlschema-0

[24] Zip Algoritm: www.zlib.net

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Figure Directory

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Table Directory

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Licensed to NIO Inc.
Downloaded by bing.liu4@nio.com on 18.03.2024

Email: support@asam.net
Web: www.asam.net

© by ASAM e.V., 2019